

Ingegneria della Conoscenza

1 – Ingegneria della Conoscenza

Sistemi Intelligenti

L'**Intelligenza Artificiale** è la disciplina che studia **sintesi** e **analisi** degli **agenti computazionali**, ovvero i sistemi che agiscono in modo *intelligente*.

Ovviamente questi sistemi lavorano in un ambiente, dunque devono essere giudicati **solo** per le loro azioni. La loro intelligenza va misurata secondo una *scala* adeguata.

Degli aspetti importanti dell'*azione intelligente* sono:

- L'appropriatezza rispetto alle circostanze, agli obiettivi, ai limiti;
- La considerazione delle conseguenze a breve/lungo termine delle azioni;
- L'apprendimento dall'esperienza;
- La flessibilità rispetto ai cambiamenti.

Un **agente computazionale** è un agente le cui azioni siano spiegabili in termini di computazione.

Ovviamente, essendo implementato su un sistema *fisico*, ha delle *limitazioni* (memoria e tempo limitati, dominio applicativo).

La disciplina dell'AI può essere vista sotto diverse prospettive:

- Prospettiva **scientifica**: disciplina finalizzata a comprendere i principi del comportamento intelligente, analizzando sistemi sia naturali che artificiali, e formulando teorie sulla costruzione di agenti intelligenti;
- Prospettiva **ingegneristica**: disciplina che studia nuove tecnologie utili a risolvere specifici problemi risolti tipicamente da agenti umani (tramite progettazione e sintesi di **sistemi** intelligenti **basati su conoscenza** (KBS)).

Altri obiettivi dell'AI sono:

- **Intelligence augmentation**: accrescimento dell'intelligenza e della creatività umane;
- **Integrazione** dei compiti: agenti umani e sistemi intelligenti che collaborano alla soluzione di problemi, ovvero la modalità **human-in-the-loop**:
 - Le decisioni prese da una AI possono fornire consigli utili per i compiti degli umani, mentre gli umani possono guidare AI per decisioni da prendere.

Diremo che per l'AI vale la seguente formula:

$$\textit{ragionamento} \approx \textit{computazione}$$

L'interazione di un agente situato in un ambiente necessita di **senso comune**, ovvero un "background" di regole non scritte e conoscenze generali che gli umani danno per scontate. Possiamo definire la **conoscenza** come informazione stabile, infatti essa non è dichiarata esplicitamente.

Anche nelle **organizzazioni** possono essere riscontrate forme di intelligenza, risultando addirittura superiori a quella del singolo agente.

Dimensioni dello Spazio di Progettazione

La **complessità** dei sistemi intelligenti varia e dipende dalle finalità e dai vari contesti in cui si trova.

Le **dimensioni della complessità** nella progettazione:

- Definiscono uno **spazio di progettazione**: diversi sistemi in base alle loro tipologie;
- Forniscono forme di **decomposizione sommaria** dello spazio, suddividendo in "blocchi" o "categorie" il problema;
- Interagiscono fra loro.

Progettazione di Sistemi Intelligenti

In generale, la conoscenza del sistema/dell'agente non corrisponde a quella nella mente del progettista. I casi estremi possibili sono dunque:

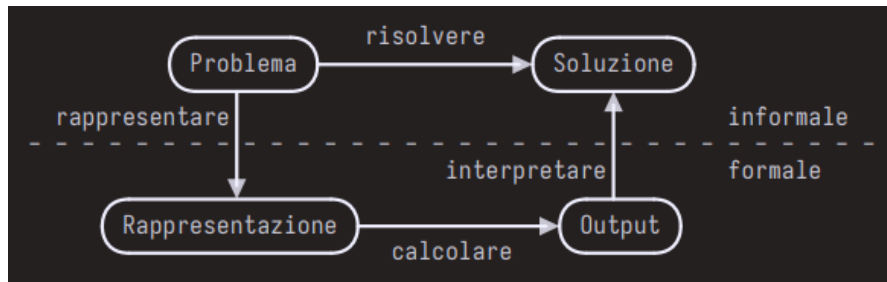
- Sistema **specializzato** nel suo dominio/task;
- Sistema **flessibile** che si adatta ai contesti e accetta nuove task.

Le strategie di costruzione alternative sono:

- Un modello del compito **semplificato**, utile a costruire sistemi di ragionamento **complessi**;
- Sistemi **semplici** per contesti **complessi** (come le auto da guida e i loro sensori).

In AI è importante il rappresentare **COSA** vada fatto e non **COME**.

Una descrizione di un **compito** deve essere disponibile a livello informale, così che, quando sarà il momento di risolverlo, possa essere formalizzato.



Le **credenze**, a differenza della conoscenza, sono più *transitorie*. Per poter ragionare con la conoscenza, occorre rappresentarla formalizzandola in termini di un **linguaggio di rappresentazione**. Una **base di conoscenza** risulterà codificata attraverso delle strutture dati adeguate.

Le **proprietà degli schemi di rappresentazione** sono:

- **Ricchezza espressiva** utile alla risoluzione del problema;
- **Vicinanza** ai termini naturali del problema;
- **Trattabilità**, propensione ad una elaborazione efficiente;
- **Acquisibilità** dagli utenti, da dati e/o esperienza pregressa.

Secondo l'**ipotesi di Newell & Simon**, si suppone che un **sistema di simboli fisico** sia incorporato nella mente umana o nelle macchine. Un **simbolo** è un *pattern* manipolabile come *singola unità*. Un **sistema di simboli** serve a elaborare simboli, tramite azioni tipiche come la creazione, la copia ecc...

Ogni agente intelligente sarebbe necessariamente un sistema di simboli fisico, contenente tutto quello che serve per un comportamento *intelligente*.

Un **modello** è una rappresentazione del mondo che può essere statica (ciò che si crede vero), mentre il suo **funzionamento** è dinamico. Dunque, diventa utile lavorare a vari **livelli di astrazione**. Il livello di astrazione determina un ordine parziale, evidenziando solo gli aspetti rilevanti per quell'analisi.

Le **descrizioni di alto livello** risultano più semplici per gli umani, sia nella loro specifica che nella loro comprensione. Mentre per quelle di *basso livello*, è tutto più accurato e produttivo, infatti si evidenziano i dettagli essenziali per la soluzione del problema.

I livelli d'astrazione comuni fra sistemi *biologici* e *computazionali* sono:

- **Livello di conoscenza** sul mondo esterno: si considera quello che il sistema assume di sapere e i suoi obiettivi, senza pensare a calcolare la soluzione;

- **Livello simbolico** interno: si descrive il modo di ragionare del sistema, utile a implementare il livello precedente, manipolando simboli per produrre risposte.

Sistemi Intelligenti Basati su Conoscenza

Un **agente intelligente** è un modello capace di percezione, ragionamento e azione, immerso in un **ambiente**.

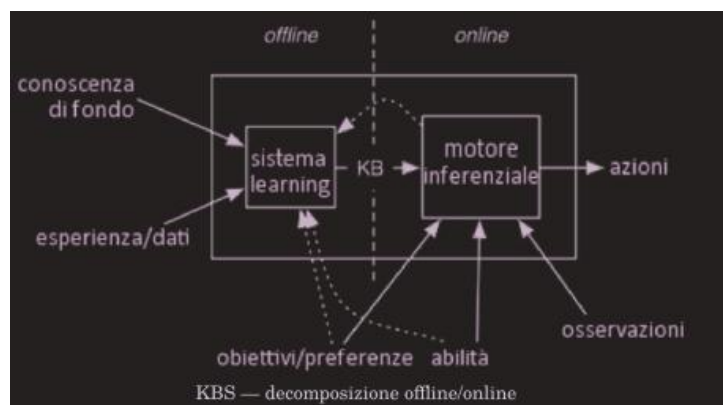


Il comportamento dell'agente/del KBS si basa su:

- **Conoscenza pregressa** su agente e ambiente;
- **Storia** dell'interazione con l'ambiente: **stimoli** ed **esperienza passata**;
- **Obiettivi** da raggiungere o **preferenze** sugli stati dell'ambiente;
- **Abilità**, azioni *primitive* di cui è capace.

Nella scatola nera si rappresenta un suo *stato interno delle credenze*, o **belief state**, comprendente la rappresentazione delle cose che si ritengono vere riguardo l'ambiente, cosa si è imparato, e gli obiettivi intermedi presenti e futuri. Esso viene aggiornato in base agli stimoli e serve per prendere decisioni sulle prossime azioni.

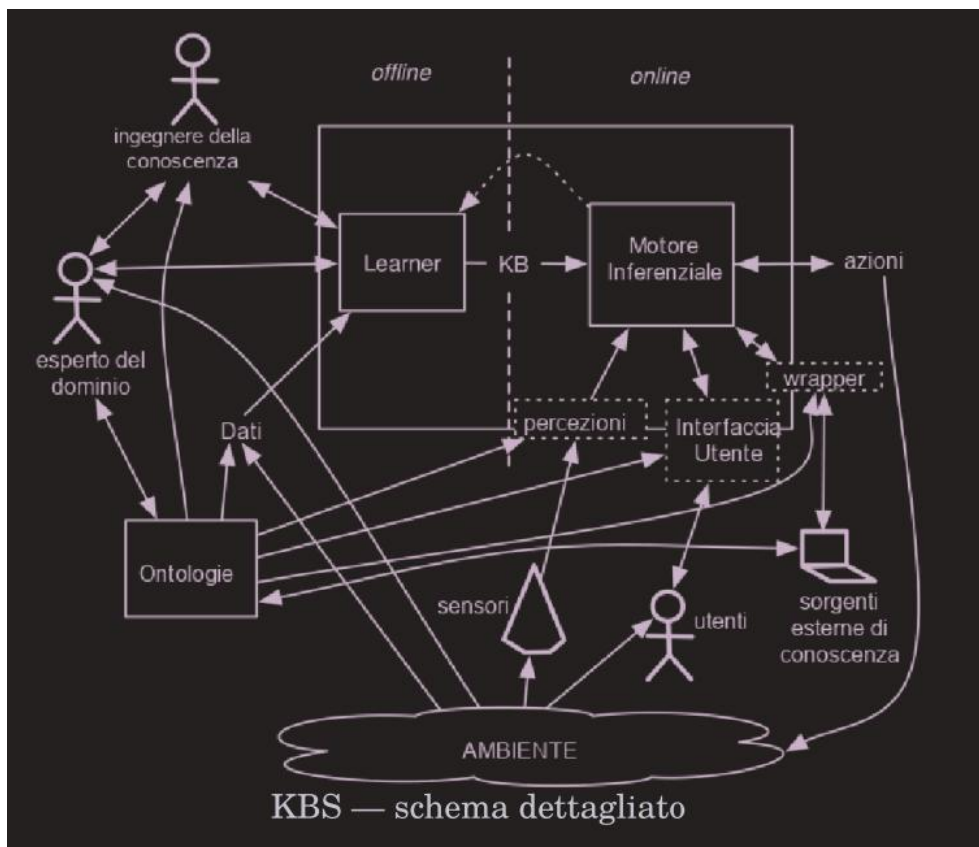
Il funzionamento di un KBS/agente intelligente può essere schematizzato in 2 fasi:



La KB viene coinvolta in momenti diversi:

- **Offline:** viene costruita/integrata (*apprendimento*) usando conoscenza pregressa ed esperienze passate (es. dati). Spesso servono molti dati e conoscenza generale. Questa attività non è automatizzata nei **sistemi esperti**;
- **Online:** si prendono decisioni/compiono azioni e si aggiorna la KB. Nel ragionamento si usano KB, osservazione, obiettivi e abilità del sistema.

Una KB come *long-term memory* è utile nel futuro: viene appresa dai dati e dall'esperienza pregressa; un belief state, *short-term memory*, è un modello dell'ambiente attuale, necessario tra successivi intervalli di tempo.



2 – Ricerca di Soluzioni in Spazi di Stati

Risoluzione di Problemi mediante Ricerca

Spesso risolvere un problema consiste nella **ricerca** in uno specifico modello del mondo.

Astrazione del problema: in un *grafo orientato*, si esegue la ricerca di un **percorso** che vada da un nodo di partenza a uno dei nodi-obiettivo (*goal*). Molti problemi possono essere mappati su questa astrazione per la quale abbiamo diversi algoritmi di ricerca (es. navigatore).

Per un'AI, la strategia che viene utilizzata è la seguente: il sistema lavora su una sua *rappresentazione interna*. Il sistema, infatti, riceve solo una descrizione di **COSA** rappresenti una soluzione; **COME** ottenerla sarà il compito di un algoritmi di ricerca (molto spesso si parla di problemi NP-completi).

Alcuni problemi sono talmente *difficili* che neanche agenti umani possono trovare la soluzione ottima in generale. Quindi si usano soluzioni “**sufficientemente buone**”, specialmente quando il problema non ha schemi chiari da sfruttare (es. *crittografia*).

Spazi di Stati

Traduciamo il problema in termini di **spazio di stati**. Uno **stato** deve contenere tutta l'informazione necessaria a predire gli effetti di un'azione e a verificare se esso soddisfi l'obiettivo (*stato goal*). Una **soluzione** sarà sequenza di azioni che portano dallo stato corrente a un goal (fine-percorso = soluzione).

Le *assunzioni* comuni che si fanno sono la conoscenza *perfetta* dello spazio, ossia che lo stato corrente sia noto e l'ambiente sia **completamente osservabile**. Importante è anche l'esistenza di stati-obiettivo, ossia *goal* da raggiungere facilmente e che siano *riconoscibili*. Inoltre, si assume che, a disposizione del sistema/agente, ci sia un insieme di *azioni* dagli **effetti deterministici** noti.

Un **problema di ricerca** in uno *spazio di stati* comprende:

- Lo *spazio di stati* (insieme di *stati*): uno **stato di partenza** e un **insieme di azioni**;
- La **funzione d'azione**: data una coppia (stato, azione) calcola il nuovo stato in cui si transita tramite l'azione;

- Un *obiettivo*: funzione booleana $goal(s)$ che ha valore vero se e solo se s è uno *stato obiettivo* (si potrebbe anche elencare gli stati-obiettivo se in numero finito);
- Un **criterio** di *qualità* per soluzioni *accettabili*: una **soluzione ottimale** massimizza tale criterio.

Alcune estensioni utili del problema potrebbero essere:

- Possibilità di sfruttamento di una *struttura interna* agli stati;
- Caso di stati *non completamente osservabili*;
- Problemi che ammettono azioni *stocastiche*;
- Invece gli stati finali si possono definire *preferenze aggiuntive* complesse, in termini di *ricompense* o *punizioni*.

Ricerca su Grafo

Un modello *astratto* della soluzione sarà *indipendente* dal particolare dominio applicativo di interesse.

Dato un *grafo (orientato)* fatto di nodi connessi da archi, si dovrà trovare un *percorso* tra un nodo di partenza e uno obiettivo.

Un **grafo orientato esplicito** o anche *implicito*, se si possono generare nodi/archi, consiste di:

- Un insieme di *nodi* N , anche non finito;
- Un insieme di *archi* A , ossia di coppie *ordinate* di nodi, anche non finito;
- Un **percorso** dal nodo s al nodo g denotato da $\langle n_0, n_1, \dots, n_k \rangle$ è una *sequenza* di nodi tale che: $s = n_0, g = n_k$ e $\forall i = 1, \dots, k: \langle n_{i-1}, n_i \rangle \in A$; in alternativa, si può indicare una *sequenza di archi* $\langle n_0, n_1 \rangle, \langle n_1, n_2 \rangle, \dots, \langle n_{k-1}, n_k \rangle$ o anche *sequenza di etichette* su tali archi: $\langle n_0, \dots, n_i \rangle$ *parte iniziale* di $\langle n_0, n_1, \dots, n_k \rangle$, con $i \leq k$.

Nel grafo esiste un insieme dei *nodi-obiettivo* (*goal*) che si possono identificare tramite il predicato $goal(\cdot)$, funzione booleana definita sui nodi. Una **soluzione** è un percorso dal nodo di partenza a uno obiettivo.

In alcuni casi è associato un **costo** a ogni arco: dato $\langle n_i, n_j \rangle$

$$cost(\langle n_i, n_j \rangle) \in [0, +\infty[$$

Estendibile anche ai percorsi: dato $p = \langle n_0, n_1, \dots, n_k \rangle$

$$\text{cost}(p) = \sum_{i=1}^k \text{cost}(\langle n_{i-1}, n_i \rangle) = \text{cost}(\langle n_0, n_1 \rangle) + \dots + \text{cost}(\langle n_{k-1}, n_k \rangle)$$

Una soluzione **ottimale** p avrà costo minimo: $\nexists p'$ soluzione con $\text{cost}(p') < \text{cost}(p)$.

Uno **ciclo** è un percorso non vuoto in cui primo e ultimo nodo coincidono:

$$\langle n_0, n_1, \dots, n_k \rangle \text{ tale che } k > 0 \text{ e } n_0 = n_k$$

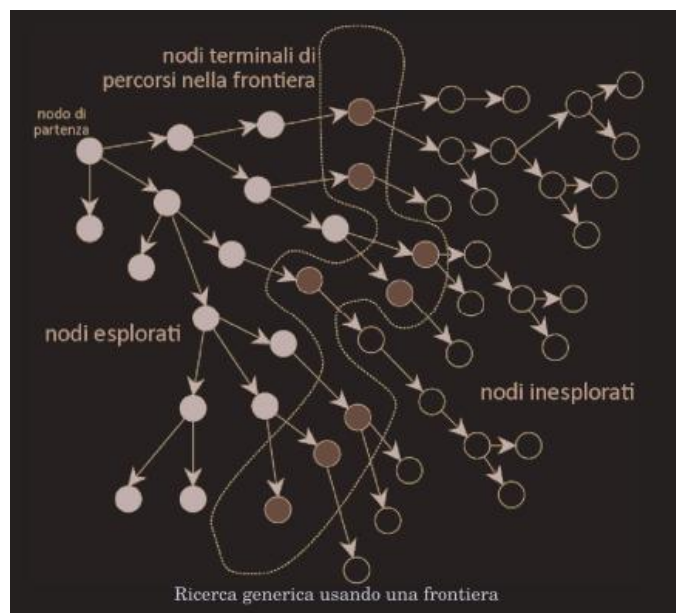
Un **albero** è un DAG (*directed acyclic graph*) con un solo nodo, la **radice** e ha le **foglie**. Si definiscono relazioni fra nodi come padre-figlio, fratello-sorella ecc...

Algoritmo di Ricerca Generico

Dato un grafo, si esplorano *incrementalmente* percorsi dai nodi di partenza verso nodi-obiettivo. Si usa una struttura dati per rappresentare la **frontiera** (*fringe*) dei percorsi già esplorati.

Quello che viene fatto è espandere la conoscenza dei percorsi (oltre a quelli iniziali, di partenza) verso nodi inesplorati, fino ad incontrare un goal:

- Si seleziona un percorso (togliendolo dalla frontiera);
- Si estende il percorso con ogni arco uscente dall'ultimo nodo;
- Si aggiungono alla frontiera i percorsi ottenuti.



Non è altro che la ricerca in ampiezza (BFS)

Strategie di Ricerca Non Informate

Il problema determina il grafo e l'obiettivo, mentre la **strategia di ricerca** specifica il percorso da selezionare dalla frontiera.

Le strategie **non informate** non prendono in considerazione la posizione dell'obiettivo.

Nella **ricerca in ampiezza** (BFS) la frontiera è implementata con una *coda* (FIFO).

Ovviamente la BFS risulta utile quando non si hanno problemi di spazio e stiamo cercando una soluzione avente un numero di archi *minimale*. Il contro è che diventa inutile quando abbiamo una conoscenza euristica, o quando il grafo è generato dinamicamente.

Per la **ricerca in profondità** (DFS), la frontiera è implementata tramite una *pila* (LIFO).

Con una pila, la ricerca avviene in profondità, e quindi avremo:

- Completamento di un singolo percorso prima di provare alternative;
- **Backtracking**: si seleziona una prima alternativa per ogni nodo, tornando indietro alla successiva solo dopo aver tentato tutti i completamenti;
- *Ordine di aggiunta* (ovvero in che ordine esploro) dei vicini alla frontiera/pila non specificato: questo ovviamente impatta sull'efficienza, e può essere *statico* o *dinamico*, in base all'obiettivo.

La DFS è utile in caso di spazio limitato e quando abbiamo molteplici soluzioni.

Diventa inefficiente in caso di possibili percorsi infiniti (grafo infinito o cicli) o in caso di "perdita di tempo" su percorsi lunghi, avendo la soluzione su percorsi corti.

Per rimediare a questi problemi, utilizziamo l'**iterative deepening**, un algoritmo che unisce l'efficienza spaziale del DFS con l'ottimalità del BFS. Quello che verrà fatto sarà **ricalcolare gli elementi della frontiera** del BFS tramite il DFS, che usa meno spazio.

Nella pratica: si sceglie il livello 0 e si fa un DFS, se non trova la soluzione va al livello 1, eseguendo ancora una volta un DFS e così via. Questo garantisce il ritrovamento di una soluzione tramite DFS ma con una profondità limitata ogni volta, in modo tale da non perdere troppo tempo su percorsi inutilmente lunghi.

Quindi il *fallimento* lo avremo in maniera:

- *Innaturale*: raggiunto il limite di profondità, si riparte con il limite+1;
- *Naturale*: l'intero spazio di ricerca è esaurito, non esiste soluzione a nessun livello di profondità.

In caso di archi con costi non unitari (costo $\neq 1$), si cerca la soluzione di *minimo costo totale*.

Gli algoritmi precedenti non garantiscono soluzioni di costo minimo, poiché non viene preso in considerazione il costo stesso.

La **Lowest Cost-First Search (LcFS)** si comporta come BFS ma con la selezione dei percorsi di costo minimo. Viene implementato usando una *coda con priorità* come frontiera ordinata da *cost*.

Il fattore di ramificazione è finito se i costi sono limitati inferiormente da una costante positiva, garantendo la soluzione ottimale, quando esiste.

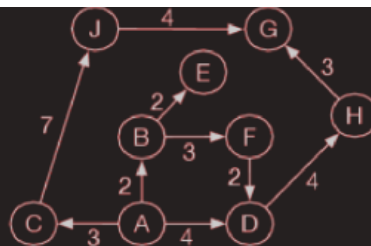
Se manca un limite inferiore allora sono possibili percorsi *infiniti* (esempio di nodi con archi avente costo $\frac{1}{2^i}$, paradosso di Zenone: Achille e la tartaruga).

La complessità è *esponenziale* in spazio e tempo (come per la BFS).

Ricerca Informata (Euristica)

Questa ricerca prende in considerazione *informazioni sull'obiettivo*, quando sceglie i nodi da esplorare, attraverso l'uso di una **funzione euristica** h che associa ad ogni nodo n un numero reale *non-negativo*, stimando il costo minimo di un percorso dal nodo n al nodo-goal.

L'euristica h si dice **ammissibile** se *sottostima* il costo reale: $h(n)$ minore o uguale rispetto al costo minimale effettivo del percorso.



Considerando come *euristica* la distanza in linea retta tra nodo e obiettivo più vicino, quindi supponendo che tali distanze siano ad esempio:

$$\begin{array}{l|l|l} h(A) = 7 & h(B) = 5 & h(C) = 9 \\ h(D) = 6 & h(E) = 3 & h(F) = 5 \\ h(G) = 0 & h(H) = 3 & h(J) = 4 \end{array}$$

h risulta ammissibile e in particolare

- essa è *esatta* per il nodo **H**;
- essa è molto *sottostimata* per il nodo **B**: sembra vicino al goal (costo **5**), ma raggiungere l'obiettivo costa molto di più;
- essa è *ingannevole* per il nodo **E**: sembra vicino al goal (costo **3**) ma non permette di raggiungere l'obiettivo.

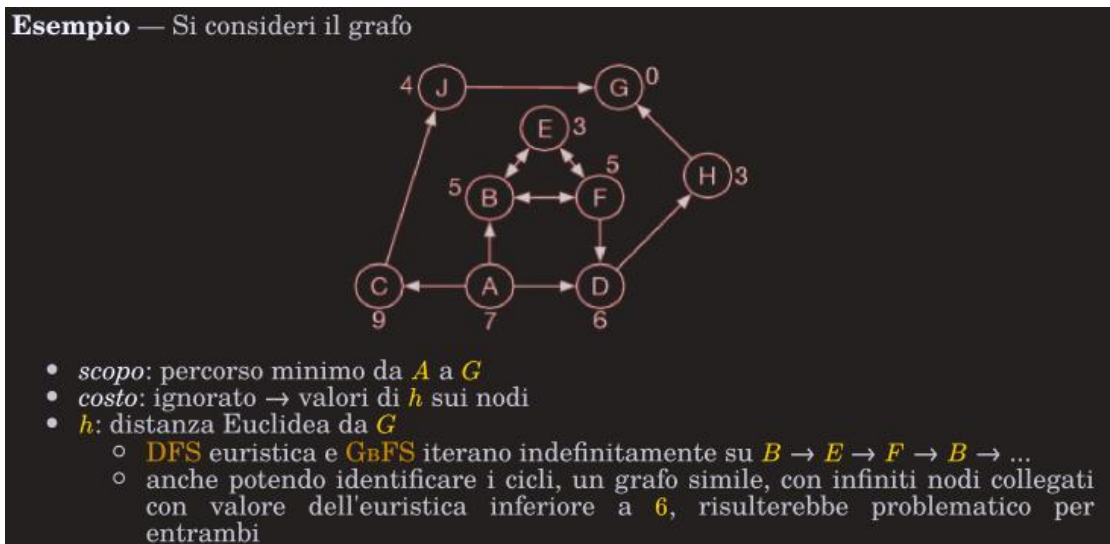
L'euristica h può essere *estesa* al caso dei percorsi (non vuoti):

$$h(< n_0, \dots, n_k >) = h(n_k)$$

Quindi prende in input il percorso che ha già fatto (es. da A fino a C) e restituisce il costo da quel nodo finale (del percorso) fino al nodo g .

La DFS **Euristica** usa h per ordinare i vicini aggiunti alla pila/frontiera della DFS standard. I vicini vengono aggiunti in modo che il *migliore* vada *in cima*. Tale scelta, ovviamente, risulta *locale*, non sapendo cosa ci si aspetterà dopo. Problemi (come per la DFS): non c'è la garanzia di terminare e di trovare una soluzione ottimale.

Nella **Greedy Best-First Search (GbFS)**, si sceglie il percorso della frontiera avente valore di h minimo. Purtroppo, potrebbe seguire cammini apparentemente promettenti che, però, potrebbero continuare a estendersi indefinitamente (colpa della scelta locale).



A* combina le idee degli algoritmi *LcFS* e *GbFS*. Nella selezione del percorso si considerano: il costo effettivo del percorso parziale fino al nodo corrente e l'euristica, ossia una stima del costo fino a un obiettivo.

Per ogni percorso $p = < s, \dots, n >$ della frontiera, si stima il costo di un percorso che passi per p e prosegua fino a un goal g :

$$s \rightarrow n \rightarrow g$$

$$f(p) = \text{cost}(p) + h(p)$$

Nell'algoritmo A* si rappresenta la *frontiera* con una coda con priorità, ordinata da $f(p)$. Se l'euristica $h(n)$ è *ammissibile* allora $f(p)$ non sovrastima il costo reale minimo di un percorso completo che passi attraverso p .

A* migliora le prestazioni degli algoritmi da cui origina (*LcFS* e *GbFS*).

Un algoritmo è **ammissibile** se, quando esistono soluzioni, ne trova sempre una ottimale.

Proposizione – A^* è ammissibile se:

- il fattore di ramificazione è limitato ($\leq b$);
- i costi degli archi sono maggiori di un certo $\varepsilon > 0$;
- h è ammissibile, cioè non supera mai il costo minimo dei percorsi da ciascun nodo a un obiettivo.

Osservazione importante: A^* garantisce che la prima soluzione sarà ottimale anche in presenza di cicli, ma non assicura che ciascun nodo *intermedio* selezionato dalla frontiera sia su un cammino ottimo.

Il miglioramento dell'efficienza, in A^* , è dato dall'uso dell'euristica h . Sia c il costo del percorso di costo minimo. Se h è ammissibile, A^* espande ogni percorso dal nodo di partenza dell'insieme

$$\{p \mid cost(p) + h(p) < c\}$$

Più alcuni percorsi nell'insieme

$$\{p \mid cost(p) + h(p) = c\}$$

Per l'efficienza di A^* , h è migliore se riduce la cardinalità del primo insieme, più grande è più vicini alla soluzione ci si trova.

L'idea dell'algoritmo **Branch and Bound** è quello di combinare l'efficienza in spazio delle DFS con l'informazione euristica. Risulta efficace quando ci sono molti cammini verso un obiettivo, assumendo che $h(n)$ sia ammissibile,

Branch-And-Bound memorizza il percorso di costo minimo verso un goal trovato e il suo costo che diventa un *limite*, *bound*:

- un percorso p tale che $cost(p) + h(p) \geq bound$ può essere scartato;
- si memorizza p completo se $cost(p) < bound$ aggiornando *bound*;
- si continua poi a cercare un'eventuale soluzione migliore.

In questo modo si genera una sequenza di soluzioni via via migliori, meno costose, restituendo quella finale. L'algoritmo viene tipicamente implementato usando la DFS per la ricerca in profondità, limitata dal valore di *bound*.

Per definire un'euristica h (funzione non negativa definita sui nodi, tale che $h(n)$ non superi mai il costo reale del percorso da n al goal) si trovano soluzioni di un problema più semplice, ossia con *meno vincoli*, e quindi *più facile da risolvere*.

Una soluzione ottimale avrà un costo pari o inferiore a una soluzione del problema originario.

Per le euristiche tramite ricerca:

- Un problema in forma semplificata può essere risolto tramite una ricerca più semplice rispetto al problema originario;
- Conviene memorizzare (magari in *cache*) tali risultati in un **DB di pattern (PDB)** che associa ai nodi del problema semplificato il valore dell'euristica;
- Nel problema semplificato, spesso si hanno meno nodi, dove più nodi originari sono mappati su uno *stesso* nodo di quello semplificato, come fosse una *classe d'equivalenza*.

Potatura dello Spazio di Ricerca

Si possono migliorare gli algoritmi considerando i diversi percorsi che possono attraversare un nodo. Non tutti sono utili, quindi possono essere eliminati.

Ci sono due tipi di **pruning** (*potatura*) con relative strategie:

- *Potare i cicli* per trovare percorsi dal costo minimo;
- *Potare cammini molteplici*: per ogni nodo si considererà un solo percorso che lo attraversi, eliminando tutti gli altri.

Nella **potatura dei cicli** (*cycle/loop pruning, CP*), si prevede un controllo aggiuntivo preventivo: dato un nodo da aggiungere, va testata l'occorrenza nel percorso.

Nello schema di algoritmi di ricerca generici si aggiungono alla frontiera solo percorsi $< n_0, \dots, n_k, n >$ tali che $n \notin \{n_0, \dots, n_k\}$, scartando il percorso selezionato; in alternativa si può effettuare il controllo *dopo* la selezione del nodo.

La complessità di questi controlli aggiuntivi dipende dal metodo di ricerca adottato, che può essere:

- *Costante* per metodi che memorizzano un unico percorso (in una lista o in un insieme): si può usare una funzione *hash*, oppure si può associare un *bit* a ogni nodo (flag booleano), che sarà acceso quando viene aggiunto a un percorso e viene *spento* in caso di backtracking;
- *Lineare* nella lunghezza del percorso corrente per metodi che agiscono su più percorsi: serve una ricerca per evitare di aggiungere al percorso parziale un nodo già presente.

Spesso più di un percorso porta ad uno stesso nodo. Occorre quindi *eliminare dalla frontiera* ogni percorso che porti a un nodo per il quale ne esista già un altro (di costo inferiore).

Una strategia di **multiple-path pruning (MPP)** può essere implementata gestendo una lista di nodi terminali di percorsi già esplorati, detta **closed list** o *explored set*: essa è vuota, poi, selezionato un percorso $\langle n_0, \dots, n_k \rangle$ della frontiera, se n_k è già nella lista esso può essere scartato, altrimenti si aggiunge n_k alla lista e si prosegue.

Problema: questa strategia non garantisce che non venga eliminato il percorso di costo minimo.

Per garantire di preservare le soluzioni ottimali ci sono alcune alternative:

- Assicurando che il *primo* percorso trovato per un dato nodo sia ottimale quindi gli altri si possono *eliminare*;
- Se nella frontiera si dovesse inserire un percorso $p = \langle s, \dots, n, \dots, m \rangle$ ma fosse già presente un percorso $p' = \langle s, \dots, n \rangle$ la cui parte fino ad n ha un costo minore, allora si può eliminare p oppure sostituire in p la parte fino a n con p' .

In *LcFS*, il primo percorso verso un dato nodo è quello di costo minimo. Pertanto, potare altri percorsi non elimina un cammino a costo minimo per il nodo. Ciò assicura di poter trovare una soluzione ottimale.

A^* *non garantisce* che quando si seleziona un percorso verso un nodo per la prima volta, questo sia quello di costo minimo: il teorema sull'ammissibilità lo garantisce SOLO per i percorsi verso nodi-goal; per quelli verso altri nodi dipende dalle *proprietà dell'euristica*.

Un'**euristica consistente** h (non negativa) soddisfa il vincolo:

$$h(n) \leq \text{cost}(n, n') + h(n')$$

per ogni coppia di nodi n e n' .

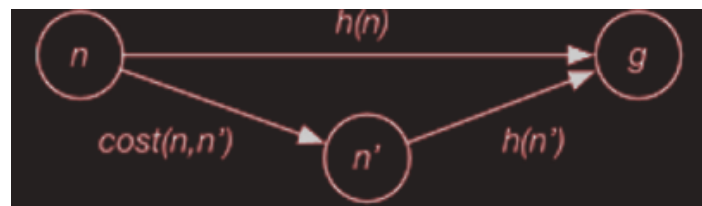
Se $h(g) = 0$ per ogni goal g , allora h sarà consistente e non sovrastimerà mai il costo dei percorsi da un nodo verso un obiettivo.

La consistenza è garantita se h soddisfa la **restrizione di monotonicità**:

$$h(n) \leq \text{cost}(n, n') + h(n')$$

per ogni arco $\langle n, n' \rangle$. Ciò è più facile da testare: dipende solo dagli archi, non dalle coppie di nodi considerate.

Considerato un grafo in cui:



Per la *consistenza*, il costo stimato del percorso da n al goal g non deve essere maggiore della stima di quello che obbliga al passaggio da n' (vale con la distanza euclidea).

La *restrizione monotona* richiede che i valori di f dei percorsi selezionati dalla frontiera siano monotonamente non decrescenti (quindi non torna mai indietro visitando vecchi nodi).

Proposizione – Data un’euristica consistente, MPP non impedisce ad A* di trovare una soluzione ottimale.

Quindi, nelle condizioni della proposizione sull’ammissibilità di A* che garantiscono il ritrovamento della soluzione ottima, se l’euristica è consistente, anche A* con MPP lo potrà trovare. Nella pratica, A* include MPP per default.

Un *consuntivo* sulle strategie di ricerca:

Strategia	Selezione da Frontiera	Soluzione Garantita	Complessità
BFS	primo nodo aggiunto	con meno archi	$O(b^d)$
DFS	ultimo nodo aggiunto	no	$O(b^d)$
ID	n/a	con meno archi	$O(b^d)$
G _B FS	$h(p)$ minimale	no	$O(b^d)$
L _c FS	$cost(p)$ minimale	costo minimo	$O(b^d)$
A*	$cost(p) + h(p)$ minimale	costo minimo	$O(b^d)$
DF B&B	n/a	costo minimo	$O(b^d)$
IDA*	n/a	costo minimo	$O(b^d)$

Un **algoritmo** di ricerca **completo** garantisce la soluzione quando esiste.

Strategie di Ricerca più Sofisticate

Per raffinare le strategie di ricerca conosciute, possiamo pensare ad una ricerca retrograda per soluzioni ottimali da qualunque nodo. Per la ricerca euristica, invece, si potrebbe usare la *programmazione dinamica*: metodi basati sulla *riduzione* del problema in un numero di problemi equivalenti ma di minore complessità.

Nella ricerca in **avanti** si comincia da un nodo di partenza e si prosegue fino a raggiungere i goal.

Nella ricerca all'**indietro** si parte da un goal e si usa il *grafo inverso* alla ricerca del nodo di partenza; i vicini del goal considerati per analizzare la frontiera sono tutti gli altri goal: $\{n : goal(n)\}$.

La direzione di ricerca è *indifferente* quando il numero di goal è *finito* e, per ogni nodo n , si possono generare i vicini nel **grafo inverso** $\{n' : < n', n > \in A\}$.

Le dimensioni dello spazio di ricerca sono date da b^d , dove b è il fattore di ramificazione e d dalla lunghezza del cammino. Riducendo questi parametri aumenta l'*efficienza*.

È possibile che vi sia una differenza tra i fattori di ramificazione uscente ed entrante; in tal caso un *principio generale* per la scelta della direzione della ricerca è quello di considerare b inferiore fra i due.

Nella **ricerca bidirezionale** si riduce il tempo di ricerca, procedendo in entrambe le direzioni: quando le due frontiere *si intersecano*, si ricostruisce un singolo cammino dal nodo di partenza al goal.

Ma come ci si assicura che le due frontiere si incontrino?

Nel caso del *DFS* è *difficile*, date le ridotte dimensioni delle frontiere, mentre per il *BFS* è *garantito*.

Combinando BFS e DFS in direzioni opposte, si garantirebbe l'intersezione, ma quale direzione per ciascun algoritmo? Il *fattore* per tale scelta è per la BFS il *costo del mantenimento della frontiera* (tempo proporzionale a b^k), mentre per la DFS è il *costo della ricerca nella frontiera per trovare un'intersezione*; se si tratta di ricerca simmetrica bidirezionale, si è nell'ordine di $2b^{k/2}$, con risparmi in tempo, ma la complessità rimane esponenziale.

L'idea sarebbe quella di decomporre/astrarre un problema in uno più facile, e diverse sono le modalità per farlo.

La **programmazione dinamica** (PD) fornisce un metodo generale di ottimizzazione basato sulla conservazione di *soluzioni parziali*: una soluzione per un problema più semplice già prodotta (e memorizzata) deve essere solo ritrovata, non ricalcolata.

Si può ricorrere alla PD per la costruzione offline di un'*euristica perfetta* $cost_to_goal()$, ovvero il costo *esatto* di un cammino completo di costo minimo su *grafi finiti*:

$$cost_to_goal(n) = \begin{cases} 0, & \text{se } goal(n) \\ \min_{\langle n, m \rangle} [cost(\langle n, m \rangle) + cost_to_goal(m)], & \text{altrimenti} \end{cases}$$

Dunque, l'algoritmo dice: "Per trovare la strada migliore da qui, provo a fare un passo verso **ogni mio vicino** e scelgo quello che mi offre la somma minore tra '**passo appena fatto**' + '**strada rimanente**'".

Calcolato usando LcFS + MPP sul *grafo inverso* a partire dai nodi-obiettivo: si cerca il cammino di costo minimo da ogni nodo verso i goal e si conserva il valore di *cost_to_goal* per ogni nodo.

Una **policy** è una specifica dell'arco da considerare per ogni nodo; essa si definisce **ottimale** se i suoi costi non sono mai superiori ai costo calcolati con altre policy:

- *cost_to_goal* viene calcolata *offline* e usata nella costruzione di una policy: da *n* si dovrebbe andare al vicino *m* che minimizza $cost(\langle n, m \rangle) + cost_to_goal(m)$; ma questa policy porta sempre ad un obiettivo con un cammino di costo minimo partendo da qualsiasi nodo;
- Alternative: 1) memorizzare il vicino per tutti i nodi *offline*, sfruttando le associazioni tra nodi nelle decisioni online sulle azioni oppure 2) fornire *cost_to_goal* pre-calcolata e calcolare i vicini online.

La PD può servire a costruire euristiche per A* e Branch-And-Bound, semplificando il problema fino ad ottenere spazi di ricerca ridotti e trovando in tali spazi soluzioni di lunghezza ottimale (PDB).

Inoltre, la PD è utile quando i nodi obiettivo sono elencabili *esplicitamente* (senza usare *goal()*), la soluzione cercata è un cammino di *costo minimo*, il grafo ridotto è *finito*, con memoria sufficiente per contenere la tabella, l'obiettivo non cambia.

La policy può essere riusata per i diversi goal: si ammortizza il costo per produrre la tabella su diverse istanze dello stesso problema; tuttavia, essa va calcolata per ogni diverso goal.

3 – Ragionamento con Vincoli

Variabili e Vincoli

Una **feature** viene descritta attraverso un insieme di variabili, spesso non indipendenti tra loro, e da **vincoli rigidi** che specificano le combinazioni lecite di assegnazioni alle variabili e/o da **vincoli flessibili** che esprimono preferenze sulle assegnazioni.

Il *ragionamento* si svolge generando assegnazioni che soddisfino i vincoli rigidi e ottimizzino i vincoli flessibili.

Considereremo *problemi* descritti in termini di **variabili algebriche** ossia simboli usati per denotare caratteristiche del mondo: *mondi possibili*.

La *notazione* adotta nomi che iniziano per maiuscola, ad es. X : ogni variabile ha un **dominio** associato, ossia un insieme di valori che può assumere, denotato $dom(X)$.

Si considereranno *variabili discrete* con dominio finito o almeno enumerabile, ad es. variabili *binarie*, ossia con un dominio di 2 valori, come le variabili *booleane*, che hanno un dominio $\{true, false\}$. Un altro tipo è quello delle *variabili continue*, cioè non discrete. Un altro esempio: variabili con dominio $\mathbb{R}$, o un suo intervallo come $[0,1]$.

Un'**assegnazione** è una funzione da un insieme di variabili ai loro domini: dato $\{X_1, X_2, \dots, X_k\}$ a X_i , si assegna $v_i \in dom(X_i)$ per ogni $i = 1, \dots, k$:

$$\{X_1 = v_1, X_2 = v_2, \dots, X_k = v_k\}$$

Essendo una *funzione*, ad ogni variabile verrà assegnato un solo valore.

Un'**assegnazione totale** riguarda tutte le variabile, altrimenti verrà detta **parziale**.

Un'assegnazione totale rappresenta uno *stato* del mondo, detto *mondo possibile*.

Date n variabili, con domini di cardinalità d , si hanno d^n assegnazioni totali.

L'uso delle variabili offre un *vantaggio*: con poche variabili si descrivono molti stati:

- Con 10 variabili binarie: $2^{10} \approx 10^3$ stati;
- Con 20 variabili binarie: $2^{20} \approx 10^6$ stati;
- Con 30 variabili binarie: $2^{30} \approx 10^9$ stati;
- Con 100 variabili binarie: $2^{100} \approx 10^{30}$ stati;

Ragionare con 30 variabili è, ovviamente, più facile che con un miliardo di *stati*, lo stesso per 100 variabili. Tuttavia, molti *problemi reali* possono essere definiti solo in termini di migliaia, o milioni, di variabili (es. le previsioni del meteo).

Dato un problema, le assegnazioni possono essere *ammissibili* o *non ammissibili*.

Un **vincolo rigido**/*hard constraint* specifica le assegnazioni lecite per una o più variabili e comprende:

- Un **ambito** (o *scope*), ossia l'insieme S di variabili *coinvolte*, con una sua *arietà* (cardinalità) $|S|$;
- Una **condizione**, ovvero una funzione booleana sulle assegnazioni alle variabili del vincolo che dovrà risultare vera solo per assegnazioni *lecite*.

Esempi di vincoli di diverse arietà:

- $B \leq 3$ *unario*;
- $A \leq B$ *binario*;
- $A + B = C$ *ternario*.

La *definizione* di un vincolo può essere:

- **Intensionale**, ossia tramite *formule logiche*;
- **Estensionale**, ovvero tramite elencazione delle assegnazioni lecite, come con le *relazioni* ovvero le *tabelle* di tuple nei DB relazionali.

Dati il vincolo c con ambito S e l'assegnazione A su variabili contenute in S , anche non tutte, si dirà che A **soddisfa** c se la condizione è *vera* per A ristretta all'ambito di c (ovvero che A **viola** c in caso contrario).

Un **problema di soddisfacimento di vincoli** (CSP) è definito da un insieme di *variabili*, ognuna con un proprio *dominio*, e un insieme di *vincoli*. Un sua **soluzione** è un'assegnazione totale che soddisfa tutti i vincoli.

Un *CSP finito* ha un numero finito di variabili, di dominio finito.

Dunque, i problemi legati ai CSP riguarderanno:

- Determinare se esista una soluzione o meno;
- Trovare una soluzione;
- Contare il numero di soluzioni;
- Trovare la soluzione migliore (data una misura di qualità);
- Determinare se alcuni enunciato siano veri per tutte le soluzioni.

Il compito-base è quello di *trovare una soluzione*.

Risoluzione di CSP tramite Ricerca

Un algoritmo **Generate-and-Test** è un algoritmo *esaustivo* per CSP finiti (è semplice, ma inefficiente). Per trovare *una* soluzione:

- Si generano e controllano, in modo sistematico, una alla volta, le possibili assegnazioni totali;
- Si restituisce la prima che soddisfa *tutti* i vincoli.

Per trovarle tutte, si deve continuare ad iterare, conservando tutte quelle trovate.

Con n domini di cardinalità d si hanno d^n possibili assegnazioni totali, e con e vincoli il numero totale di test è $O(ed^n)$. Al crescere di n , la situazione è intrattabile, occorre trovare alternative.

L'idea è quella di testare i vincoli su assegnazioni *parziali* di ambito limitato crescente: se un'assegnazione parziale viola un vincolo, allora anche le assegnazioni totali lo faranno, estendendola. Dunque, è possibile potare lo spazio di ricerca.

Utilizziamo la DFS per cercare tutte le soluzioni di un CSP con variabili Vs e vincoli Cs , estendendo via via l'assegnazione *context*, dove:

- Vs l'insieme delle variabili senza assegnazione in *context*;
- Cs l'insieme dei vincoli che coinvolgono almeno una variabile in Vs ;
- Chiamata iniziale: $\text{DFS_solver}(Vs, Cs, \emptyset)$ con Vs insieme con tutte le variabili e Cs insieme con tutti i vincoli.

Algoritmi Basati su Consistenza

Per **consistenza** si intende la compatibilità fra assegnazioni e vincoli.

Una **rete di vincoli** (*constraint network*) indotta da un CPS è costituita da:

- Un nodo (*circolare*) per ogni variabile, con un dizionario dom di possibili valori per ogni variabile X , denotati con $dom[X]$, inizialmente impostato al suo dominio;
- Un nodo (*rettangolare*) per ogni vincolo c ;
- Un arco $\langle X, c \rangle$ per ogni variabile X nell'ambito del vincolo c .

L'arco $\langle X, c \rangle$ è **consistente rispetto ai domini** (*domain consistent*) se e solo se $\forall x \in dom[X] : \{X = x\}$ soddisfa c .

Dato il vincolo c su $\{X, Y_1, \dots, Y_k\}$, l'arco $\langle X, c \rangle$ è **consistente** se e solo se $\forall x \in \text{dom}[X], \exists y_1, \dots, y_k: y_i \in \text{dom}[Y_i]$ tale che $\{X = x, Y_1 = y_1, \dots, Y_k = y_k\}$ soddisfi c .

Una **rete consistente** (rispetto agli archi) contiene solo archi consistenti.

Vogliamo dunque rendere la rete consistente restringendo i domini.

Si considera l'insieme to_do degli archi potenzialmente non consistenti:

- Si inizializza to_do con tutti gli archi del grafo;
- Si ripete fino a svuotare to_do :
 - Estratto un arco $\langle X, c \rangle$ da to_do
Se $\langle X, c \rangle$ non è consistente, si deve restringere $\text{dom}[X]$;
 - Aggiungere a to_do gli archi resi non consistenti dal passo precedente:
 $\langle Z, c' \rangle, c' \neq c$, con ambito che comprende X e una diversa Z .

Questo algoritmo è il **Generalized Arc Consistency** (GAC).

GAC termina con una rete consistente con domini *ridotti* e 3 casi possibili:

1. Un dominio è vuoto, quindi *nessuna soluzione*: se uno è vuoto, lo saranno anche altri domini ridotti adesso connessi, già prima della terminazione;
2. I domini sono tutti ridotti a un solo valore, quindi la *soluzione* è unica;
3. Altrimenti, si ha un CSP *semplificato* a cui applicare altri metodi.

Separazione dei Domini

L'idea dietro agli algoritmi di **Separazione dei Domini** è quella di decomporre il CDP in una serie di casi disgiunti da risolvere separatamente; le soluzioni sono ricostruite riunendo quelle trovate per i diversi casi.

Tramite un algoritmo, integriamo l'approccio basato sulla consistenza, in un algoritmo ricorsivo:

- Si semplifica il CSP in input tramite GAC();
- Se non è risolto direttamente:
 - Si seleziona una variabile, con dominio almeno binario;
 - Si partiziona il dominio ottenendo (2+) problemi semplificati (casi);
 - Si risolvono *ricorsivamente* tali problemi.

Importante: questo algoritmo fornisce *tutte* le soluzioni.

Eliminazione di Variabili

L'**eliminazione di variabili** (*variable elimination*, VE) è una tecnica che semplifica la rete dei vincoli, rimuovendo variabili. L'idea è quella di eliminare le variabili, una alla volta, ottenendo dei CSP semplificati da risolvere e, infine, ricostruire le soluzioni dei CSP più complessi:

- Eliminando X si costruisce un *nuovo vincolo* sulle rimanenti che rifletta gli effetti di X : esso sostituisce tutti i vincoli su X producendo una rete semplificata (CSP ridotto);
- Alla fine, ogni soluzione del CSP ridotto va *estesa* per ottenere una soluzione del CSP comprendente X .

Per eliminare una variabile, invece, occorrerà:

Data X da eliminare:

1. Considerate le relazioni di tutti i vincoli su X , sia $r_X(X, \bar{Y})$ il *join* di tali relazioni *influenza* di X sulle altre
 - a. $\bar{Y}$: ins. delle altre variabili nell'ambito di r_X *vicine* di X nel grafo dei vincoli
2. La *proiezione* di r_X su $\bar{Y}$ sostituisce tutte le relazioni in cui occorre X ;
3. Si ottiene un CSP ridotto, senza X , da risolvere *ricorsivamente*:
 - a. Al *ritorno*, si estendono le tabelle-soluzioni per il CSP ridotto tramite *join* con r_X , per aggiungere la colonna delle assegnazioni a X ;
 - b. *Caso base*: resta una sola variabile $\rightarrow$ soluzione da restituire
 - i. =tabella con i valori del dominio consistenti con i vincoli.

Esempio — Si consideri un CSP su A, B, C di dominio $\{1, 2, 3, 4\}$ e sia B la variabile da eliminare, inclusa nei vincoli: $A < B$ e $B < C$ (altre variabili possibili con i rispettivi vincoli). Per eliminarla, si fa il *join* tra le relazioni dei vincoli su B :

A	B		B	C		A	B	C
1	2		1	2		1	2	3
1	3		1	3		1	2	3
1	4	$\bowtie$	1	4	=	1	2	4
2	3		2	3		1	3	4
2	4		2	4		2	3	4
3	4		3	4				

- la sua proiezione su A e C induce una nuova relazione senza B :

A	C
1	3
1	4
2	4

vincolo che sostituisce tutti quelli su B

- che contiene tutte le info utili alla soluzione del resto della rete;

- con VE poi si risolve il resto della rete semplificata.

Per avere una/tutte le soluzioni a partire dalla soluzione del CSP ridotto: si memorizza la relazione del *join* su A, B, C per estendere la soluzione della rete ridotta includendo B .

VE può essere *combinato* con algoritmi basati su consistenza, da usare per semplificare il problema quando si elimina una variabile. Le tabelle intermedie risulteranno più piccole.

Ricerca Locale

Cosa si fa in caso di spazi molto grandi o addirittura *infiniti*?

Potremmo puntare su metodi *mediamente efficienti* per trovare soluzioni, ma *senza garanzie*, anche quando esistono.

La classe dei metodi di **ricerca locale** comprende molte tecniche, con uno stesso schema-base:

- Si inizia con un'assegnazione totale di un valore a ciascuna variabile;
- Si tenta di migliorare l'assegnazione iterativamente effettuando passi di *miglioramento*, passi *casuali* e *ripartenze* da assegnazioni iniziali differenti.

```

procedure Local_search( $Vs, dom, Cs$ )
  Input
     $Vs$ : insieme di variabili
     $dom$ : funzione che restituisce il dominio di una variabile
     $Cs$ : insieme di vincoli da soddisfare
  Output
    assegnazione totale che soddisfa i vincoli
  Local
     $A$  dizionario di valori indicizzato dalle variabili in  $Vs$ 
  repeat // try
    for each  $X \in Vs$  do
       $A[X] \leftarrow$  valore (casuale) da  $dom(X)$ 
    // walk
    while not stop_walk() and  $A$  non soddisfa  $Cs$  do
      Selezionare  $Y \in Vs$  e un valore  $w \in dom(Y)$ 
       $A[Y] \leftarrow w$ 
    if  $A$  soddisfa  $Cs$  then
      return  $A$ 
  until terminazione
  
```

Ogni iterazione della *repeat* rappresenta un **tentativo** (*try*):

- Con il primo *for each* si ha l'**inizializzazione casuale** di A ;
- Per ogni assegnazione casuale successiva si ha una **ripartenza casuale** (*random restart*), in alternativa ci sarebbero anche congetture più informate basate su euristiche o conoscenza pregressa.

Nel ciclo *while* si effettua una **ricerca locale** (*walk*) nello spazio delle assegnazioni:

- Si seleziona una assegnazione tra i possibili **successori** di A che differiscono per il valore assegnato a una variabile;
- Si ha lo *stop* se una soluzione è stata trovata o si avvera il criterio di *stop_walk()*, ad esempio raggiunto un numero massimo di iterate.

La fermata *non* è *garantita*: può divergere se il CSP non ha soluzione.

Random Sampling è la versione della ricerca locale in cui:

- Stop_walk() risulta sempre *true* quindi il ciclo *while* non viene mai eseguito: si continua indefinitamente a provare assegnazioni casuali che possano soddisfare tutti i vincoli;
- L'algoritmo risulta *completo* ossia garantisce di trovare la soluzione se questa esiste, tuttavia richiede tempo, risultando molto lento;
- La sua efficienza dipende dalle dimensioni dei domini e dal numero di soluzioni esistenti.

Random Walk è la versione della ricerca locale in cui:

- Stop_walk() risulta sempre *false* per cui non si hanno ripartenze casuali: si esce dal ciclo *while* solo se si trova una soluzione e, nel ciclo, si ripete la selezione casuale una variabile e un valore da assegnarle;
- L'algoritmo è *completo*, con passi più veloci rispetto al resampling di tutte le variabili, ma può richiedere più passi in base alla distribuzione delle soluzioni;
- Quando le dimensioni dei domini delle variabili differiscono, si può, in alternativa, selezionare a caso una variabile e poi un valore del suo dominio oppure selezionare casualmente una coppia variabile-valore, il che favorisce la selezione di variabili con dominio più grande.

L'**Iterative Best Improvement** è un metodo di ricerca locale con selezione del *miglior successore* (un'assegnazione) in termini di una **funzione di valutazione**. Essa sarà da minimizzare, nel caso di **Greedy Descent** oppure da massimizzare, nel caso di **Greedy Ascent** (detto anche Hill Climbing).

Nei casi di *parità* di valore (*tie*) della funzione, si opera una scelta casuale. Una funzione di valutazione tipica per i CSP è il numero di **conflitti**, cioè i vincoli violati: con 0 conflitti si ha una soluzione. Tale funzione può essere raffinata *pesando* i vincoli in maniera differenziata.

Si distinguono:

- **Ottimi locali**: ossia assegnazioni non migliorabili da alcun successore, come i *minimi* o *massimo locali* trovati, rispettivamente, con Greedy Descent o Ascenti;
- **Ottimi globali**: quelli con valutazione massima fra tutte le assegnazioni, che risultano comunque anche ottimi locali.

Ottimizzando il *numero di conflitti*, un CSP è *soddisfacibile* se si trova un ottimo globale con valore nullo, ovvero è *non soddisfacibile* con un ottimo globale con valore positivo: se si trova un minimo locale con valore positivo, non è detto che esso sia globale o meno.

Esempio — ancora esempio precedente sulle consegne:

Se **GREEDY DESCENT** inizia da $\{A = 2, B = 2, C = 3, D = 2, E = 1\}$ si hanno **3** conflitti: $A \neq B, B \neq D, C < D$.

Tracciando le assegnazioni (evidenziando il nuovo valore assegnato a ogni passo):

A	B	C	D	E	valutazione
2	2	3	2	1	3
2	4	3	2	1	1
2	4	3	4	1	2
4	4	3	4	1	2
4	2	3	4	1	0

• con **0** conflitti si ha una *soluzione*.

N.B.: L'algoritmo non si ferma se il miglior successore ha lo stesso valore della soluzione, come per i numero di conflitti. Esempio: soluzione attuale = 2 conflitti, soluzione vicina = 2 conflitti, non è migliorata la situazione e quindi si rischierebbe di cascare in un loop, senza trovare mai una soluzione migliore. Dunque, non garantisce di trovare una *soluzione soddisfacente*, per questo si dice **incompleto**.

Per evitare di rimanere bloccati in soluzioni che sembrano buone (minimi locali), ma non sono le migliori possibili (minimo globale), si introduce la casualità.

Le mosse casuali previste saranno:

1. Il **random restart**, in cui i valori sono scelti a caso per tutte le variabili: costituisce una mossa casuale *globale* (ovviamente più costosa) che consente di ripartire da regioni anche completamente diverse dallo spazio;
2. Il **random step**, una mossa casuale *locale* da alternare a passi di ottimizzazione: utilizzata nel Greedy Descent / Greedy Ascent, permette dei passi in direzione opposta, in modo da sfuggire a minimi / massimi locali.

Integrando il *massimo miglioramento iterativo* e le mosse *casuali*, possiamo definire algoritmi per la **ricerca locale stocastica**.

Con spazi aventi diverse caratteristiche in regioni differenti si può pensare di ricorrere a un **algorithm portfolio**.

Varianti della Ricerca Locale

Molte altre varianti sono possibili nella *scelta del successore* e nelle scelte *casuali*.

La **tabu search** è una forma di ricerca locale *con memoria*, che evita la modifica di assegnazioni introdotte di recente. Si memorizzano le variabili modificate negli ultimi t passi (**tenure**) che andranno considerate *non selezionabili*. In tal modo si eviteranno i cicli dopo poche assegnazioni.

Andrà ottimizzato il parametro t . Nella sua implementazione, se è *piccolo* si può considerare una lista delle variabili modificate di recente, altrimenti si memorizza, per ogni variabile, il passo in cui si è avuta la sua ultima modifica.

Nel metodo di ricerca locale di **passo di massimo miglioramento**, si seleziona una *coppia* variabile-valore che porta al *miglioramento* di valutazione *massimale*.

Nella sua implementazione *ingenua*, data l'assegnazione totale corrente, per ogni variabile X e ogni valore $v \in \text{dom}(X)$ diverso da quello corrente, si confronta l'assegnazione con quella in cui si assegna $X = v$.

Si seleziona quindi una delle coppie di *massimo miglioramento*. Importante però, è il fatto che ciò potrebbe portare a dei peggioramenti.

In un'implementazione *alternativa*, si adotta una *coda con priorità* fatta di coppie variabile-valore pesate. Per ogni X e ogni $v \in \text{dom}(X)$ non assegnato in A (assegnazione corrente), in cosa ci sarà una coppia $\langle X, v \rangle$ con peso $w = h(A') - h(A)$. Si considera allora il miglioramento dell'assegnazione A' ottenuta sostituendo $X = v$ rispetto ad A . Questo dipenderà dai valori assegnati ad X e dai suoi vicini nella rete dei vincoli, non da quelli assegnati alle altre variabili.

Ad ogni iterata si seleziona una *coppia-successore* di massimo miglioramento, ossia con *peso minimale*.

L'algoritmo di **scelta a due fasi** prevede la selezione della coppia, articolata in:

1. Selezione della variabile per la riassegnazione;
2. Selezione del valore.

Si gestisce una *coda* con priorità di variabili, con associato un peso pari al *numero dei conflitti* in cui ognuna sia coinvolta. Ad ogni passo:

1. Si seleziona X che partecipa a *più conflitti*;
2. Le si cambia il valore assegnato, scegliendolo fra quelli che minimizzano il numero di conflitti, oppure *casualmente*.

Infine, vanno ricalcolati i pesi per le variabili coinvolte in vincoli il cui soddisfacimento è cambiato.

L'idea-base dell'algoritmi **Any-Conflict** è quella di scegliere, per la modifica, una **variabile conflittuale**, ossia una che partecipa ad uno o più conflitti. Ad ogni iterata: si seleziona casualmente una variabile conflittuale, non necessariamente quella con più conflitti, quindi le si assegna, in alternativa, un valore che *minimizzi* il numero di conflitti, oppure un *valore casuale*.

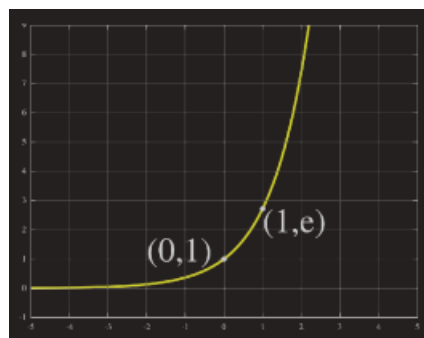
Il **simulated annealing** si comporta così: ad alte temperature deve comportarsi con maggiore casualità o plasticità, mentre a basse temperature deve consentire minore casualità, in conseguenza di una maggiore durezza (preso dalla metallurgia, termodinamica).

Come funzione di valutazione si adotta un'*euristica* basata sul numero di conflitti da minimizzare. L'algoritmo riduce lentamente la *temperatura* a cui è legata una misura di probabilità:

- Ad *alte temperature* si deve comportare come *random walk*, per poter evitare i minimi locali, alla ricerca di regioni con bassi valori dell'*euristica*; quindi, i passi *peggiorativi* risultano più probabili;
- A *basse temperature* si deve comportare come *greedy descent*, portando direttamente verso i minimi (locali).

Ad ogni passo, data l'assegnazione *corrente* A : si sceglie a caso una variabile e un valore, ottenendo una *nuova* assegnazione A' ; se A' non peggiora l'*euristica*, allora andrà a rimpiazzare l'assegnazione corrente, altrimenti può farlo ma con una *probabilità* che dipende dalla temperatura e dal valore dell'*euristica*.

Temperatura $T \in \mathbb{R}^+$



- Sia $h(A)$ l'*euristica* da minimizzare, (tipicamente) il numero dei conflitti legati ad A ;
- Se $h(A') \leq h(A)$ (A' migliorativa), si accetta direttamente la nuova A' , altrimenti (A' peggiorativa), la si può accettare con *probabilità* $e^{-(h(A') - h(A))/T}$ (**distribuzione di Gibbs / Boltzmann**). Si noti che $h(A') > h(A)$, l'esponente è negativo, e tenendo $h(A') - h(A)$ a 0, sarà più probabile accettare A' . Infatti,

ad alte temperature, l'esponente tende a 0 e la probabilità a 1, mentre, a basse temperature, l'esponente tende a $-\infty$ e la probabilità a 0 (ciò dimostra che ad alte temperature si accetta mentre a basse non si accetta quasi mai mosse peggiorative).

Esempio — Probabilità di accettazione di passi peggiorativi a diverse temperature T e differenze $k = h(A') - h(A)$:

T	$k = 1$	$k = 2$	$k = 3$
10	0.9	0.82	0.74
1	0.37	0.14	0.05
0.25	0.018	0.0003	0.000006
0.1	0.00005	$0.2 \cdot 10^{-8}$	$0.9 \cdot 10^{-13}$

Il cosiddetto **programma di annealing** specifica come ridurre la temperatura al progredire della ricerca: molto usato il *raffreddamento geometrico*, ad esempio si parte da $T = 10$ e si moltiplica per 0.99 ad ogni passo, arrivando a 0.07 dopo 500 passi. Avremo dunque 3 situazioni:

- A temperature *alte* (es. $T = 10$) si tende ad accettare passi che peggiorano di poco, con una leggera preferenza rispetto a passi che migliorano;
- A temperature *ridotte* (es. $T = 1$) i passi peggiorativi sono accettati molto meno frequentemente;
- A temperature *basse* (es. $T = 0.1$) i passi peggiorativi sono accettati molto raramente.

L'algoritmo **random restart** (*ripartenza casuale*) permette di migliorare le prestazioni di un *algoritmo casuale debole*, uno che abbia successo in pochi casi specifici.

Nel seguito sarà indicata con p la probabilità di successo di una singola esecuzione. Per stimare le prestazioni di *Random Restart*, si considera una *sequenza* di n sue esecuzioni *indipendenti*. La probabilità di successo in almeno una di tali esecuzioni sarà:

$$1 - (1 - p)^n$$

essendo $(1 - p)^n$ la probabilità di fallimento in tutti gli n tentativi.

Random Restart diventa *costoso* se sono coinvolte molte variabili. Nella sua variante *Partial Restart* si fanno assegnazioni solo ad *alcune* variabili (es. una *percentuale* di esse), per consentire di spostarsi verso un'altra regione. In tal caso le esecuzioni non sono più *indipendenti* fra loro (più complesso).

Algoritmi Basati su Popolazioni

A differenza dagli algoritmi esaminati in precedenza, che considerano un'assegnazione alla volta, tali metodi gestiscono **popolazioni** di **individui**, ossia *insiemi di assegnazioni*. Nella *beam search* si considerano le migliori k , variabile casualmente nella sua variante stocastica.

Beam Search

La **beam search** si comporta in maniera simile agli algoritmi di massimo miglioramento iterativo, con la differenza di lavorare su più assegnazioni (fino a k), anziché su una sola.

Ha *successo* quando viene trovata un'assegnazione soddisfacente. A ogni passo, si selezionano i migliori k *successori* (anche meno in caso non ce ne fossero di meno). La selezione è casuale in caso di parità di valutazione. Si itera con il nuovo insieme di k assegnazioni.

In caso di *memoria limitata* si sceglie k in base alla memoria disponibile. È possibile usare le diverse *varianti* di ricerca locale viste in precedenza. Occorrerà ovviamente più tempo nel cercare i migliori k successori. Per una ricerca più rapida, si può ricorrere ad approssimazioni dei migliori k individui.

In tal caso si selezionano k individui *casualmente*, favorendo quelli con valutazione migliore. La probabilità di selezione sarà definita in funzione dell'euristica: per un individuo A , la probabilità di selezione è proporzionale a:

$$e^{-h(A)/T}$$

(*distribuzione di Gibbs / Boltzmann*) essendo $h(A)$ la funzione di valutazione e una misura della T temperatura, parametro di controllo della casualità, ovvero: "quanto voglio espormi agli errori?". Più T è alta, più l'algoritmo è esplorativo (accetta soluzioni peggiori), altrimenti più è basso più è selettivo.

Questa variante permette di mantenere una maggiore **diversità nella popolazione di soluzioni**. La funzione h rappresenta la capacità di adattamento (*fitness*) di una soluzione, cioè quanto essa è efficace rispetto all'obiettivo da raggiungere.

Le soluzioni con valori di h più elevati hanno una *maggiore probabilità di essere selezionate* per generare nuove soluzioni, che possono essere leggermente modificate da piccoli cambiamenti casuali (mutazioni).

È inoltre possibile che una stessa soluzione venga selezionata più volte.

Algoritmi Genetici

Questi algoritmi operano su un'intera **popolazione di soluzioni** (*individui*). Ad ogni iterazione vengono create nuove soluzioni, combinando coppie di soluzioni già presenti nella popolazione.

L'operazione di combinazione più comune è il **crossover**, che genera una nuova soluzione copiando parte delle assegnazioni da una soluzione, e il resto dall'altra.

In questo modo, l'algoritmo esplora lo spazio delle soluzioni in modo più ampio, cercando progressivamente assegnazioni con valori di h migliori.

Sono possibili diverse tecniche di selezione:

- Nella *selezione proporzionale alla fitness*, ad ogni iterata si *generano* k nuovi individui. Si opera una selezione casuale delle coppie, che favorisca gli individui più adatti, in base al fitness e alla temperatura. Per ogni coppia, si effettua un *crossover* e si fanno *mutare* casualmente alcuni (pochissimi) valori, per alcune variabili scelte a caso (Random Walk – like). Infine, si considera la successiva generazione;
- Nella *selezione a torneo*, si selezionano t individui (misura di *greediness*) e poi si scelgono i più adattabili fra questi.

Ci sono diverse forme di crossover:

- **Crossover uniforme**: si considerano due individui *genitori* e si generano due *figli*; per ogni variabile, si copia nel primo figlio il valore da uno dei genitori scelto in maniera casuale, quello dell'altro sarà copiato nel secondo;
- **One-point crossover**: assumendo un *ordinamento* sulle variabili, si seleziona casualmente un *indice* i per generare ciascun figlio selezionando i valori per le variabili fino a i , da un genitore e per le successive ($> i$) dall'altro (per l'altro figlio, si agisce in maniera *complementare*). L'*efficacia* dipenderà dall'ordinamento scelto in fase di progettazione.

Ottimizzazione

Un **problema di ottimizzazione** richiede di trovare le *migliori* assegnazioni totali. Esso può essere formalizzato come segue:

- *Dati*:
 - Un insieme di *variabili* con un *dominio* associato;

- Una **funzione-obiettivo** dalle assegnazioni totali a $\mathbb{R}$, tipicamente una funzione di costo/errore/loss;
- Un **criterio di ottimalità**, tipicamente *minimizzare* la funzione-obiettivo.
- *Trovare* un'assegnazione totale **ottimale** per il criterio adottato.

Un **problema di ottimizzazione vincolato** prevede anche *vincoli rigidi* che specificano le assegnazioni *ammissibili*, cioè che soddisfino i vincoli (*feasible*). L'*obiettivo* è quello di trovare un'assegnazione ottimale ammissibile.

Rispetto ai CSP, l'ottimizzazione presenta un'ulteriore difficoltà: riconoscere le soluzioni solo *per confronto* fra assegnazioni.

Si possono trattare i vincoli rigidi come quelli flessibili ma assegnando un costo *infinito* in caso di violazione (numero finito = no violazioni). In alternativa, si può assegnare, alla violazione di vincolo rigido, un costo *elevato*.

Si parla di **ottimizzazione discreta** in caso di problema che coinvolge solo variabili *discrete*.

Possono essere adattati i metodi di ricerca di percorsi ottimali su grafo visti in precedenza. In questo caso, il grafo avrà nodi contenenti assegnazioni a un insieme di variabili. I vicini di un nodo n si trovano scegliendo un variabile *var* non presente in n e le relative assegnazioni di valori, consistenti con gli eventuali vincoli rigidi. Il costo di un arco è dato dalla somma dei costi dei vincoli valutati dopo l'assegnazione a *var*.

Il nodo di partenza conterrà l'assegnazione vuota mentre un nodo-goal conterrà un'assegnazione totale (consistente con tutti i vincoli).

I costi sono assegnati solo quando un vincolo soft può essere valutato, inoltre, per usare gli algoritmi A*, Branch-And-Bound, i costi-arco devono essere non-negativi.

L'uso della *ricerca locale* nel caso di problemi di ottimizzazione mira a minimizzare la funzione-obiettivo.

Gli algoritmi di ricerca locale aggiornano via via, restituendo, infine, la *migliore assegnazione trovata*. La terminazione può essere garantita fissando un numero massimo di iterate (o un tempo massimo).

È difficile stabilire se un'assegnazione totale trovata sia ottimale: si ha un **ottimo locale** trovando un'assegnazione non peggiore di tutte le *successive* possibili; un **ottimo globale** è dato da un'assegnazione non peggiore di *tutte* le assegnazioni. Senza la ricerca sistematica non si può sapere se la migliore assegnazione trovata localmente costituisca un ottimo globale, o se ne esiste altrove una migliore.

Nel caso di domini continui la ricerca locale è più complicata: bisogna stabilire come definire il *successore* di un'assegnazione.

L'idea dell'algoritmo di **discesa del gradiente**, *Gradient Descent*, è quella di minimizzare la funzione di valutazione (purché *continua* e *differenziabile*). Vengono fatti solo passi nelle direzioni più rapide. Il *successore* di un'assegnazione viene calcolato facendo un passo in discesa (segno negativo) di lunghezza proporzionale alla *pendenza* della funzione, ossia alla sua *derivata*. Esiste anche una ricerca di massimi di funzioni obiettivo tramite **risalita del gradiente**, *Gradient Ascent*, facendo passi di segno positivo.

Nel caso *monodimensionale*, se a X è assegnato $v \in \mathbb{R}$, il valore successivo sarà:

$$v - \eta * \frac{df}{dX}(v)$$

Dove η è la misura del *passo* che determina la *rapidità* della discesa; la derivata viene valutata in v :

$$\lim_{\epsilon \rightarrow 0} \frac{f(X = v + \epsilon) - f(X = v)}{\epsilon}$$

Nel caso *multidimensionale*, si fanno passi in tutte le direzioni, proporzionali a ogni *derivata parziale*. Date le variabili $\langle X_1, \dots, X_n \rangle$ e un'assegnazione iniziale di valori $\langle v_1, \dots, v_n \rangle$, l'assegnazione *successiva* si ottiene muovendosi lungo ogni direzione, in proporzione alla relativa pendenza di h (funzione di valutazione da minimizzare). Il nuovo valore per X_i si ottiene calcolando:

$$v_i \leftarrow v_i - \eta * \frac{\partial h}{\partial X_i}(\vec{v}) \quad \forall i = 1, \dots, n$$

dove $\frac{\partial h}{\partial X_i}$ è la *derivata parziale* rispetto a X_i (le altre vengono trattate come costanti):

$$\frac{\partial h}{\partial X_i}(\vec{v}) = \lim_{\epsilon \rightarrow 0} \frac{h(v_1, \dots, v_i + \epsilon, \dots, v_n) - h(v_1, \dots, v_i, \dots, v_n)}{\epsilon}$$

4 – Rappresentazione e Ragionamento Proposizionale

Proposizioni

Le **proposizioni** sono *enunciati* che possono essere veri o falsi, espresse in un *linguaggio formale*, descritto in termini di *sintassi* e *semantica*, la **Logica Proposizionale**. Il *ragionamento* potrà essere definito in termini di proposizioni.

Una **proposizione atomica**, o **atomo**, viene rappresentata/o da un simbolo, *convenzionalmente* una stringa alfanumerica, contenente eventuali *_*, che inizi con una lettera *minuscola*. Esempi di atomi leciti sono i seguenti: *ai_divertente*, *lampadina1_accesa*, *piove*, *soleggiato*. Troviamo inoltre i connettivi logici (**formula logica**).

La **semantica** definisce il *significato* delle proposizioni. Un'**interpretazione** è definita come funzione: $\pi: \text{Atomi} \rightarrow \{true, false\}$

- Se $\pi(a) = true$, si dice che *a* **vero** nell'interpretazione π ;
- Se $\pi(a) = false$, si dice che *a* **falso** nell'interpretazione π .

Un'interpretazione si può anche definire come l'*insieme I* di tutti e i soli gli atomi veri, ossia associato al valore di verità *true*. Tutti gli altri atomi saranno interpretati come associati a *false*.

Si noti che la verità di una proposizione dipende *solo* dall'interpretazione dei suoi atomi.

Una **Base di Conoscenza** (*KB, knowledge base*) è costituita da un insieme di proposizioni dette **assiomi**: essi sono dichiarati come veri. La base di conoscenza può essere vista come una congiunzione implicita di assiomi; quindi, sarà vera se tutti sono veri.

Gli assiomi servono a specificare il *mondo* da rappresentare attraverso quella che si dice la sua **interpretazione intensa**, ossia quella che dovrebbe trovarsi nella mente dell'*esperto di dominio* e che va formalizzata dall'*ingegnere della conoscenza* che fornisce la sua *assiomatizzazione*. Un **modello** di una base di conoscenza *KB* è un'interpretazione per la quale sia vero ogni assioma di *KB*.

La proposizione *g* (*goal*) è **conseguenza logica** di *KB*, ovvero *g segue logicamente* da *KB* se e solo se *g* è vera in ogni modello di *KB*, in simboli:

$$KB \models g$$

In inglese questa relazione fra *KB* e *g* si dice *entailment*.

Quindi, se vale $KB \models g$, non esistono modelli di KB nei quali g sia falsa. Si può scrivere $KB \not\models g$ per dire che “ g non è conseguenza logica di KB ”, ciò significa che esiste (almeno) un modello di KB in cui g è falsa. Un caso da distinguere è quello in cui $KB \models \neg g$.

Esempio — Data la KB contenente gli assiomi

- $sam_is_happy.$
- $ai_is_fun.$
- $worms_live_underground.$
- $night_time.$
- $bird_eats_apple.$
- $apple_is_eaten \leftarrow bird_eats_apple.$
- $switch_1_is_up \leftarrow sam_is_in_room \wedge night_time.$

[C]

si può scrivere

- $KB \models bird_eats_apple$
- $KB \models apple_is_eaten$

mentre

- $KB \not\models switch_1_is_up$ ossia esiste un modello di KB in cui $switch_1_is_up$ è falso; tale modello, dovendo risultare vero l'ultimo assioma [C] in KB , può essere definito assumendo che $sam_is_in_room$ sia falso.

Per caratterizzare un dominio/mondo (il suo stato), il/la *progettista* della KB lo definisce come **interpretazione intesa**: il significato dei simboli viene definito attraverso proposizioni, ossia assiomi che si possono assumere veri rispetto a tale stato del mondo; quindi, le conseguenze logiche di KB saranno *vere* rispetto all'*interpretazione intesa*. Il significato viene così *condiviso* e potrà essere usato per interpretare le risposte a interrogazioni alla KB .

Il *progetto* della KB si sviluppa nelle seguenti fasi:

1. Si sceglie il *dominio*/mondo da rappresentare, ossia l'*interpretazione intesa*;
2. Si scelgono gli *atomi* per rappresentare le proposizioni d'interesse, la cosiddetta **concettualizzazione**, a cui associare un significato preciso rispetto all'*interpretazione intesa*;
3. Si definiscono gli *assiomi*, ossia *proposizioni* vere nell'*interpretazione intesa*, l'**assiomatizzazione** del dominio applicativo per il sistema;
4. Si pongono al sistema **domande** sull'*interpretazione intesa*; le sue **risposte** dovranno essere interpretate in base ai significati associati ai simboli.

Il sistema elabora la KB per decidere se g è vera nell'*interpretazione intesa*. I modelli di KB rappresentano tutti e soli i modi in cui il mondo potrebbe essere fatto.

Data una concettualizzazione e un'assiomatizzazione corrette, l'*interpretazione intesa* sarà uno dei modelli di KB , ma il sistema non sa determinarlo esattamente.

Il sistema deve determinare se $KB \models g$: se $KB \models g$, esso potrà concludere che g è vera in tale modello, essendo g vera in tutti i modelli di KB .

Se, invece, $KB \not\models g$, allora non può trarre conclusioni: g potrebbe essere falsa proprio nella interpretazione intesa, ma il sistema non può stabilirlo dato che non la conosce.

Si noti che $KB \not\models g$ non comporta necessariamente $KB \models \neg g$.

Esempio — Data KB dell'esempio precedente:

- l'utente saprebbe interpretare il significato dei simboli;
- la macchina non comprende il significato, ma può trarre conclusioni basandosi su quello che è assertito, ad es.:
 - $KB \models \text{apple_is_eaten}$ quindi apple_is_eaten è vera nell'interpretazione intesa
 - $\text{bird_eats_apple.} \in KB$
 - $\text{apple_is_eaten} \leftarrow \text{bird_eats_apple.} \in KB$
 - $KB \not\models \text{switch_1_is_up}$ è falsa in qualche modello (interpretazione intesa?)
 - $\text{switch_1_is_up} \leftarrow \text{sam_is_in_room} \wedge \text{night_time.} \in KB$
 - $\text{night_time.} \in KB$
 - sam_is_in_room : vero o falso nell'interpretazione intesa?

⚠ Il/la progettista può sbagliare e codificare nella KB assiomi falsi nell'interpretazione intesa. In tali casi non può essere garantito che le risposte della macchina siano vere in tale interpretazione.

Vincoli Proporzionali

Si considereranno ora CSP con vincoli rappresentati come *formule logiche* dalla struttura concisa e formale, per consentire l'elaborazione automatica.

Un *problema* di **soddisfacibilità proposizionale** (SAT) è definito da:

- Un insieme di *variabili booleane*, cioè con dominio $\{true, false\}$; si usa la sintassi $X = true$ abbreviata in x , e $X = false$ oppure $\neg x$ quindi, ad esempio, data la variabile *Happy*, *happy* sta per $Happy = true$ e $\neg happy$ sta per $Happy = false$;
- Un insieme di *vincoli casuali* o **clausole**, ossia espressioni logiche della forma:

$$l_1 \vee l_2 \vee \dots \vee l_k,$$

in cui ogni **letterale** l_i è un atomo a o la sua negazione $\neg a$, ossia un'assegnazione a una variabile booleana; si dirà che a *occorre negativamente* (risp. *Positivamente*) nel letterale $\neg a$ (risp. a).

La clausola $l_1 \vee l_2 \vee \dots \vee l_k$ è vera in un'interpretazione (*mondo possibile*) se e solo se $\exists i \in \{1, \dots, k\}$: l_i vero in tale interpretazione. In tal caso, si dice anche che il vincolo clausale è *soddisfatto*.

Nei CSP, una clausola è un *vincolo* su un insieme di variabili booleane che è *soddisfatto* se almeno uno dei suoi letterali è vero: $l_1 \vee l_2 \vee \dots \vee l_k$ equivale a $\neg(l_1 \wedge l_2 \wedge \dots \wedge l_k)$, ciò esclude le assegnazioni in cui *tutti* i letterali risultino falsi.

Un CSP finito si può trasformare in un problema di soddisfacibilità di una KB proposizionale, in cui:

- Per ogni variabile Y del CSP, con $dom(Y) = \{v_1, \dots, v_k\}$ si considerano le **variabili indicatrici** booleane $\{Y_1, \dots, Y_k\}$ tali che $\forall i = 1, \dots, k: Y_i$ vera se $Y = v_i$ e falsa altrimenti; quindi, per rappresentare Y in KB si considereranno gli atomi $y_1, \dots, y_k$;
- Si aggiungeranno a KB le seguenti *clausole* (vincoli):
 - $\neg y_i \vee \neg y_j$ per ogni $i, j \in \{1, \dots, k\}$ con $i < j$, poiché y_i e y_j non possono essere entrambi veri (si può assegnare un solo valore a Y);
 - $y_1 \vee \dots \vee y_k$, poiché uno degli y_i dev'essere necessariamente vero, ossia Y avrà un certo valore i nell'assegnazione totale;
- Per ogni vincolo del CSP, si definirà una clausola per (negare) ogni *assegnazione* che lo violi:
 - $\neg(l_1 \wedge l_2 \wedge \dots \wedge l_k)$, che equivale a $\neg l_1 \vee \neg l_2 \vee \dots \vee \neg l_k$, cioè i letterali per le *assegnazioni* alle Y_i *non ammesse* dal vincolo;
 - Per semplificare, si possono combinare le clausole; ad esempio, combinando $a \vee b \vee c$ con $a \vee b \vee \neg c$ si ha $a \vee b$.

L'algoritmo DPLL (*Davis-Putnam-Logemann-Loveland*) prevede:

- Riduzione dei domini (*pruning*) e dei vincoli;
- Separazione dei domini;
- Assegnazione di letterali *puri*.

L'algoritmo risulta efficiente con strutture dati indicizzate ad hoc.

Clausole Definite Proposizionali

Il linguaggio delle **clausole definite proposizionali** è un sottolinguaggio della logica proposizionale, con la stessa semantica. Esso ammette una forma più specifica di proposizioni, le clausole con un unico letterale positivo, della forma seguente:

$$a \vee \neg b_1 \vee \dots \vee \neg b_m$$

Le *proposizioni atomiche*, o *atomi*, sono gli stessi della logica proposizionale, Valendo l'equivalenza $A \vee \neg B \equiv A \leftarrow B$, una **clausola definita** proposizionale.

Tale clausola definita si può scrivere come:

$$h \leftarrow a_1 \wedge \dots \wedge a_m.$$

(con h, a_i atomi) formula che si legge “ h se a_1 e ... e a_m ”.

In essa, h si dice **testa** (*head*) e $a_1 \wedge \dots \wedge a_m$ si dice **corpo** della clausola. Una clausola definita è una **regola** se $m > 0$, altrimenti è un **fatto** o una **clausola atomica** quando $m = 0$ (*corpo vuoto*, si può omettere “ $\leftarrow$ ”).

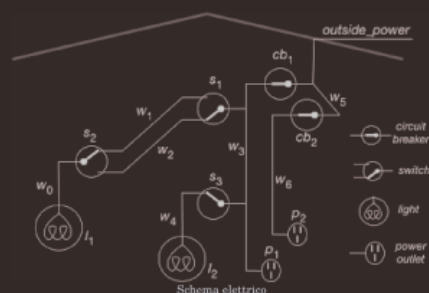
Una **base di conoscenza** può essere specificata come insieme di clausole definite.

La clausola $h \leftarrow a_1 \wedge \dots \wedge a_m$ è **falsa** nell’interpretazione I se $a_1, \dots, a_m$ sono tutti veri in I ma h è falsa, altrimenti essa è **vera** in I (tavola di verità del connettivo ‘ $\leftarrow$ ’).

La clausola definita è una forma **ristretta** di clausola, nella quale compare **esattamente** un letterale positivo.

Es. $a \leftarrow b \wedge c \wedge d$. è equivalente a $a \vee \neg b \vee \neg c \vee \neg d$.

Esempio — Domotica: ambiente intelligente per la simulazione (diagnostica)



Schema elettrico

- si definisce una **rappresentazione** dello stato di cavi (*wire*), prese (*outlet*), luci (*light*), deviatori (*switch*) e fusibili (*circuit breaker*), ... I dettagli trascurabili non verranno modellati (ad es. colori, peso, lunghezze, altezze, ecc.);
- bisogna decidere il **livello di astrazione** per la rappresentazione: per *specialisti* (i voltaggi, le correnti, le frequenze, ...) oppure comprensibile anche a non specialisti (in base al *senso comune*);
- poi si decide **cosa** rappresentare: proposizioni su accensione luci, funzionamento cavi, stato deviatori, ecc.;
- si definiscono **atomi** con significati precisi per il mondo da modellare: nomi descrittivi come *up_s2* per indicare che un deviatore/switch s_2 è in posizione *up*, oppure *live_l1* che indica che il punto-luce l_1 è *sotto tensione*.
- si definisce anche la **conoscenza di fondo** (BK) che fornisce altri fatti veri riguardo il mondo, ed è rappresentata nel sistema attraverso altre clausole definite, spesso in forma semplice, come fatti:
 - *light_l1*.
 - *light_l2*.
 - *ok_l1*.
 - *ok_l2*.
 - *ok_cb1*.
 - *ok_cb2*.
 - *live_outside*.
- si può anche considerare una parte del dominio e definire altre **regole**:
 - *live_l1* $\leftarrow$ *live_w0*. “(punto-luce) l_1 in tensione se (cavo) w_0 in tensione”
 - *live_w0* $\leftarrow$ *live_w1* $\wedge$ *up_s2*.
 - *live_w0* $\leftarrow$ *live_w2* $\wedge$ *down_s2*.
 - *live_w1* $\leftarrow$ *live_w3* $\wedge$ *up_s1*.
 - *live_w2* $\leftarrow$ *live_w3* $\wedge$ *down_s1*.
 - *live_l2* $\leftarrow$ *live_w4*.
 - *live_w4* $\leftarrow$ *live_w3* $\wedge$ *up_s3*.
 - *live_p1* $\leftarrow$ *live_w3*.
 - *live_w3* $\leftarrow$ *live_w5* $\wedge$ *ok_cb1*.
 - *live_p2* $\leftarrow$ *live_w6*.
 - *live_w6* $\leftarrow$ *live_w5* $\wedge$ *ok_cb2*.
 - *live_w5* $\leftarrow$ *live_outside*.
 - *lit_l1* $\leftarrow$ *light_l1* $\wedge$ *live_l1* $\wedge$ *ok_l1*.
 - *lit_l2* $\leftarrow$ *light_l2* $\wedge$ *live_l2* $\wedge$ *ok_l2*.
- a run-time, si forniranno al sistema delle **osservazioni**, ad es. fatti sullo stato dei deviatori, come:
 - *down_s1*.
 - *up_s2*.
 - *up_s3*.

Fatto ciò, si deve determinare la verità di fatti riguardanti il mondo, ossia il dominio rappresentato:

- Un utente può porre *domande* sulle conseguenza logiche della base di conoscenza fornita alla macchina;
- Il sistema è in grado di dare *risposte* decidendo se, date proposizioni, seguano logicamente dalla base di conoscenza o meno;
- L'utente potrà comprendere il *significato* di tali risposte, se conosce la semantica degli atomi utilizzati.

A tale scopo si definisce una **query**, in modo tale da sapere se una proposizione segua logicamente da una base di conoscenza, denotata come *ask b*, dove *b* è un atomo o una congiunzione di atomi, il che corrisponde al *corpo* di una regola:

$\leftarrow b_1 \wedge \dots \wedge b_m.$

Le *risposte* possibili sono:

- *Yes* se *b* è conseguente logica, cioè $KB \models b$;
- *No* altrimenti, cioè $KB \not\models b$; ciò non significa che *b* sia falso nell'interpretazione intesa, bensì che non è possibile determinarlo con la conoscenza disponibile.

La relazione $\models$ definisce le risposte corrette secondo la *model theory*. La **proof theory** si occupa invece di come *calcolare* tali risposte, ossia della dimostrazione delle conseguenza logiche. Si tratta di un problema di *deduzione*, una forma di *inferenza*.

Una **prova** (*proof*) è una *dimostrazione*, derivabile anche *automaticamente*, del fatto che una proposizione segua logicamente da un insieme di assiomi. Una proposizione dimostrabile si dice anche **teorema**.

Una **procedura di dimostrazione** (*proof procedure*) è un algoritmo, eventualmente non-deterministico, utile a costruire prove. Data una procedura di dimostrazione, si denota con:

$$KB \vdash g$$

La possibilità di produrre una prova del fatto che *g* è conseguenza di *KB*.

Le proprietà più importanti delle procedure riguardano le relazioni tra prove e modelli:

- Una procedura è **corretta** (*sound*) se e solo se ogni proposizione dimostrata dalla procedura è conseguenza logica di *KB*:

se $KB \vdash g$ allora $KB \models g$

- Una procedura è **completa** se e solo se ogni conseguenza logica di KB può essere dimostrata dalla procedura:

se $KB \models g$ allora $KB \vdash g$

Esistono due tipi principali di procedure: le *Bottom-Up* (BU) e le *Top-Down* (TD).

L'*obiettivo* di una procedura *bottom-up* è quello di derivare tutte le conseguenze logiche di una base di conoscenza. A tal fine si costruisce la prova gradualmente, basandosi su quanto è stato già dimostrato. Si procede per **concatenazione in avanti** (*forward chaining*) su clausole definite, derivando fatti nuovi a partire da quanto è già stato provato.

Serve una sola *regola di derivazione* che costituisce una generalizzazione della regola d'inferenza detta **modus ponens**:

$$\frac{p \rightarrow q \quad p}{q}$$

Se $h \leftarrow a_1 \wedge \dots \wedge a_m$ è in KB e ogni a_1 è già stato provato, allora si potrà derivare h . Ogni fatto ($m = 0$) viene considerato *immediatamente* provato.

La procedura trova C , l'**insieme delle conseguenze** di KB . La risposta restituita è *yes* ossia $KB \vdash g$ quando $g \in C$, se g atomica, ovvero quando $\{g_1, \dots, g_k\} \subseteq C$ se g è una congiunzione di atomi $g_1 \wedge \dots \wedge g_k$.

Proprietà della procedura BU:

- *Punti Fissi*: C restituito è un **minimo punto fisso**:
 - Ogni ulteriore applicazione della regola di derivazione non lo cambia;
 - Detta I l'interpretazione in cui ogni atomo in C è vero e gli altri sono falsi, I dev'essere un modello di KB ;
 - I è un modello *minimale* di KB : ha il minor numero di proposizioni vere tra tutti i suoi modelli.
- *Correttezza*: ogni atomo in C è conseguenza logica di KB ;
 - Se $KB \vdash g$ allora $KB \models g$
- *Completezza*: tutte le conseguenze logiche sono derivabili;
 - Se $KB \models g$ allora g è vera in ogni modello di KB , quindi anche in quello *minimo*, ovvero in C , per cui $KB \vdash g$.

La procedura *Top-Down* si basa su una ricerca *backward*, ossia *top-down*, a partire da una query per determinare se questa segua logicamente dalle clausole di KB . Essa si basa su una diversa regola di ragionamento (inferenza su clausole) detta **risoluzione**:

$$\frac{l_1 \vee \dots \vee a \vee \dots \vee l_r \quad l'_1 \vee \dots \vee \neg a \vee \dots \vee l'_m}{l_1 \vee \dots \vee l_r \vee l'_1 \vee \dots \vee l'_m}$$

si noti che si applica alla forma generale delle clausole: basta individuare un atomo a che compare positivamente in una delle due premesse e negativamente nell'altra.

In particolare, si usa la **risoluzione SLD** (Selezione di un atomo usando una strategia Lineare su clausole Definite):

$$\frac{a \leftarrow a_1 \wedge \dots \wedge a_i \wedge \dots \wedge a_r \quad a_i \leftarrow b_1 \wedge \dots \wedge b_m}{a \leftarrow a_1 \wedge \dots \wedge b_1 \wedge \dots \wedge b_m \wedge \dots \wedge a_r}$$

- versione *proposizionale* di una forma più generale presentata in seguito.

Una **clausola di risposta** ha la forma:

$$yes \leftarrow a_1 \wedge a_2 \wedge \dots \wedge a_m$$

Dove *yes* è un atomo *speciale*, vero se la risposta alla query è *yes*.

Data la query $ask \ q_1 \wedge \dots \wedge q_m$, si parte dalla clausola di risposta *iniziale* $yes \leftarrow q_1 \wedge \dots \wedge q_m$; successivamente si ripete:

- la *selezione* di un atomo del corpo (*sotto-goal*) da dimostrare;
- la *scelta* di una clausola definita in KB avente come testa l'atomo selezionato per effettuare un passo di **risoluzione**; qualora non vi fossero altre clausole per dimostrare l'atomo selezionato, la dimostrazione *fallisce* (risposta: *no*).

Il **risolvente** della clausola di risposta e della clausola scelta sarà una nuova clausola di risposta:

$$\frac{yes \leftarrow a_1 \wedge a_2 \wedge \dots \wedge a_m \quad a_i \leftarrow b_1 \wedge \dots \wedge b_p}{yes \leftarrow b_1 \wedge \dots \wedge b_p \wedge a_2 \wedge \dots \wedge a_m}$$

in sostanza l'atomo *goal* selezionato viene sostituito dal corpo della clausola scelta.

In un dimostrazione di successo si raggiunge la **risposta**, ossia una clausola di risposta con corpo vuoto (tutti i goal sono stati dimostrati):

$$yes \leftarrow$$

Nella **derivazione SLD** per una query $ask \ q_1 \wedge \dots \wedge q_k$ da KB si avrà una sequenza di clausole di risposta $\gamma_0, \gamma_1, \dots, \gamma_n$ tali che:

- γ_0 è la *query originaria* $yes \leftarrow q_1 \wedge \dots \wedge q_k$;
- γ_i è il *risolvente* di γ_{i-1} con una clausola definita in KB ;
- γ_n è la *risposta*.

Una procedura *Top-Down* alternativa potrebbe essere la seguente. Si parte da un insieme G (*goal*) di *atomi da dimostrare*, quindi:

- Si inizializza G con tutti gli atomi della query (*sotto-goal*), quindi G corrisponde a $yes \leftarrow \bigwedge_{g_i \in G} g_i$;
- Selezionato a , la clausola scelta $a \leftarrow b_1 \wedge \dots \wedge b_p$ indica che a può essere sostituito in G dai sotto-goal $b_1, \dots, b_p$;
- Si ha una terminazione con successo se $G = \emptyset$.

Qualsiasi atomo di G può essere *selezionato*. Se una selezione non porta a terminare la prova, non serve tentare di selezionarne un altro, l'intera dimostrazione fallisce.

La procedura di *scelta* della clausola per la risoluzione è *non-deterministica*. Se ci sono scelte che portano a G vuoto, l'algoritmo ha successo (rispondendo *yes*), *altrimenti* il tentativo fallisce (e si può rispondere *no*).

```
scegliere da  $KB$  una clausola definita  $a \leftarrow B$  con  $a$  come testa
// se possibile, altrimenti il tentativo fallisce
 $G \leftarrow (G \setminus \{a\}) \cup B$ 
```

Il corpo della clausola B e G sono considerati come *insiemi* di atomi. In alternativa G può essere costituito da una *lista* ordinata di atomi che possono occorrere più volte.

L'algoritmo TD su un grafo di ricerca è un algoritmo alternativo, indotto dalla strategia di selezione basato su un *grafo di ricerca*, in cui:

- I *nodi* contengono clausole di risposta;
- I *vicini* di $yes \leftarrow a_1 \wedge \dots \wedge a_m$ rappresentano tutte le possibili clausole di risposta ottenute risolvendo su a_1 e possono essere generati *dinamicamente*, uno per ogni clausola definita con testa a_1 ;
- I *nodi obiettivo* sono del tipo $yes \leftarrow$;
- Si può usare qualsiasi metodo di ricerca su grafo, l'importante è trovare un percorso di successo in modo tale che la query sia conseguenza logica della base di conoscenza; per ogni nodo, lo spazio viene determinato dalla query e dall'atomo selezionato.

La *BU* prova ogni atomo *una sola volta*, mentre *TD* potrebbe provare lo stesso atomo *più volte*, ma si concentra su quelli *rilevanti* per la query.

Con *TD* sono possibili *cicli infiniti*.

Esempio — Data $KB = \{g \leftarrow a. \quad a \leftarrow b. \quad b \leftarrow a. \quad g \leftarrow c. \quad c.\}$ e la query $ask \ g.$

- le uniche conseguenze logiche (atomiche) sono: g e c ;
- *BU* termina con il punto fisso $\{c, g\}$;
- *TD* con *DFS* semplice potrebbe continuare *indefinitamente*, a meno che non sia prevista la *potatura* dei cicli.

La *strategia di selezione* dell'atomo per la risoluzione condiziona *efficienza e terminazione*:

- Negli esempi precedenti si sceglieva l'atomo più a sinistra, il che risulta problematico senza controllo sui cicli;
- Una strategia migliore seleziona l'atomo che porti al *fallimento* più rapidamente;
- Una strategia comune prevede l'*ordinamento* degli atomi per la selezione: ciò consente l'uso di un'*euristica*.

Interrogare l'Utente

A run-time il sistema riceve **osservazioni** dagli utenti, da sensori o altre sorgenti. Esse sono rappresentate come *insiemi di atomi* (non direttamente come regole), ad esempio nella diagnosi medica il paziente comunica i propri sintomi.

Le osservazioni interagiscono con la **conoscenza di fondo** (BK): possono esservi *aggiunte* o essere tenute *separate* dalla BK.

Le informazioni fornite dall'**utente** sono incomplete: è possibile che non sappia usare il *vocabolario* o non sappia giudicarne la *rilevanza*. Per questo servirebbero:

- 1) Un'**ontologia** che specifichi il significato dei simboli;
- 2) Un'**interfaccia** che possa aiutare a fornire informazioni rilevanti.

Analogamente, i *sensori* possono essere *passivi*, quando raccolgono osservazioni dirette (coniunzioni di atomi) oppure *attivi*, se atti a raccogliere risposte a esplicite richieste di informazioni necessarie.

Per l'*acquisizione* si può pensare a un meccanismo **ask-the-user** integrato nella procedura di ragionamento: si considerano atomi detti **askable** per rappresentare fatti veri acquisibili a run-time dall'utente (o da sensore). Nella procedura TD, selezionato un atomo da dimostrare, si usa una clausola della base di conoscenza, oppure, se askable, si chiede se sia vero o falso.

N.B.: ciò vale solo per atomi *rilevanti*, quindi si considerano tre diverse *classi* di atomi:

1. Non askable;
2. Askable senza risposta-utente: si può chiedere e memorizzare;
3. Askable con risposta-utente memorizzata: usati senza chiedere di nuovo.

È possibile adottare un meccanismo simile anche con procedure BU, ma vanno evitate *troppe richieste* all'utente.

Debugging a Livello di Conoscenza

Data una semantica esplicita si possono realizzare spiegazione e debugging a livello di conoscenza. Il **knowledge-level debugging** consiste nel trovare *errori* nelle basi di conoscenza facendo riferimento *solo* al significato dei simboli e quanto risulta vero nel mondo codificato e non ai passi di ragionamento.

I diversi tipi di *Errori* (non sintattici) *possibili* sono:

- Risposta *errata*: derivazione di atomo falso nell'interpretazione intesa;
- Risposta *mancante*: dimostrazione fallita per qualche atomo vero nell'interpretazione intesa;
- *Loop infinito*: solo con procedure TD, in base alla strategia di selezione. Si può evitare con meccanismi di *cycle pruning* (simili a quelli visti in precedenza).

Le *risposte errate* sono risposte fornite dalla procedura di dimostrazione che sono *false* nell'interpretazione intesa, cioè casi di **falsi positivi**. Se la procedura è *sound* allora nella dimostrazione sarà stata impiegata una *clausola errata*.

Per il *debugging dei falsi positivi* si considerano l'atomo g dimostrato (conseguenza logica) che, invece, dovrebbe essere falso nell'interpretazione intesa e la clausola usata per dimostrarlo $g \leftarrow a_1 \wedge \dots \wedge a_k$. I casi possibili sono i seguenti:

- Tutti gli a_i sono veri nell'interpretazione intesa; quindi, la clausola $g \leftarrow a_1 \wedge \dots \wedge a_k$ è *errata*;
- Uno o più a_i dimostrati sono a loro volta falsi nell'interpretazione intesa, per cui si ha un analogo problema da risolvere (ricorsivamente).

Nelle *risposte mancanti*, una risposta attesa (ossia goal vero nell'interpretazione intesa) non viene derivata dal sistema. Il *fallimento* dipende da un atomo g vero nell'interpretazione intesa ma tale che $KB \not\models g$ si tratta, cioè, di un caso di **falso negativo**.

Tale atomo segnala il caso di una o più *clausole mancanti* tra gli assiomi della base di conoscenza e si può individuare con una procedura per trovare atomi a cui manchino clausole utili alla loro dimostrazione.

Dimostrazione per Contraddizione

Si vuole estendere il ragionamento ammettendo regole che producano *contraddizioni*, ossia utili a specificar casi *impossibili*. Ciò ha anche lo scopo di implementare successivamente forme di *diagnostica*. Ad esempio, nel caso della domotica, attraverso il ragionamento (e le osservazioni) si potrà dedurre che alcuni componenti *non* siano *funzionanti*.

A tale scopo il linguaggio delle clausole definite può essere esteso per esplicitare le contraddizioni. Definito un atomo speciale, detto *false*, che codifica la *contraddizione*, ossia che è falso in tutte le interpretazioni, saranno ammesse regole, dette **vincoli d'integrità**, rappresentate da clausole con l'atomo *false* come testa:

$$false \leftarrow a_1 \wedge \dots \wedge a_k$$

Una **clausola di Horn** è una clausola definita o un vincolo d'integrità, ossia la sua testa è un atomo normale oppure l'atomo speciale *false*.

Da una base di conoscenza di clausole di Horn si possono derivare **negazioni** di atomi tramite la regola di ragionamento detta *modus tollens*:

$$\frac{p \rightarrow q \quad \neg q}{\neg p}$$

Inoltre, valendo le seguenti equivalenze:

$$(false \leftarrow a_1 \wedge \dots \wedge a_k) \equiv \neg(a_1 \wedge \dots \wedge a_k) \equiv \neg a_1 \vee \dots \vee \neg a_k$$

Dimostrare che una *congiunzione di atomi* sia *falsa* in tutti i modelli di *KB* equivale a provare una **disgiunzione di negazioni di atomi**.

Un insieme di clausole *KB* è **insoddisfacibile** se non ha modelli, ovvero **non esiste alcun mondo possibile** (modello) in cui tutte le sue affermazioni siano vere contemporaneamente, e quindi:

$$KB \models false$$

Un insieme di clausole *KB* è **inconsistente** rispetto a una procedura di dimostrazione $\vdash$ se consente di derivare *false*:

$$KB \vdash false$$

Se la procedura $\vdash$ è *corretta e completa* allora *KB* inconsistente se e solo se insoddisfacibile.

Una base di conoscenza di clausole definite è *sempre* soddisfacibile, per esempio basta considerare l'interpretazione che assegna il valore *true* a tutti gli atomi. Ciò non vale per basi di clausole di Horn.

Le *contraddizioni* sono utili a individuare combinazioni di *assunzioni incompatibili*, ossia supposizioni fatte (per mancanza di conoscenza) che possono dimostrarsi false. Ad esempio, nella *diagnostica* di un sistema, il funzionamento *normale* delle componenti potrebbe risultare inconsistente rispetto a nuove osservazioni (aggiuntive rispetto alla base di conoscenza). In caso di malfunzionamento serve una diagnosi del guasto.

Si definisce **assumibile** un atomo che può essere assunto (come vero) in una *dimostrazione per contraddizione*. In diagnostica, sarà utile a determinare una *disgiunzione di negazioni* di assumibili, provando la contraddizione a partire da una base di conoscenza e una combinazione di assumibili.

Si dice **conflitto** di KB l'insieme di assumibili $C = \{c_1, \dots, c_r\}$ tale che $KB \cup C \models false$. Si tratta di una **risposta** per una dimostrazione per contraddizione del tipo: $KB \models \neg c_1 \vee \dots \vee \neg c_r$. Un conflitto è **minimale** se nessun suo sottoinsieme costituisce, a sua volta, un conflitto.

Lo *scopo* della *diagnosi* è quello di determinare possibili guasti in base a un *modello del sistema*, una base di conoscenza, e a *osservazioni* aggiuntive sul sistema stesso.

In sostanza si fanno assunzioni sul funzionamento *normale*, considerano quindi *assumibili* in assenza di guasti. Si derivano (per contraddizione) i componenti *anormali*, ossia i *conflitti* (inconsistenza), le *anomalie* nel sistema.

Dato l'insieme dei conflitti, l'utente può diagnosticare il problema: ciò è anche utile a farsi un'idea della numerosità dei possibili guasti.

Nella **diagnosi basata su consistenza** (CBD) dato un insieme di conflitti, cioè un insieme di assumibili con almeno un elemento in ogni conflitto, si costruisce una *diagnosi* che sarà **minimale** se nessun suo sotto-insieme è una diagnosi.

Intuitivamente, una sola delle diagnosi minimali è quella giusta: quella contenente solo conflitti falsi nell'interpretazione intesa.

L'*obiettivo* è la ricerca dei *conflitti* in basi di conoscenza di clausole di Horn attraverso dimostrazioni per contraddizione dei due tipi considerati in precedenza.

Si estende l'algoritmo bottom-up per clausole definite:

- Le conclusioni sono *coppie* $\langle a, A \rangle$, con A insieme di assumibili che implicano l'atomo a rispetto a KB ;

- Nell'inizializzazione, l'insieme di conclusioni sarà $C \leftarrow \{ \langle a, \{a\} \rangle \mid a \text{ assumibile} \}$;
- Si usano poi le clausole in KB per derivare nuove conclusioni:
se $\exists h \leftarrow b_1 \wedge \dots \wedge b_m \in KB$ tale che ogni $\langle b_i, A_i \rangle \in C$,
allora si può aggiungere $\langle h, A_1 \cup \dots \cup A_m \rangle$ a C (per i fatti basterà aggiungere $\langle h, \{ \} \rangle$).

Anche l'implementazione top-down deriva dall'omologa per clausole definite, con due *differenze*:

- La query da provare è data dall'atomo speciale *false*;
- Nella dimostrazione, gli atomi assumibili non vanno dimostrati ma semplicemente raccolti via via essendo assunti come veri.

Assunzione di Conoscenza Completa

Nella semantica delle *basi di dati* quello che non è derivabile viene considerato falso: la conoscenza su dominio viene considerata *completa*. In *Logica* la mancanza di conoscenza *non consente* di trarre conclusioni dal fallimento delle dimostrazioni.

Assumendo la **completezza della conoscenza**, le clausole con uno stesso atomo come testa coprono *tutti e soli* i casi in cui esso è vero. Tale assunzione si dice anche **closed-word assumption** (CWA). In base ad essa, se non si riesce a dimostrare che a sia vero (nei modelli della base), si conclude che è falso, ossia che la sua negazione $\neg a$ è vera. Si assume quindi che tutto quello che è rilevante sia stato asserito *esplicitamente* nella base di conoscenza.

In Logica normalmente non si può assumere che una base di conoscenza possa definire tutta la conoscenza su un dato dominio nella sua completezza, si parla pertanto di **open-world assumption** (OWA). Quindi, non potendosi trarre conclusioni dalla mancanza di conoscenza, nel caso precedente ($KB \not\models a$), la base di conoscenza di clausole definite *non può avere* $\neg a$ come conseguenza logica.

Per codificare l'assunzione di conoscenza completa negli assiomi della base di conoscenza, essa dev'essere trasformata ossia *completata*.

Dato l'atomo a e tutte le clausole della base di conoscenza che lo definiscono, del tipo:

$$\begin{array}{c} a \leftarrow b_1 \\ \vdots \\ a \leftarrow b_n \end{array}$$

dove b_i (body) è una congiunzione di atomi oppure *true* (per clausole atomiche). In mancanza di clausole per a si considera: $a \leftarrow false$.

Tali clausole possono essere riassunte da un'unica proposizione:

$$a \leftarrow b_1 \vee \dots \vee b_n$$

(si noti che non è una clausola).

Se a è vero (nei modelli di KB) allora, per la CWA, dovrà *necessariamente* essere vero uno dei corpi b_i quindi si può aggiungere:

$$a \rightarrow b_1 \vee \dots \vee b_n$$

Il significato delle clausole per l'atomo a sotto CWA è la congiunzione delle due proposizioni, ossia l'*equivalenza*:

$$a \Leftrightarrow b_1 \vee \dots \vee b_n$$

detta *completamento di Clark* delle clausole per a . Si noti che se non ci sono regole per a , il completamento sarà $a \Leftrightarrow false$.

Il **completamento di Clark** della base di conoscenza ricomprende i completamenti di tutti gli atomi della base.

Esempio — Si consideri la base di conoscenza precedente con le clausole:

- $down_s_1.$
- $up_s_2.$
- $ok_cb_1.$
- $live_l_1 \leftarrow live_w_0.$
- $live_w_0 \leftarrow live_w_1 \wedge up_s_2.$
- $live_w_0 \leftarrow live_w_2 \wedge down_s_2.$
- $live_w_1 \leftarrow live_w_3 \wedge up_s_1.$
- $live_w_2 \leftarrow live_w_3 \wedge down_s_1.$
- $live_w_3 \leftarrow live_outside \wedge ok_cb_1.$
- $live_outside.$

Si noti che non sono definite clausole per up_s_1 e $down_s_2$.

Completamento della base di conoscenza:

- $down_s_1 \Leftrightarrow true.$
- $up_s_1 \Leftrightarrow false.$
- $up_s_2 \Leftrightarrow true.$
- $down_s_2 \Leftrightarrow false.$
- $ok_cb_1 \Leftrightarrow true.$
- $live_l_1 \Leftrightarrow live_w_0.$
- $live_w_0 \Leftrightarrow (live_w_1 \wedge up_s_2) \vee (live_w_2 \wedge down_s_2).$
- $live_w_1 \Leftrightarrow live_w_3 \wedge up_s_1.$
- $live_w_2 \Leftrightarrow live_w_3 \wedge down_s_1.$
- $live_w_3 \Leftrightarrow live_outside \wedge ok_cb_1.$
- $live_outside \Leftrightarrow true.$

Così, ad esempio, up_s_1 è falso, $live_w_1$ è falso e $live_w_2$ è vero.

Un **letterale** è un atomo o la negazione di un atomo. Con il completamento si possono dimostrare *negazioni* di proposizioni, quindi la negazione può essere ammessa anche nel corpo delle clausole.

Si considereranno clausole definite *estese* contenenti nel corpo letterali (e non atomi). La *negazione sotto assunzione di conoscenza completa*, detta anche **negation as failure** (NAF), verrà denotata con $\sim a$ per distinguerla dalla negazione classica (in pratica $\neg a$ dice “ho le prove che a è falso”, mentre $\sim a$ dice “non ho le prove che a sia vero”).

Sotto CWA, il corpo g è una conseguenza di KB se e solo se $KB' \models g$, dove KB' è il completamento di KB . Si noti come $\sim a$ nel corpo di una clausola diventi $\neg a$ nel completamento.

Con la NAF, le basi di conoscenza non acicliche diventano *problematiche* dal punto di vista semantico. Ad esempio, data la base non aciclica:

- $a \leftarrow \sim b$;
- $b \leftarrow \sim a$.

Il completamento equivalente a $a \Leftrightarrow \neg b$, indica che a e b hanno valori di verità opposti: solo uno può essere vero.

Altro esempio di base di conoscenza non aciclica è una con l'assioma:

- $a \leftarrow \sim a$.

con completamento:

- $a \Leftrightarrow \neg a$

che risulta *inconsistente* dal punto di vista logico.

Il completamento di una *base aciclica* è sempre *consistente* e prevede *un solo* valore di verità per ogni atomo.

In una logica **monotona** ogni proposizione derivabile da una base di conoscenza rimane derivabile dopo l'aggiunta di altre proposizioni: aggiungendo assiomi *non* si riduce l'insieme delle proposizioni derivabili. Ad esempio, si consideri la *Logica delle clausole definite*. In una *Logica non monotona* alcune conclusioni possono essere invalidate con l'aggiunta di altri assiomi. Ad esempio, si consideri la *Logica delle clausole definite* con l'aggiunta della NAF.

Si dice **default** una regola che resta valida fintantoché non si verifichi un'*eccezione*. Ad esempio, per asserire che *normalmente* b è vera se c è vera si può definire la regola:

$$b \leftarrow c \wedge \sim ab_a$$

con ab_a che indica l'*anormalità* rispetto a qualche aspetto di a . Dato c si può derivare b , a meno che non venga asserito ab_a , in quanto l'aggiunta di ab_a *inibisce* la possibilità di concludere b . Le regole che implicano ab_a possono inibire il default, sotto le condizioni del corpo della regola. Ad esempio, nella maggior parte dei casi gli uccelli (c) sono volatili (b), ma i pinguini fanno eccezione (ab_a).

La procedura BU per clausole definite va modificata: quando p *fallisce* va aggiunto il letterale $\sim p$ all'insieme C delle conseguenze derivate.

Si ha una definizione ricorsiva del **fallimento**: p fallisce se fallisce il corpo di *ogni clausola* che abbia p come testa: un corpo fallisce se *almeno uno* dei suoi *letterali* fallisce; un atomo b_i fallisce se si può derivare $\sim b_i$, e la negazione $\sim b_i$ fallisce se si può derivare b_i .

Osservazione: Sono ricompresi i casi di *corpo vuoto* e *atomo non definito*.

La procedura TD è analoga a quella delle clausole definite, ma procede per NAF. È non-deterministica e, come tale, si può implementare come ricerca di scelte di successo: selezionato un atomo $\sim a$, parte una nuova dimostrazione per a ; se questa fallisce allora $\sim a$ ha successo altrimenti l'algoritmo fallisce: deve tentare altre scelte se possibile.

Quanto detto vale solo nel caso di *fallimento finito*: nessuna conclusione è possibile in caso di divergenza. Ad esempio, data una base di conoscenza con la sola regola $p \leftarrow p$ e la query *ask* p , l'algoritmo non converge: il completamento è $p \Leftrightarrow p$ ma, pur potendosi accertare l'indimostrabilità di p , una procedura *corretta* non può concludere $\sim p$ poiché non segue logicamente dal completamento.

Abduzione

L'**Abduzione** è una forma di ragionamento nella quale si fanno *ipotesi* ossia *assunzioni* al fine di *spiegare* osservazioni. Le ipotesi da farsi *possono* implicare le osservazioni fatte: se verificate, esse giustificano quanto osservato ma non si devono introdurre *contraddizioni* poiché queste giustificerebbero qualsiasi conclusione. Ad esempio, se si osserva che qualche punto-luce non sta funzionando, si fanno ipotesi per spiegare le ragioni di quanto sta succedendo.

L'abduzione si differenzia dalla *deduzione* la quale mira alle conseguenze logiche di un set di assiomi e dall'*induzione* che comporta l'inferenza di relazioni generali da casi particolari (esempi).

Possiamo schematizzare questa forma di ragionamento in questo modo:

Dati:

- Una base KB di Horn;
- Un insieme A di atomi utili a costruire ipotesi, detti *assumibili* o anche *abducibili*,

Trovare: spiegazioni per delle osservazioni.

Si dice **scenario** di $\langle KB, A \rangle$ un insieme di abducibili $H \subseteq A$ tale che $KB \cup H$ sia soddisfacibile (ossia ammetta modelli). In tal caso deve esistere un modello in cui tutti gli elementi di KB e di H sono veri, quindi nessun sottoinsieme di H dev'essere un conflitto per KB .

Una **spiegazione** (*ipotesi*) della proposizione g da $\langle KB, A \rangle$ è uno scenario $H \subseteq A$ che, assieme a KB , abbia g come conseguenza; quindi, $KB \cup H \models g$ e $KB \cup H \not\models \text{false}$. Essa è **minimale** se H non contiene altre spiegazioni: per ogni altra H' di g da $\langle KB, A \rangle$, deve risultare $H' \not\subseteq H$.

La *diagnosi abduttiva* (AbD) formula ipotesi su problemi riscontrati e funzionamento normale.

Confrontandola con quella basata su consistenza (CbD):

- Al livello di *rappresentazione*, la CbD mira *solo* al comportamento normale quindi le ipotesi saranno assunzioni sul comportamento normale, mentre con la AbD si modella anche il comportamento anomalo, si devono prevedere assumibili anche per ciascun guasto (anomalia);
- Per quanto riguarda le *osservazioni*, la CbD le *aggiunge* alla base di conoscenza per poi dimostrare *false* mentre la AbD le considera come goal da *spiegare* attraverso ipotesi;
- In termini di *qualità*, la AbD risulta più dettagliata, le diagnosi sono migliori: si distinguono base di conoscenza e assunzioni, le quali consentono di *dimostrare* le osservazioni, anche casi per cui non sia definibile la normalità.

Il ragionamento abduttivo è basato su *assunzioni* e si realizza mediante le procedure su *clausole di Horn*:

- Con la procedura BU si calcolano spiegazioni minimali per ogni atomo, essa è ulteriormente specializzabile con il pruning delle spiegazioni *dominate*, ovvero quelle meno efficienti o ridondanti rispetto ad altre;
- Con la procedura TD si trovano spiegazioni di un g generando i conflitti, ma mirando a dimostrare g anziché *false*: la spiegazione minimale di g sarà un insieme minimale di assumibili, ovvero le ipotesi che spiegano il problema.

5 – Rappresentazione e Ragionamento Relazionale

Struttura Relazionale

Nel rappresentare la *struttura* del mondo, è utile ragionare in termini di individui e relazioni.

Gli **individui** sono entità, cose, oggetti del dominio, e potranno essere *concreti* (es. persone, edifici), *immaginari* (es. unicorni, programmi che superino il test di Turing), o anche *concetti astratti*, come i processi (es. la lettura di un libro, andare in vacanza) o altri concetti (es. il denaro, i corsi di studio, gli istanti di tempo, ecc...).

Le **relazioni**, in generale, specificano verità riguardanti gli individui. Possono essere *proprietà*, vere o false, dei singoli individui oppure anche *proposizioni* che possono essere vere o false indipendentemente da specifici individui, e anche *associazioni* che possono intercorrere tra più individui.

Con questa rappresentazione, il ragionamento su più istanze potrà risultare più semplice perché più generale.

I *vantaggi* della nuova rappresentazione sono molteplici:

- Essa è più *naturale* poiché sfrutta la struttura delle feature come proprietà degli individui;
- È utile anche nel caso di *domini sconosciuti* per i quali non vi sia nessuna conoscenza sugli individui, sul loro numero e delle loro caratteristiche: si deve interagire con l'ambiente per apprendere la rappresentazione;
- Consente forme di *ragionamento* più *general*i e indipendenti dai singoli individui, con obiettivi quali la derivazione di *conclusioni* che valgono *per tutti* gli individui senza conoscerli necessariamente, la scoperta dell'*esistenza* di nuovi individui e loro proprietà, a prescindere dagli altri e, inoltre, la possibilità di rispondere a *query* che riguardano intere popolazioni e non singoli individui;
- Rende possibile il trattamento dell'*esistenza incerta* di individui: non si conoscono completamente individui attuali e futuri, nonché le loro proprietà;
- Rende possibile il ragionamento su un *infinito* numero di individui (e le loro proprietà).

Ad esempio, si può ragionare su frasi (individui), il cui numero è potenzialmente infinito, pur considerandone un numero finito, ovvero quelle utili al compito da svolgere.

Simboli e Semantica

La *Logica* può risultare molto utile nella progettazione di basi di conoscenza. I progettisti devono caratterizzare un particolare *mondo* formalizzando la cosiddetta **interpretazione intesa**: scegliendo *significati* per i simboli in base a tale interpretazione e descrivendo ciò che dev'essere vero attraverso una base di conoscenza fatta di *clausole*. Chiunque conosca il significato dei simboli può interpretare, rispetto a tale modello, le *conseguenze logiche* derivate dalla base: ciò che è vero in tutti i modelli della base sarà vero, in particolare, nell'interpretazione intesa.

Estendendo il linguaggio delle clausole proposizionali si possono definire atomi con una *struttura interna* descritta in termini di relazioni e individui.

Una **concettualizzazione** è un'associazione fra simboli nella mente, gli individui e le relazioni da essi denotati.

Vi è una separazione tra l'aspetto *semantico* e quello *computazionale*: la *correttezza* della base di conoscenza e delle risposte alle query è definita dalla *semantica*, indipendentemente dall'*algoritmo* di ragionamento, che si può ottimizzare a patto di rimanere semanticamente corretto.

Calcolo dei Predicati

Il **calcolo dei predicati**, o *logica dei predicati* o *logica del primo ordine* estende il calcolo proposizionale in due modi. Gli atomi sono dotati di una loro *struttura* con la possibilità di includere variabili; inoltre, le variabili logiche sono *quantificate*.

Per definire la sintassi, occorre specificare le seguenti nozioni:

- **Simbolo**: stringa alfanumerica comprendente anche _;
- **Variabile logica**: simbolo che inizia con maiuscola o _
es. *Stanza*, *_B₄*, *Lista* e *The_Dude*
- **Costante**: simbolo che inizia con una minuscola o numero o stringa (tra apici);
- Simbolo di **predicato**: simbolo che inizia con una minuscola (costanti e predicati distinguibili in base al *contesto*)
es. *nico*, *r123*, *f_1*, *insegna_a* e *aula_IV*, ma 2018 è solo una costante
- **Termine**: variabile o costante
es. *X*, *nico*, *dib*, *uniBA* o *Docenti*
- **Atomo** (simbolo atomico): forma *p* oppure *p(t₁, ..., t_n)* con *p* simbolo di predicato e ogni *t_i* è un termine, *i*-esimo *argomento* del predicato

esempi: $aperto, giallo(C), insegna(nicoF, 63507), in(vitoR, 522), padre(alanT, Y), inserire(Dessert, menuT)$. In $padre(alanT, Y)$, dal contesto si desume che $padre$ è un simbolo predicato mentre $alanT$ è una costante.

Una **formula logica** viene definita costruttivamente a partire dagli atomi, usando connettivi logici e quantificatori. Date la variabile X e la formula logica p (in cui possa occorrere X):

- $\forall X p$
 - Da leggersi “per ogni X , p ” si dice che X è **quantificata universalmente**;
- $\exists X p$
 - Da leggersi “esiste un X tale che p ”, e si dice che X è **quantificata esistenzialmente**.

Un’espressione **ground**, o di (livello-)base, non contiene variabili. Essa è un atomo o, più in generale, una formula che si riferisce solo a precisi individui.

Un’**interpretazione** delle espressioni *ground* è definita da una tripla

$$I = \langle D, \phi, \pi \rangle$$

dove:

- Il **dominio** D è l’insieme non vuoto di **individui**;
- La funzione $\phi: c \mapsto \phi(c) \in D$ assegna a ogni costante c un individuo di D ;
- La funzione $\pi: p \mapsto \pi(p)$ assegna a ogni simbolo di *predicato* n -ario p una funzione $\pi(p): D^n \mapsto \{true, false\}$ che per ogni n -pla di individui, $\pi(p)$ specifica se la relazione p sia vera/falsa;
 - π è una *funzione di ordine superiore* in quanto associa un nome di predicato a una funzione (predicativa);
 - Se p non ha argomenti allora $\pi(p)$ sarà una (funzione) costante: *true* o *false* (semantica del calcolo proposizionale).

Esempio — Scena da descrivere: tavolo con forbici, modellino e matita



- le *costanti* potrebbero essere *plane*, *pencil* e *model*
 - si noti l’assenza intenzionale di un nome più preciso (per gli umani) come *scissors*;
- i simboli di *predicato* potrebbero essere *noisy* (unario) e *left_of* (binario);
- una particolare *interpretazione* per la descrizione della scena potrebbe essere costituita dalla tripla $\langle D, \phi, \pi \rangle$ che segue:
 - $D = \{\langle \times, \rightarrow, \circ \rangle\}$ (contiene individui: oggetti del mondo reale)
 - $\phi(plane) = \rightarrow, \phi(pencil) = \circ, \phi(model) = \times$
 - si noti che i nomi *plane* e *model* si riferiscono allo stesso individuo ($\rightarrow$) quindi gli stessi enunciati che li riguardano risulteranno veri;
 - $\pi(noisy): arg$ è rumoroso, definito da

$$\langle \times \rangle \mapsto false, \langle \rightarrow \rangle \mapsto true, \langle \circ \rangle \mapsto false;$$
 - $\pi(left_of): arg_1$ è a sinistra di arg_2 , definito da

$$\begin{aligned} \langle \times, \times \rangle &\mapsto false, \langle \times, \rightarrow \rangle \mapsto true, \langle \times, \circ \rangle \mapsto true, \\ \langle \rightarrow, \times \rangle &\mapsto false, \langle \rightarrow, \rightarrow \rangle \mapsto false, \langle \rightarrow, \circ \rangle \mapsto true, \\ \langle \circ, \times \rangle &\mapsto false, \langle \circ, \rightarrow \rangle \mapsto false, \langle \circ, \circ \rangle \mapsto false \end{aligned}$$

Nell'interpretazione I , ogni termine ground c denota un individuo $\phi(c)$.

Semantica degli atomi ground: nell'interpretazione I , l'atomo ground $p(t_1, \dots, t_n)$ è **vero** se $\pi(p)(\langle t'_1, \dots, t'_n \rangle) = \text{true}$ (dove $t'_k = \phi(t_k) \in D$ è l'individui denotato dal termine t_k) ed è **falso** altrimenti.

Un'**assegnazione di variabili** ρ è una funzione dall'insieme delle variabili a D che associa un elemento del dominio a ogni variabile.

Data l'interpretazione $I = \langle D, \phi, \pi \rangle$ e l'assegnazione ρ , ogni *termine* denota un individuo in D interpretato da ϕ se costante mentre da ρ se variabile. La semantica degli *atomi* è definita come visto in precedenza, ottenuti associando individui ai termini.

Una formula *universalmente quantificata* $\forall X p$ è **vera** in un'interpretazione se p è vera per ogni assegnazione di X , e sarà falsa se c'è un'assegnazione a X per la quale la formula risulti falsa: la formula p costituisce l'**ambito** (o *scope*) di X .

Una formula *esistenzialmente quantificata* $\exists X p$ è **vera** in un'interpretazione se e solo se p è vera per qualche assegnazione di X : l'individuo assegnato può dipendere da variabili universalmente quantificate nel cui ambito compaia $\exists X p$.

- In $\forall Y(\exists X p)$, X può dipendere da Y
es. per ogni numero Y c'è un X tale che $X = Y + 1$ (vero);
- In $\exists X(\forall Y p)$, X non può dipendere da Y poiché X non è nell'ambito di Y
es. esiste un numero X tale che per ogni numero Y risulti $X = Y + 1$ (falso).

Una **formula** con variabili è **vera** in I se e solo se per qualsiasi assegnazione ρ la formula ground ottenuta applicando ρ alle sue variabili risulta vera.

Una formula si dice **aperta** se ha almeno una variabile non quantificata, per cui la sua semantica non può essere determinata.

La nozione di modello e la relazione di **conseguenza logica** fra una data base di conoscenza KB e un goal g si estende al caso delle formule logiche del calcolo dei predicati:

$$KB \models g \text{ se } g \text{ è vera in ogni modello di } KB$$

Estendendo la metodologia introdotta per le basi di conoscenza proposizionali:

1. Si seleziona il *dominio* del task ossia il *mondo* da rappresentare, definendo:
 - a. D insieme di tutti gli *individui* a cui si farà riferimento e sui quali ragionare;
 - b. Le *relazioni* da rappresentare:

- i. Aspetti specifici del mondo *reale*, ad esempio la struttura dei corsi e degli studenti presso un'università o un laboratorio in un determinato momento;
 - ii. Mondi *immaginari* o *ipotetici*, ad esempio quello di *Alice nel paese delle meraviglie*, o lo stato di un'apparecchiatura elettrica;
 - iii. Mondi *astratti*, es. numeri e insiemi, contesti finanziari, ...
2. Si associano le *costanti* del linguaggio a individui del mondo da nominare:
 - a. A ogni elemento di D si assegna una costante per riferirvisi, ad esempio *kim* per un particolare docente, *cs322* per un corso, *two* per il successore del numero uno e *red* per il colore delle luci di stop.
3. Si associa un simbolo di *predicato* a ogni relazione da rappresentare. Le associazioni simbolo-significato verranno usate per costruire l'**interpretazione intesa**:
 - a. Un simbolo (n -ario) denota una funzione $D^n \rightarrow \{true, false\}$, basta specificare le n ple per le quali la relazione sia vera, ad esempio il predicato *teaches* binario rappresenta una relazione vera tra un individuo (primo argomento) e un corso insegnato (secondo argomento);
 - b. Le relazioni possono avere qualunque arietà (anche nulla), ad esempio *is_red* predicato unario.
4. Si definiscono *formule* vere nell'interpretazione intesa: **assiomatizzazione del dominio**
 - a. **Assiomi** del dominio in forma di clausole, ad esempio se la persona denotata da *kim* insegna il corso denotato da *cs322*, la formula *teaches(kim, cs322)* sarà vera nell'interpretazione intesa.
5. Si possono formulare *query* riguardanti l'interpretazione intesa:
 - a. Risposte del sistema da interpretare nella semantica attribuita ai simboli.

DATALOG: Linguaggio Relazionale a Regole

Il DATALOG estende il linguaggio delle clausole definite proposizionali con la sintassi del calcolo dei predicati.

Una **clausola definita** si scrive nella forma

$$h \leftarrow a_1 \wedge \dots \wedge a_m$$

e si legge "*h* se a_1 e ... e a_m ", dove:

- L'atomo h è la **testa** della clausola;
- Se $m > 0$ si dice **regola** e la congiunzione di atomi $a_1 \wedge \dots \wedge a_m$ ne costituisce il **corpo**;
- Se $m = 0$ la *clausola* si chiama clausola **atomica** o **fatto** (e si può omettere la freccia);
- Le variabili sono da intendersi, esplicitamente *universalmente quantificate*, ad esempio $h(X) \leftarrow b(X, Y)$ è da intendersi con $\forall X \forall Y (h(X) \leftarrow b(X, Y))$, che è equivalente a $\forall X (h(x) \leftarrow \exists Y b(X, Y))$.

Esempio — data una relazione in forma di tabella:

Course	Section	Time	Room
cs111	7	830	dp101
cs422	2	1030	cc208
cs502	1	1230	dp202

il predicato corrispondente *scheduled(Course, Section, Time, Room)* in una base di conoscenza sarà definito dai fatti:

- *scheduled(cs111, 7, 830, dp101).*
- *scheduled(cs422, 2, 1030, cc208).*
- *scheduled(cs502, 1, 1230, dp202).*

Si può considerare anche *enrolled(StudentNum, Course, Section)*, definita attraverso fatti e poi anche *busy(StudentNum, Time)*, ottenuta dal join delle precedenti, proiettato su *StudentNum* e *Time*:

$$busy(StudentNum, Time) \leftarrow enrolled(StudentNum, Course, Section) \wedge scheduled(Course, Section, Time, Room).$$

studente impegnato a un dato orario se iscritto a una sezione d'un corso previsto per quell'ora.

Una **query** serve a chiedere se un enunciato sia conseguenza logica di una base di conoscenza. Le query *decisionali* sono quelle tipiche della logica proposizionale (cioè di una rappresentazione ground, senza variabili) e hanno risposta yes o no. Le query *con variabili* consentono, in aggiunta, di determinare e restituire tuple di *individui* per le quali la query si avvera.

Si dirà **istanza** di una query la formula ottenuta sostituendo termini a variabili: ogni occorrenza di una variabile dev'essere sostituita con uno stesso termine. L'**estrazione della risposta** serve a determinare quali istanze seguano dalla base di conoscenza.

Le **risposte** possibili per una query con variabili libere sono:

- *Istanze* della query che seguono logicamente dalla base, una per ciascuna distinta tupla di valori assegnabile alle variabili della query, oppure
- no, se nessuna istanza segue logicamente dalla base (ciò non significa che la query sia falsa nell'interpretazione intesa, ma che non c'è una conoscenza sufficiente per poter rispondere).

Sostituzioni e Dimostrazioni

Per implementare il ragionamento su basi di conoscenza e query DATALOG si possono generalizzare le procedure BU e TD per basi proposizionali. Si ponga attenzione al fatto che in una clausola DATALOG una variabile *libera* indica che tutte le *istanze* della clausola dovranno essere vere, al variare delle possibili assegnazioni. Una dimostrazione potrà anche basarsi su istanze *diverse* di una stessa clausola.

Come per le query, un'**istanza** di clausola si ottiene sostituendo variabili con termini uniformemente: ogni occorrenza di una data variabile dev'essere sostituita con lo stesso termine. Una **sostituzione** specifica, per ciascuna variabile, il termine da sostituire:

$$\{V_1/t_1, \dots, V_n/t_n\}$$

- ogni V_i è una *distinta* variabile e ogni t_i è un termine;
- ogni V_i/t_i si dice **binding** (*associazione*) per la variabile V_i e si dirà in **forma normale** se nessuna V_i occorre in un t_j .

Esempio — Sostituzioni:

- $\{X/Y, Z/chile\}$ è una sostituzione in forma normale;
- $\{X/Y, Z/X\}$ non è in forma normale: X occorre sia a sinistra sia a destra di associazioni diverse.

Un'*espressione* è un termine, un atomo o una clausola. Un'**istanza** $e\sigma$ dell'espressione e si ottiene dall'**applicazione** di $\sigma = \{V_1/t_1, \dots, V_n/t_n\}$ a e : ogni occorrenza di V_i va sostituita con t_i . L'*istanza* si dice **ground** se non contiene variabili.

Un **unificatore** delle espressioni e_1 e e_2 è una sostituzione σ tale che

$$e_1\sigma = e_2\sigma$$

ovvero quando una sostituzione applicata a più espressioni produce un'unica istanza. Quando esiste un tale unificatore si dice che e_1 e e_2 sono *unificabili* o che si *unificano*.

Esempio — unificatori

- $\{X/chile, Y/peru\}$ è un unificatore per $borders(peru, X)$ e $borders(Y, chile)$
 - dalla cui applicazione si ottiene: $borders(peru, chile)$
- $\{X/a, Y/b\}$ unificatore di $t(a, Y, c)$ e $t(X, b, c)$
 - per cui: $t(a, Y, c)\{X/a, Y/b\} = t(X, b, c)\{X/a, Y/b\} = t(a, b, c)$

Diversi unificatori possibili di $p(X, Y)$ e $p(Z, Z)$ sono:

- $\{X/b, Y/b, Z/b\}$, $\{X/c, Y/c, Z/c\}$, $\{X/Z, Y/Z\}$, $\{Y/X, Z/X\}$, ecc.
 - i primi due specificano precisamente i valori sostituiti
 - gli altri due sono *più generali*.

L'**unificatore più generale** (*most general unifier*, MGU) σ di e_1 ed e_2 è l'unificatore tale che, considerato qualunque altro loro unificatore σ' , risulta che $e\sigma'$ è un'istanza di $e\sigma$ per ogni espressione e .

Si dirà che e_1 è una **ridenominazione** (renaming) di e_2 se e solo esse sono istanze l'una dell'altra, ossia differiscono solo nei nomi delle variabili. Due espressioni unificabili avranno almeno un MGU e, applicando loro degli MGU, si otterranno renaming reciproci: dati σ e σ' MGU di e_1 ed e_2 , si ha che $e_1\sigma$ è un renaming di $e_1\sigma'$ (analogamente per e_2).

Esempio — Due MGU degli atomi $p(X, Y)$ e $p(Z, Z)$ sono $\{X/Z, Y/Z\}$ e $\{Z/X, Y/X\}$. Applicando entrambi a $p(X, Y)$ si ottengono: $p(X, Y)\{X/Z, Y/Z\} = p(Z, Z)$ e $p(X, Y)\{Z/X, Y/X\} = p(X, X)$, che è un renaming di $p(Z, Z)$.

Per estendere la *procedura BU* proposizionale al DATALOG, si deve effettuare preventivamente il **grounding** degli assiomi della base di conoscenza ottenendo le *istanze ground* delle clausole, sostituendo uniformemente nelle clausole costanti a variabili. Le costanti da considerare sono quelle che occorrono nella base di conoscenza o nella query.

Qualora non ve ne fossero, se ne deve inventare una nuova.

Si può dimostrare che la procedura BU, applicata al *grounding* di una base di conoscenza, è **corretta** e **completa**.

Dati due termini o atomi, occorre determinare se si unificano e, nel caso, restituire un loro unificatore.

Il seguente algoritmo consente di trovare un eventuale MGU di due atomi o termini e, altrimenti, restituisce $\perp$ per indicare la mancata unificabilità.

```

procedure Unify( $t_1, t_2$ )
  Inputs
     $t_1, t_2$ : atomi o termini
  Output
    MGU di  $t_1$  e  $t_2$  se esiste, altrimenti  $\perp$ 
  Local
     $E$ : insieme di uguaglianze
     $S$ : sostituzione
     $E \leftarrow \{t_1 = t_2\}$ 
     $S = \emptyset$ 
    while  $E \neq \emptyset$  do
      selezionare e rimuovere  $\alpha = \beta$  da  $E$ 
      if  $\beta$  non identica ad  $\alpha$  then
        if  $\alpha$  variabile then
          sostituire  $\alpha$  con  $\beta$  ovunque in  $E$  e  $S$ 
           $S \leftarrow \{\alpha/\beta\} \cup S$ 
        else if  $\beta$  variabile then
          sostituire  $\beta$  con  $\alpha$  ovunque in  $E$  e  $S$ 
           $S \leftarrow \{\beta/\alpha\} \cup S$ 
        else if  $\alpha$  è  $p(\alpha_1, \dots, \alpha_n)$  e  $\beta$  è  $p(\beta_1, \dots, \beta_n)$  then
           $E \leftarrow E \cup \{\alpha_1 = \beta_1, \dots, \alpha_n = \beta_n\}$ 
        else return  $\perp$ 
    return  $S$ 
  
```

N.B.: in caso α e β siano atomi, devono avere stesso simbolo di predicato e stessa arietà, altrimenti l'unificazione fallisce.

Esempio — Volendo unificare $p(X, Y, Y)$ e $p(a, Z, b)$:

- $E = \{p(X, Y, Y) = p(a, Z, b)\}$ e $S = \{ \}$, inizialmente;
- $E = \{X = a, Y = Z, Y = b\}$, dopo la prima iterata;
- $E = \{Y = Z, Y = b\}$, $S = \{X/a\}$, estratta $X = a$, si aggiunge X/a a S ;
- $E = \{Z = b\}$, $S = \{X/a, Y/Z\}$, estratta $Y = Z$, si aggiunge Y/Z a S dopo la sua applicazione a E ;
- $E = \{ \}$, $S = \{X/a, Y/b, Z/b\}$, estratta $Z = b$, si aggiunge Z/b a S ;
- risultato finale: S (un MGU).

Provando a unificare $p(a, Y, Y)$ e $p(Z, Z, b)$:

- $E = \{p(a, Y, Y) = p(Z, Z, b)\}$, inizialmente;
- $E = \{a = Z, Y = Z, Y = b\}$, successivamente;
- $E = \{Y = a, Y = b\}$, estratta $a = Z$, si applica Z/a a E ;
- $E = \{a = b\}$ estratta $Y = a$, si applica Y/a a E ;
- resta $a = b$ ma le costanti a e b non sono unificabili quindi si restituisce $\perp$.

La *procedura TD* proposizionale può essere estesa per gestire le variabili ammettendo istanze di regole nella derivazione.

Una **clausola di risposta generalizzata** avrà la forma:

$$yes(t_1, \dots, t_k) \leftarrow a_1 \wedge a_2 \wedge \dots \wedge a_m$$

dove i $t_1, \dots, t_k$ sono termini e gli $a_1, \dots, a_m$ sono atomi.

L'uso del predicato *yes* consente l'*estrazione della risposta*, ossia determinare quali istanze delle query (con variabili) seguano logicamente da KB .

Data la query q , inizialmente si considera la clausola di risposta:

$$yes(V_1, \dots, V_k) \leftarrow q$$

dove $V_1, \dots, V_k$ sono le variabili della query q . Intuitivamente, un'istanza di $yes(V_1, \dots, V_k)$ risulta vera se la corrispondente istanza di q è vera.

In ogni fase successiva, l'algoritmo:

1. Seleziona un *atomo* nel *corpo* della clausola di risposta corrente;
2. Sceglie una *clausola* di KB la cui *testa* si unifichi con tale atomo.

Nella **risoluzione SLD** della clausola di risposta generalizzata

$$yes(t_1, \dots, t_k) \leftarrow a_1 \wedge a_2 \wedge \dots \wedge a_m$$

avendo *selezionato* a_1 , se la clausola *scelta* da KB è

$$a \leftarrow b_1 \wedge \dots \wedge b_p$$

con a_1 e a unificabili da un MGU σ , la nuova clausola di risposta (*risolvente*) risulterà:

$$(yes(t_1, \dots, t_k) \leftarrow b_1 \wedge \dots \wedge b_p \wedge a_2 \wedge \dots \wedge a_m) \sigma$$

Ripetendo passi di risoluzione si ottiene una **derivazione SLD**, ossia una sequenza $\gamma_0, \gamma_1, \dots, \gamma_n$ di clausole di risposta in cui:

- γ_0 è una clausola di risposta corrispondente alla query originaria 1;
date $V_1, \dots, V_k$ variabili libere di q :

$$yes(V_1, \dots, V_k) \leftarrow q$$

- Ogni γ_i si ottiene per risoluzione di SLD:
 - Selezionando a_1 nel corpo di γ_{i-1} ;
 - Scegliendo della base una *copia* della clausola scelta

$$a \leftarrow b_1 \wedge \dots \wedge b_p$$

la cui testa a si unifichi con a_i tramite un MGU σ ;

- Sostituendo a_1 con il corpo $b_1 \wedge \dots \wedge b_p$;
 - Applicando σ alla clausola di risposta risultante.
- γ_n sarà la **risposta** dalla forma:

$$yes(t_1, \dots, t_k) \leftarrow$$

in una dimostrazione terminata con *successo*. In risposta alla query si avrà:

$$V_1 = t_1, \dots, V_k = t_k$$

dove i t_i sono i termini determinati dai binding delle variabili della query V_i .

Simboli di Funzione e Strutture Dati

Il DATALOG richiede un nome (simbolo di costante) per ogni individui sul quale si debba ragionare.

Spesso risulta più semplice identificare un individuo indirettamente, ossia *in termini di altri individui*.

Considerando una costante per individuo, si può rappresentare solo un numero *finito* di individui fissato alla costruzione della base di conoscenza. Come fare per ragionare su un dominio potenzialmente *infinito*? Ci vengono in aiuto la *programmazione logica* e, in particolare, il linguaggio PROLOG.

Le funzioni consentono di descrivere individui indirettamente, in termini di altri individui. Alla sintassi si aggiunge la nozione di *simbolo di funzione*, una parola che cominci con una lettera minuscola. Quindi il linguaggio viene *esteso* nella nozione di **termine** che potrà essere una variabile, una costante o avere la forma

$$f(t_1, \dots, t_n)$$

dove f è un simbolo di funzione (n -aria) e i t_i sono termini.

Nelle clausole i termini posso comparire *esclusivamente* all'interno di atomi: le funzioni sono distinte dai predicanti.

In ogni interpretazione

$$\langle D, \phi, \pi \rangle$$

la ϕ dev'essere estesa: si assegna una funzione $D^n \mapsto D$ a ogni simbolo di funzione n -aria: essa, per ogni termine ground, deve specificare l'individuo denotato.

Le costanti possono essere viste come funzioni 0-rie, ovvero senza argomenti.

Qualunque funzione computabile può essere calcolata usando una base di clausole con simboli di funzione ossia è interpretabile come **programma logico** (adottando una *semantica operativa*).

Il linguaggio dei programmi logici è *Turing equivalente*. Con un solo simbolo di funzione e una sola costante, si può definire un numero *infinito* di termini o atomi ground che consente di ragionare su un numero di individui potenzialmente *infinito*. Inoltre, con le funzioni si possono definire le tipiche **strutture dati** per i linguaggi di programmazione come, ad esempio, liste, alberi, ecc.

Per gestire i termini con funzioni vanno estese le procedure definite per il DATALOG. Il problema principale è che il numero di termini possibili è *infinito*.

Nel definire una *procedura BU* sulla base di quella per la logica proposizionale, se si operasse preventivamente il *grounding* delle clausole, come visto in precedenza, una clausola potrebbe generare un'*infinita* sequenza di conseguenze sulla base del numero di termini formabili usando le funzioni.

Una procedura ingenua di grounding potrebbe innescare un problema di **starvation**, con clausole che vengano sistematicamente trascurate.

Va pertanto considerata una *proprietà* come la **fairness** del criterio di selezione delle clausole da elaborare che assicura che ogni clausola della base prima o poi debba essere selezionata. La procedura BU con un criterio di selezione *equo* (*fair*) risulta *completa*: ogni conseguenza potrà essere generata.

Per estendere la *procedura TD* per il DATALOG con il trattamento di termini costruiti da funzioni, occorre una procedura di *unificazione* che possa seguire ricorsivamente la struttura dei termini in profondità, considerando anche le applicazioni di funzioni a diversi livelli (di composizione). Nell'algoritmo esteso per trovare MGU, una variabile X non potrà unificarsi con termini t (con $t \neq X$) in cui essa occorra come, ad esempio, $g(f(X))$. Serve quindi un **controllo di occorrenza** (*occurs-check*) aggiuntivo, senza del quale la procedura non risulterebbe più corretta (*sound*).

In un *algoritmo di unificazione* con simboli funzione e *occurs-check*, avendo selezionato $\alpha = \beta$, si restituisce $\perp$ se α è una variabile e β è un termine che la contiene essendo diverso da α stesso (o viceversa).

Omettendo il controllo, tipicamente per aumentarne l'efficienza, la procedura non sarà più corretta. Una procedura non sound potrebbe derivare un goal g anche quando questo non fosse una conseguenza logica della base di conoscenza.

Abbreviazioni ammesse nella notazione del Prolog:

- $[]$ o *nil* denotano la lista *vuota*;
- $[E | R]$ è la lista con primo elemento E e resto della lista R , scritta anche con $\text{cons}(E, R)$;
- *Semplificazione*: $[X | [Y]]$ si può scrivere $[X, Y]$, se Y è una sequenza di valori
 - Ad esempio: $[a]$ per $[a | []]$, $[b, a]$ per $[b | [a | \text{nil}]]$ e $[a, b | C]$ per $[a | [b | C]]$.

Uguaglianza

A volte è utile usare più termini per denotare uno *stesso* individuo: ad esempio i termini $3*5$, $273-258$ e 15 possono denotare lo stesso numero. Altre volte serve che ogni nome si riferisca a un diverso individuo: ad esempio un nome distinto per insegnamenti differenti.

Spesso non si sa se due nomi denotino lo stesso individuo: ad esempio se *postino_delle_8* e *postino_delle_15* siano la stessa persona. Fino ad ora negli esempi di ragionamento in DATALOG tutte le risposte erano valide a prescindere dal fatto che i termini denotassero individui diversi o meno.

Può essere utile considerare il predicato speciale “=” avente un'interpretazione intesa standard *indipendente* dal dominio da rappresentare. Dati due termini t_1 e t_2 , si può scrivere l'atomo (in *forma infissa*):

$$t_1 = t_2$$

da leggersi t_1 **uguale** a t_2 , sarà vero in un'interpretazione I se e solo se t_1 e t_2 in I denotano lo stesso individuo.

Non si tratta di semplice similarità o identità ma di *identità*: due nomi (costanti) per un solo individuo, e non due individui simili (o indistinguibili) del dominio.

Spesso è utile poter concludere che termini distinti debbano denotare lo *stesso* individuo, ad esempio *chairOnRight* = *chair2*.

Se non si ammettono atomi con predicato d'uguaglianza come teste delle clausole, un termine può essere uguale solo a sé stesso in ogni interpretazione. Per poter derivare relazioni d'uguaglianza, occorrono clausole con questo tipo di atomi nella testa. Le alternative possibili sono:

1. *Assiomatizzare* l'uguaglianza come un qualunque altro predicato: aggiungere clausole che codifichino l'interpretazione del predicato $=$;
2. Definire *procedure speciali* d'inferenza per l'uguaglianza: attraverso regole di riscrittura, si mappano termini diversi che denotino uno stesso individuo su una forma standard (canonica).

In alternativa all'assiomatizzazione dell'uguaglianza si può adottare una *convenzione*. Nella semantica delle clausole definite data in precedenza, la funzione ϕ non è necessariamente iniettiva; quindi, termini *diversi* possono denotare lo stesso individuo o individui distinti.

Con l'**assunzione di unicità del nome**, o *Unique Names Assumption* (UNA), termini ground diversi denotano individui distinti. Tale assunzione è tipica del contesto delle basi di dati. Per ogni coppia di termini ground distinti t_1 e t_2 , si assumerà:

$$t_1 \neq t_2$$

dove $\neq$ significa "non uguale a" e corrisponde ad assumere la verità di $\neg(t_1 = t_2)$ in ogni interpretazione. Sotto UNA, un atomo con $\neq$ può occorrere nel *corpo* delle clausole.

Quindi sotto UNA, termini ground sono *identici* solo se si unificano. Ciò non vale per termini non ground, ad esempio l'atomo $a \neq X$ ha istanze vere nelle quali X assume un certo valore, diciamo b , e un'istanza falsa in cui X assume il valore di a . La UNA è utile a integrare conoscenza proveniente da database, senza essere costretti ad esplicitare troppe distinzioni, come ad esempio quelle tra le persone con nomi diversi $kim \neq sim, kim \neq chris, chris \neq sam, \dots$

Tuttavia, la UNA risulta *inappropriata* in certi casi, come ad esempio: $2 + 2 \neq 4$ oppure $clar_kent \neq superman$.

Assunzione di Conoscenza Completa

Come visto in precedenza per la Logica proposizionale, l'**assunzione di conoscenza completa** consente di derivare conclusioni altrimenti non ammissibili rispetto alla semantica standard della Logica. Così un enunciato che *non segue* logicamente dalla base di conoscenza potrà essere considerato falso.

Come visto in precedenza, questo consente anche di definire una forma di **negazione per fallimento** (NAF).

Per estendere l'assunzione di conoscenza completa al caso dei programmi logici con variabili e funzioni si devono considerare: *assiomi d'uguaglianza*, la proprietà di *chiusura del dominio* e una nozione più sofisticata di *completamento*.

Dato un predicato p , la clausola

$$p(t_1, \dots, t_k) \leftarrow B$$

in **forma normale di Clark** diventa

$$p(V_1, \dots, V_k) \leftarrow \exists W_1 \dots \exists W_m V_1 = t_1 \wedge \dots \wedge V_k = t_k \wedge B$$

dove, se la clausola è atomica allora B sarà *true*, le variabili $W_1, \dots, W_m$ sono quelle originarie della clausola mentre le variabili $V_1, \dots, V_k$ sono *nuove variabili* aggiuntive.

Ponendo tutte le clausole per il dato predicato p in forma normale, usando uno stesso insieme di nuove variabili, si può scrivere:

$$\begin{aligned} p(V_1, \dots, V_k) &\leftarrow B_1 \\ &\vdots \\ p(V_1, \dots, V_k) &\leftarrow B_n \end{aligned}$$

riassumibili in

$$p(V_1, \dots, V_k) \leftarrow B_1 \vee \dots \vee B_n.$$

Tale enunciato è equivalente all'insieme di clausole originario.

Il **completamento** del predicato p sarà la formula logica:

$$\forall V_1, \dots, \forall V_k p(V_1, \dots, V_k) \leftrightarrow B_1 \vee \dots \vee B_n$$

dove, nei corpi, la **negazione per il fallimento** ($\sim$) viene sostituita dalla negazione logica standard ($\neg$). Dal punto di vista della *semantica*, $p(V_1, \dots, V_k)$ sarà vero se e solo se è vero almeno uno dei corpi B_i .

il **completamento** di una base di conoscenza comprende i completamenti di ogni predicato che occorre nella base.

Sotto l'assunzione di conoscenza completa, si possono definire relazioni per la cui definizione le clausole definite non sono sufficientemente espressive.

Risulta problematico adottare una procedura TD che possa trattare oltre a variabili e funzioni anche la NAF:

Esempio — Si consideri la base di conoscenza:

- $p(X) \leftarrow \sim q(X) \wedge r(X)$.
- $q(a)$.
- $q(b)$.
- $r(d)$.

e la query **ask** $p(X)$.

Per la semantica, ci sarebbe una sola risposta: $X = d$

- essendo $r(d)$ e $\sim q(d)$ vere, $p(d)$ segue logicamente dalla base di conoscenza.

Ma la dimostrazione di $p(X)$ con la procedura TD *fallisce*:

- selezionato $\sim q(X)$, la procedura riesce a dimostrare $q(X)$ con $\{X/a\}$;
- mentre $p(X)$ vero se $\{X/d\}$, essendo $\sim q(d)$ e $r(d)$.

Questo mostra che la TD risulta *incompleta* e *non corretta*:

- ad esempio, aggiungendo alla base di conoscenza la clausola $s(X) \leftarrow \sim q(X)$., dal fallimento di $q(X)$ si arriverebbe a poter derivare *erroneamente* $s(X)$.

Per evitare i problemi causati da *variabili libere nei goal negati* la procedura dovrebbe *posticipare* tali sotto-obiettivi fino a quando le variabili libere non vengono *legate* (in un binding).

Qualora ciò non fosse possibile in alcun modo, la dimostrazione del goal *si inceppa* (**flounders**) e non si potrà concludere *nulla* su tale goal.

Esempio — Data la base di conoscenza:

- $p(X) \leftarrow \sim q(X)$
- $q(X) \leftarrow \sim r(X)$
- $r(a)$

con completamento:

- $p(X) \leftrightarrow \neg q(X)$

- $q(X) \leftrightarrow \neg r(X)$
- $r(X) \leftrightarrow X = a$

e la query: **ask** $p(X)$.

Sostituendo $X = a$ ad $r(X)$, si ha $q(X) \leftrightarrow \neg(X = a)$ quindi: $p(X) \leftrightarrow X = a$

La risposta giusta sarebbe $X = a$ ma il differimento del goal non aiuterebbe a trovarla. La procedura dovrebbe essere capace di *analizzare i casi* nei quali il goal è fallito (*argomento avanzato* per il quale si rimanda a trattazioni più complete).

6 – Knowledge Graph e Ontologie

Knowledge Graph

Dato un *linguaggio* di rappresentazione logico e un *mondo* sul quale ragionare, i progettisti devono scegliere *individui* e *relazioni*, ossia come decomporre il dominio d'interesse per formalizzarne un suo modello. Il livello di dettaglio dipende dal compito da svolgere.

Ad esempio, la proprietà di essere di colore rosso, *red*, ascrivibili a diversi oggetti del mondo reale, può essere rappresentata in base allo *spettro di frequenze* della luce assorbite o riflesse. La proprietà potrebbe indicare un certo insieme o intervallo di frequenza con una mappatura su termini, come pink, scarlet, ruby e crimson. In alternativa, lo spettro può anche essere diviso in regioni che, pur non corrispondendo a termini precisi del linguaggio comune, risultano più utili a distinguere diverse categorie di individui.

Le *relazioni* andranno definite seguendo alcune *linee guida* ingegneristiche:

Esempio — *red* può essere una categoria di classificazione appropriata per gli individui rappresentata come *relazione unaria*:

- “l'oggetto *a* è rosso” si traduce con *red(a)*;
- una query possibile sarebbe: “Cosa si conosce di colore rosso?”
traducibile in: *ask red(X)*
 - i valori di *X* restituiti rappresentano individui di colore rosso;
- con questa rappresentazione, però, è difficile chiedere “Di che colore è l'oggetto *a* ?”
Nel linguaggio delle clausole definite, non si può chiedere *ask X(a)*. I nomi dei predicati non possono essere variabili. Ciò diventa possibile in logiche di ordine superiore per qualsiasi proprietà di *a*.

Si può pensare a una forma di **reificazione**, ossia alla riduzione a oggetto di un concetto astratto, ad esempio la relazione fra un individuo e il suo colore; quindi, il colore stesso può essere modellato come *individuo*:

- Usando la costante *red* per denotare il colore rosso che diventa un individuo referenziabile come gli altri;
- Definendo un predicato *color(Ind, Val)*, dove *Val* è il colore dell'individuo *Ind*, si ha una relazione binaria tra individui fisici e i loro colori: ad esempio “l'oggetto *a* è rosso” si può scrivere *color(a, red)*;
- Si potrà allora chiedere: “Cosa è di colore rosso?” con la query
ask color(X, red)
e anche “Di che colore è l'oggetto *a*?” con
ask color(a, C)

Però *color* come predicato binario non consente domande come “*Quale proprietà di questo oggetto ha valore red?*”, con risposta *color*.

Il predicato ternario *prop* avrà la forma:

$$prop(Ind,, Prop, Val)$$

con l'individuo *Ind* avente proprietà *Prop* con valore *Val*.

In alternativa si può considerare la struttura di **tripla** $\langle s, p, o \rangle$ con:

- *s* **soggetto**;
- *p* **predicato verbale**;
- *o* **oggetto**.

che può essere scritta anche come *enunciato*:

$$soggetto \quad verbo \quad oggetto$$

equivalente all'atomo $prop(soggetto, verbo, oggetto)$ che, in notazione *funzionale*, si può scrivere anche $verbo(soggetto, oggetto)$.

In una tripla, il *verbo* è una **proprietà** *p* con un **dominio**, l'insieme di individui che possono essere *soggetti* in triple con verbo *p* e un **codominio**, l'insieme di valori che possono essere *oggetti* in triple con verbo *p*.

Un **attributo** è una coppia *proprietà-valore*, ad esempio un certo plico ha *colore rosso*.

Esempio — Per trasformare in forma di tripla l'atomo *parcel(a)*:

1. *parcel* può essere rappresentato come *concetto* (reificato come valore):

$$prop(a, type, parcel).$$

- l'individuo *a* è nella classe *parcel*;
- *type* è una proprietà speciale che collega individui a classi, spesso indicata con *is_a* (ossia relazione di appartenenza $\in$ della matematica);
- la costante *parcel* denota una *classe* di individui: l'insieme di tutte le cose, reali o potenziali, che sono plichi;

2. *parcel* può essere rappresentato come *proprietà booleana*, con valore *true* per soggetti che sono plichi:

$$prop(a, parcel, true).$$

- una *proprietà booleana* ha codominio $\{true, false\}$, dove *true* e *false* sono costanti.

Le relazioni con specifiche proprietà possono essere rappresentate con parole, numeri, ad esempio #prenotazione, #volo, #ordine.

Esempio — L'azione del verbo “dare” coinvolge tre attori, detti *agente*, *ricevente* e *paziente*. Data la frase “Alex ha dato un libro a Chris”, una sua rappresentazione relazionale possibile è:

- *`gave(alex, chris, book)`*

Introducendo una costante come *`ga3545`* per un'azione specifica (*`giving_act`*), si può scrivere in forma di triple:

- *`prop(ga3545, type, giving_act)`*.
- *`prop(ga3545, agent, alex)`*.
- *`prop(ga3545, patient, b342)`*.
- *`prop(ga3545, recipient, chris)`*.
- *`prop(b342, type, book)`*.

dove agente/ricevente/paziente sono *relazioni tematiche* ossia **ruoli semantici**.

La reificazione consente di aggiungere altre proprietà, come ad esempio la data dell'evento, il prezzo del libro, ...

Gli individui vanno denotati usando identificatori univoci. Ad esempio, nel contesto universitario si usa la matricola per riferirsi univocamente a uno studente, essendo il solo nominativo meno adeguato a evitare casi di omonimia.

Nel contesto del web, si può usare uno **Uniform Resource Identifier** (URI), o anche un **Internationalized Resource Identifier** (IRI), come univoco per identificare qualsiasi **risorsa**, ossia qualunque cosa debba essere denominata. La forma tipica è simile a quella dello *uniform resource locator* (URL) con prefisso `http://` o `https://`.

Nel caso dell'URI, però, si denota un'entità, non un sito web: s'intende l'individuo denotato da esso. Il suo significato è dato dal suo uso, non da una definizione formale.

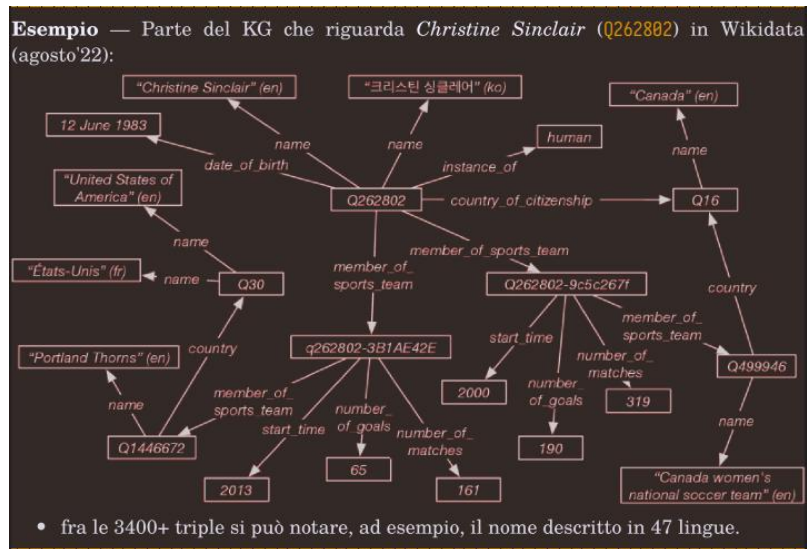
Esempio — Si consideri Wikidata, un knowledge graph aperto e collaborativo con miliardi di triple che descrivono 130M entità (statistiche del 2024).

- *Christine Sinclair* è una famosa calciatrice canadese, il cui URI in Wikidata è <http://www.wikidata.org/entity/Q262802> (abbreviabile con *`Q262802`*)
- la proprietà *paese di cittadinanza*, contraddistinta dall'URI <http://www.wikidata.org/prop/direct/P27>, ha come soggetto una persona e come oggetto il paese di cui ha la cittadinanza; ad esempio la frase “*Christine Sinclair* è cittadina canadese” si traduce nella tripla:
 - <http://www.wikidata.org/entity/Q262802>
<http://www.wikidata.org/prop/direct/P27>
<http://www.wikidata.org/entity/Q16>.
- analogamente si può usare la proprietà definita in schema.org per dare un *nome* al soggetto è <http://schema.org/name>: il valore nella tripla (oggetto) è una coppia stringa+lingua, ad esempio: (“*Christine Sinclair*”, en).

In questo esempio avremmo: *prop(Q262802, P27, Q16)*. Con l'utilizzo di schema.org potremmo usare *prop(Q262802, schema:name, (“Christine Sinclair”, en))*. La lingua è utile nei casi, ad esempio, di città (“Roma” it, “Rome” en, “Rome” fr).

La relazione *prop* può essere interpretata in termini di *grafo orientato*: l'atomo *prop(Ind, Prop, Val)* può essere raffigurato con i *nodi Ind* e *Val* congiunti da un *arco orientato* etichettato con *Prop*.

Il grafo risultante dalla rappresentazione di tali relazioni sarà un **knowledge graph** o **rete semantica**, con una mappatura diretta su una base di conoscenza tramite *prop.*



I vantaggi della notazione grafica sono:

- La facilità di visualizzazione delle relazioni senza essere costretti a imparare la sintassi di un dato linguaggio logico;
- La possibilità di ignorare le etichette con nomi privi di significato (es. Q262802-9c5c267f in figura o ga3545 in un esempio precedente): sono ammessi i cosiddetti nodi **blank**, ossia privi di etichetta, che possono servire a dare nomi arbitrari solo in caso di mappatura su un formalismo logico;
- L'efficienza della memorizzazione in un **triple store**: mentre un RDB ha bisogno della progettazione delle chiavi usate per gli indici, un triple store utilizza esattamente 8 indici per ritrovare le triple nelle diverse posizioni: una query basta a determinare se una particolare tripla sia in memoria, mediante semplice enumerazione, e sono possibili casi di query con soggetto, verbo e oggetto eventualmente non specificati.

Classi e Proprietà

Tipicamente, di un dominio si conoscono e formalizzano il database di *fatti* e le *regole* generali tramite le quali è possibile derivare altri fatti. Le modalità con cui farlo costituisce una scelta progettuale.

Si distingue la **conoscenza primitiva** specificata esplicitamente e la **conoscenza derivata**, inferita da quella primitiva e/o altra conoscenza derivata.

Le regole permettono una rappresentazione più *compatta* della conoscenza: dato che non tutto è osservabile, attraverso relazioni derivate si possono trarre conclusioni a partire da osservazioni. La conoscenza su un dato dominio sarà in gran parte *inferita*, oltre che da fatti osservati e da conoscenza più generale disponibile o anche appresa.

Grazie alle regole si potranno raggruppare gli individui in classi ed anche associare *proprietà* generali alle classi, in modo che i loro individui le *ereditino*. In tal modo si hanno rappresentazioni più *concise*: le istanze di una classe condividono attributi comuni, senza necessità di specificarli caso per caso.

Una **classe** è un insieme di individui che sono sue istanze, sia *effettivi* che *potenziali*. Essa viene definita in una delle due forme:

- *Intensionale*, ossia tramite una **funzione caratteristica** che avrà valore 1/true per i membri dell'insieme e 0/false per gli altri individui;
- *Estensionale*, ossia come elenco delle sue istanze.

Ad esempio, la classe Chair sarà l'insieme di tutte le cose che possono essere *sedie*, non limitato a quelle già osservate, per non escludere sedie ancora da inventare/produrre.

Pertanto, l'*equivalenza* tra classi non dipende dalle sole istanze conosciute. Ad esempio, la classe degli unicorni verdi e quella delle sedie alte 120m, pur contenendo entrambe nessun elemento rimangono diverse.

La definizione data consente di descrivere *qualsiasi* insieme come classe: ad esempio una class e che comprende il numero 18, il Dipartimento_di_Informatica_di_Bari, e il primo_goal_della_SerieA,_23_24. In tal caso, per, bisogna chiedersi se tale classe sia effettivamente *utile*.

Occorrerà cercare di definire una classe come **tipo naturale**, ossia dovrebbe ammettere una descrizione dell'insieme più concisa. Ad esempio, la classe mammifero è un tipo naturale: descrive attributi comuni ai mammiferi e rende più compatta la KB, evitando ripetizioni per ogni singolo individuo.

S è una **sottoclasse** di *C* se fra di esse intercorre una relazione di *sottoinsieme*: ogni individuo di tipo *S* è anche di tipo *C* (anche quelli futuri!).

La relazione tra tipi e sottoclassi è definibile come *clausola definita* nel seguente modo:

$$prop(X, type, C) \leftarrow prop(S, subClassOf, C) \wedge prop(X, type, S)$$

o, passando alla *notazione funzionale*, usando le *proprietà speciali type* e *subClassOf*:

se *subClassOf*(S, C) e *type*(E, S) allora *type*(E, C)

Per definire l'**ereditarietà di proprietà**, il valore di una proprietà, specificato a livello di classe, sarà ereditato da tutte le sue istanze. Per specificare che tutti gli individui di classe c hanno il valore v per la proprietà p , in DATALOG si può scrivere:

$prop(Ind, p, v) \leftarrow prop(Ind, type, c)$

che, insieme alla regola precedente che collega tipi e sottoclassi, serve a far ereditare (valori di) proprietà.

Le **proprietà** rappresentano relazioni *binarie* p con:

- **Dominio**: la classe C dei *soggetti* delle triple con p come verbo, per cui se $p(x, y)$ allora $x \in C$, quindi se p ha dominio C_1 e dominio C_2 , allora ogni soggetti apparterrà sia a C_1 sia a C_2 ;
- **Range (codominio)**: la classe C per gli *oggetti* delle triple con p , per cui se $p(x, y)$ allora $y \in C$;
- p è **funzionale** se c'è al più un oggetto associato a ogni soggetto: se $p(x, y_1)$ e $p(x, y_2)$ allora $y_1 = y_2$; in matematica si dice che p è una *funzione (parziale)*.

Una gerarchia di ereditarietà è possibile anche fra proprietà. Si dirà che p_1 è **sotto-proprietà** di p_2 se e solo se ogni coppia in p_1 è anche in p_2 . In notazione funzionale si può scrivere $p_1(x, y)$ implica $p_2(x, y)$, o anche, usando le triple, (x, p_1, y) implica (x, p_2, y) .

La categorizzazione degli oggetti è alla base delle moderne ontologie. Storicamente già Aristotele aveva proposto un modo di definire una nuova classe C in termini di:

- **Genus**, una super-classe di C ;
- **Differentia**, gli attributi che distinguono le istanze di C dal resto delle istanze della super-classe di C .

Ad esempio, nelle ontologie, “animale” e “conoscenza” possono essere definite come *classi* e la proprietà *essere_bipede* può avere come dominio “animale”, mentre non ha senso per un'istanza di “conoscenza” avere un valore per tale proprietà.

Per costruire un'ontologia:

- Per ogni classe da definire, occorre determinare una super-classe rilevante e poi selezionare gli attributi che la distinguono dalle altre sottoclassi. Per ogni attributo si definirà una proprietà e un valore;

- Per ogni proprietà, si definiscono come dominio e range le classi più generali che abbia senso considerare, anche definendole tramite semplice enumerazione dei valori.

La *gerarchia delle classi* costituisce un grafo aciclico orientato con archi dalle sottoclassi alle rispettive super-classi immediate.

Si tratta di un *reticolo*: sono inutili definizioni con cicli, dato che si creerebbero equivalenze. Si noti che la metodologia non porta necessariamente a gerarchie ad albero, alcuni oggetti possono appartenere a più classi distinte e ogni classe non ha una sola super-classe più specifica.

Raramente le gerarchie naturali sono definibili come alberi. Storicamente imporre forzatamente una struttura ad albero ha avuto scarso successo.

Può essere definita una struttura aciclica, con una radice, detta thing:

- Definendo ogni classe in termini di una super-classe (genus) e attributi (differentia) con una *formula normale* costituita da thing congiunta con gli attributi, ottenuta espandendo ogni super-classe con la sua definizione;
- Se gli attributi sono coppie proprietà-valore allora, in forma normale, una classe è sotto-classe di un'altra per inclusione.

Ontologie e Condivisione della Conoscenza

Diversi fattori rendono complessa la costruzione di grandi sistemi basati su conoscenza. La conoscenza viene spesso progettata integrando *più sorgenti*. Queste, a loro volta, potrebbero non condividere uno stesso modo di analizzare il mondo, ognuna potrebbe semplificarlo secondo specifiche priorità originate in campi diversi e con terminologie peculiari.

Inoltre, i sistemi *evolvono* nel tempo e risulta difficile anticipare distinzioni che potranno essere utili in futuro. Il mondo appare spesso *indistinto* e i progettisti devono scegliere *individui e relazioni* da rappresentare accordandosi su una decomposizione conveniente del dominio.

Risulta difficile tenere presenti contemporaneamente sia la propria *notazione* con il suo *significato* che quella altrui. Dato un simbolo utilizzato, se ne deve saper determinare il significato ovvero, dato un concetto, occorre saper determinare con quale simbolo rappresentarlo: il concetto potrebbe essere stato già definito e associato a un certo simbolo oppure, in alternativa, occorre determinare quali siano concetti correlati con cui definirlo.

Per condividere e comunicare conoscenza è importante sviluppare un *vocabolario* con un suo *significato* comunemente accettato. Una **concettualizzazione** è l'associazione tra un vocabolario di simboli usati nella macchina e individui/relazioni del mondo che si intende rappresentare. Essa costituisce una particolare *astrazione* del mondo e la sua *notazione*.

In piccolo si tratta di una forma di documentazione curata dal progettista, spesso informale ma difficilmente scalabile. Nel caso di *grandi sistemi* occorre una concettualizzazione *condivisa*.

Un'**ontologia**, generalmente definita *indipendentemente* dalle sue applicazioni, richiede l'*accordo* di una comunità sul significato dei simboli che vi occorrono.

Componenti:

- *Vocabolario* delle categorie di cose da rappresentare: classi e proprietà;
- *Organizzazione* delle categorie, ad esempio la gerarchia d'ereditarietà definita attraverso proprietà speciali come *subClassOf* o *subPropertyOf*, o altre modalità;
- Insieme di *assiomi* che vincolano la definizione di alcuni simboli in modo da riflettere meglio il significato inteso, ad esempio la transitività d'una proprietà, restrizioni su dominio, codominio, o sul numero di valori che la proprietà può assumere per individuo; a volte alcune relazioni sono definite in termini di relazioni *primitive*, che non hanno una vera e propria definizione (intensionale).

Non serve definire altri individui ancora *sconosciuti* in fase di progettazione, bastano quelli ritenuti necessari e da condividere per la specifica applicazione come, ad esempio, i giorni della settimana o i colori.

Lo scopo *primario* delle ontologie è *documentare* il significato dei simboli, associare simboli (nella macchina) a concetti (nella mente del progettista):

- Dato un simbolo, si usa l'ontologia per determinare cosa significhi;
- Dovendo rappresentare un concetto, si sfrutta l'ontologia per trovare il simbolo appropriato o per evidenziare la necessità di una nuova definizione.

Scopi *secondari* sono l'*inferenza* o individuare le eventuali *contraddizioni* tramite l'uso degli assiomi.

Il problema principale nella progettazione è definire l'*organizzazione* dei concetti (simboli) in modo che la macchina possa inferire conoscenza utile dai fatti asseriti.

Il **Web Semantico** costituisce una prospettiva che prevede la distribuzione di *conoscenza* attraverso l'infrastruttura del Web (server): a differenza dei documenti HTML del Web, destinati alle persone, si mira a conoscenza *interpretabile anche dalle macchine*, con una semantica esplicitata attraverso la definizione di ontologie formali che permettano il ragionamento automatico.

Gli ingredienti-base sono costituiti dagli identificatori URI utilizzati in rappresentazioni/linguaggi RDF (SPARQL), RDF-Schema e OWL.

XML (Extensible Markup Language) è un linguaggio che fornisce una *sintassi* progettata per l'elaborazione automatica leggibile anche dalle persone. Si tratta di un linguaggio testuale, con elementi costituiti da *tag* che possono essere organizzati gerarchicamente. La sintassi, anche complessa, prevede al livello basilare ambiti di tag della forma `<tag.../>` oppure `<tag...>...</tag>`.

A livello *semantico* occorre adottare ontologie di riferimento, riusando quelle esistenti anche per svilupparne di nuove in proprio. Ai fini dell'*interoperabilità semantica*, si dovrebbe tendere a riusare ontologie *standard* per i domini interessati e/o definire *mapping* dalla propria verso ontologie standard.

Un **URI (Uniform Resource Identifier)** identifica *univocamente* una *risorsa*, ovvero qualsiasi cosa debba essere identificata: individui, classi e proprietà. La sintassi è basata su quella degli URL: `<url#name>` dove url è l'indirizzo di una pagina Web, con HTTP si può *negoziare* il tipo di risposta in base alla richiesta del client; si usano anche IRI, con un set-caratteri esteso (URI usa ASCII, mentre IRI Unicode/UTF-8).

Alcuni esempi sono i seguenti:

- Individuo `<http://example.org/#spiderman>`;
- Proprietà `<http://xmlns.com/foaf/0.1/name>`.

Un URI si può esprimere in forma *abbreviata*, detta *CURIE*, `abbr:name`, dove abbr è un *prefisso* dell'URI completo (*namespace*) dichiarato localmente. Ogni URI avrà un significato convenzionale condiviso determinato dal suo uso.

RDF (Resource Description Framework) definisce un *modello di dati* in forma di *triple*:

individuo-proprietà-valore

La sintassi (serializzazione) è tipicamente basata sull'XML, in tal caso si parla di RDF/XML, ma esistono altri formati più leggibili come JASON-LD, N3 o TURTLE; in quest'ultimo, si scrive:

URI_1 URI_2 Valore, oppure URI_1 URI_2 URI_3

Un esempio concreto potrebbe essere il seguente:

<https://example.org/#spiderman> <http://xmlns.com/foaf/0.1/name> "Uomo Ragno"@it

È ammessa la *reificazione di enunciati*, formule logiche arbitrarie, quindi in generale, non è decidibile. Ciò non costituisce sempre un problema: significa solo non poter limitare il tempo necessario a un dato calcolo, analogamente a quanto accade con i programmi logici quando si ammettono funzioni.

SPARQL è il nome del protocollo e del linguaggio per l'interrogazione di KB costituite da grafi di triple RDF. A tale scopo, si utilizzano specifici *endpoint*, quali, ad esempio: DBpedia, Min. Istruzione, INPS ecc...

Le query sono definite da *pattern* costituiti da triple contenenti URI o *variabili*:

```

Esempio — Testabile attraverso l'endpoint di DBpedia

select ?canzone ?data where {
  dbr:The_Sugarcubes dbo:formerBandMember ?componente .
  ?componente rdf:type yago:Female109619168 .
  ?canzone dbo:artist ?componente .
  ?canzone dbo:releaseDate ?data .
} limit 3

risultati:

http://dbpedia.org/resource/Cosmogony_(song) 2011-07-19
http://dbpedia.org/resource/Hunter_(Björk_song) 1998-10-05
http://dbpedia.org/resource/Declare_Independence 2008-01-01
  
```

RDF-S (RDF Schema) consente di definire risorse, classi e proprietà in termini di altre risorse. Ad esempio, è possibile *restringere* domini e range di proprietà. Esso fornisce anche *contenitori*: set, sequenze e alternative.

```

Esempio — Uso di RDF e RDF-S

@prefix : < http://www.example.org/sample.rdfs# >.
@prefix rdf: < http://www.w3.org/1999/02/22-rdf-syntax-ns# >.
@prefix rdfs: < http://www.w3.org/2000/01/rdf-schema# >.

:Dog      rdfs:subClassOf :Animal.
:Person   rdfs:subClassOf :Animal.
:hasChild rdfs:range :Animal;
          rdfs:domain :Animal.
:hasSon    rdfs:subPropertyOf :hasChild.

:Max      rdf:type :Dog.
:Abel     a      :Person.
:Adam     :hasSon :Abel.
  
```

I linguaggi ontologici come OWL si basano sulle **logiche descrittive** [DL] che servono a descrivere *concetti* (classi), *ruoli* (proprietà) e *individui*.

L'idea fondante è la distinzione nella *base di conoscenza* di due parti $\mathcal{K} = \langle \mathcal{T}, \mathcal{A} \rangle$:

- $\mathcal{T}$ costituisce la parte **terminologica** o **TBox** che scrive la terminologia, ossia *assiomi* per la definizione del significato dei simboli. Ad esempio, in notazione logico-matematica:

$$C \sqsubseteq (C_1 \sqcap \exists R_1. C_2) \sqcup \forall R_2. (C_3 \sqcap \neg C_4)$$

Per la descrizione in OWL degli assiomi sono possibili diverse sintassi. Essa viene definita in fase di progettazione del sistema e ne specifica l'ontologia, tende quindi, ad essere stabile, a meno che non si debba modificare il significato del vocabolario (caso raro).

La formula nell'esempio va letta come " C se C_1 AND esiste almeno una relazione R_1 avente classe C_2 OR tutte le sue relazioni R_2 portano a individui che sono in C_3 ma non in C_4 ;

- $\mathcal{A}$ costituisce la parte **asserzionale** o **ABox** che specifica verità fattuali. Ad esempio, in DL:

$$C(a), R(a, b), P(b, v)$$

facilmente descrivibili come asserzioni in forma di *triple* RDF. Esse riguardano conoscenza su situazioni contingenti, lo *stato del mondo*, di solito completamente noto solo al momento dell'utilizzo (*runtime*). L'esempio va letto come "L'individuo a è di classe C , ha una relazione R con b e b ha una proprietà P avente valore v ".

OWL (*Web Ontology Language*) è il linguaggio con cui vengono rappresentate le ontologie pubblicate attraverso il Web.

Con OWL è possibile descrivere mondi/domini in termini di:

- **Individui**: *entità* del mondo che si descrive;
- **Classi**: *insiemi di individui*, tutte le entità reali o potenziali che potrebbero appartenervi;
- **Proprietà**: *relazioni* che descrivono individui associandoli ad altri *individui*, **object property**, oppure a *dati*, *valori* di tipi predefiniti, come gli interi o le stringhe, dette in tal caso **datatype property**;

Default in OWL:

- Non si assume la *UNA*: due nomi/URI non denotano necessariamente individui o classi diversi;
- Si assume una semantica di *mondo aperto*: non si può assumere che tutti i fatti rilevanti siano stati dichiarati (*conoscenza completa*).

I costruttori di classe vanno usati negli *enunciati*, ad esempio per dire che un individuo appartiene a tale classe o che una classe sia equivalente ad un'altra.

OWL non può essere espresso, in modo equivalente, attraverso clausole definite; per dire che tutti gli elementi di S hanno il valore v per un dato predicato p , si dice che S è sottoinsieme di tutte le cose con valore v per p (*assioma di inclusione*, $\sqsubseteq$).

Fra gli altri costruttori troviamo il *costruttore di proprietà* owl:ObjectInverseOf(p) utile a rappresentare la proprietà inversa di p , denotata anche con p^{-1} :

$$p^{-1}(y, x) \text{ se e solo se } p(x, y)$$

vale solo per *object-property*, mentre per *datatype-property* no.

Infine, sono disponibili anche costruttori per definire proprietà, *commenti*, *annotazioni*, *versioning* e *import* da altre ontologie.

Una **ontologia di dominio** riguarda un particolare dominio d'interesse: molte delle ontologie esistenti riguardano un dominio ristretto definito per specifiche applicazioni.

Le **linee guida** per definire ontologie di dominio sono le seguenti:

- Se possibile, conviene usare ontologie *esistenti*: la KB potrà interagire con altre che le adottano;
- Se un'ontologia esistente *non soddisfa pienamente i requisiti*, conviene importarla e procedere con aggiunte mirate. Questo evita di duplicare il lavoro e aumenta l'interoperabilità dell'applicazione;
- Occorre assicurarsi che l'ontologia si *integri* con domini *affini*, utilizzando una terminologia coerente;
- Si raccomanda di allinearsi a **ontologie di livello superiore** (*upper-level ontologies*) per facilitare l'integrazione della conoscenza con fonti eterogenee;
- Durante la fase di progettazione, è opportuno consultare i potenziali utenti finali per garantire che il modello risponda alle reali necessità di utilizzo;
- Le classi devono essere nominate usando il **singolare** e la denominazione deve riflettere la natura dell'individuo: più è corretto asserire che un individuo r_1 "è un Resort" piuttosto che "è un Resorts";
- Quando le ontologie sono sviluppate in modo indipendente, è necessario procedere a un **allineamento** formale. L'obiettivo deve essere quello di evitare ridondanze che renderebbero l'ontologia inutilmente complicata e difficile da mantenere.

Gli editor di ontologie, come Protégé forniscono un modo per definire ontologie a un *livello di astrazione* appropriato:

- Volendo usare un concetto, ne facilitano la *ricerca* nella terminologia o più segnarne l'eventuale assenza;
- Aiutano a determinare immediatamente il *significato* di un termine;
- Permettono controlli di *correttezza* sulle ontologie, ad esempio sulla corrispondenza con l'interpretazione intesa per i termini;
- Facilitano la *riusabilità* spingendo ad usare il più possibile un linguaggio standard.

Il **Web of Data** è una *prospettiva* del Web Semantico tesa a sfruttare l'infrastruttura del Web per creare un grafo globale di sorgenti *distribuite* di dati collegati.

I principi-guida dei **Linked Data** sono i seguenti:

1. L'uso di *URI* come nomi per le cose;
2. In particolare, *URI HTTP* per permetterne la ricerca sull'infrastruttura del Web;
3. Le informazioni vengono fornite usando linguaggi *standard* quali RDF, SPARQL, ...;
4. L'uso di *link* verso URI di risorse esterne che abilitano la scoperta di altra conoscenza.

Ogni ontologia di dominio implicitamente o esplicitamente assume un'ontologia di più alto livello alla quale confrontarsi.

Tipicamente, in un'ontologia **top-level** si troveranno definizioni a un livello di astrazione molto elevato. Essa fornirà una categorizzazione di base alla quale altre ontologie (su domini diversi) potranno conformarsi esplicitamente per facilitare la loro *integrazione*.

Inoltre, consentirà a diverse applicazioni di riferirsi a più KB, ognuna delle quali potrà adottare ontologie diverse.

Progettare un'ontologia top-level è difficile ma il suo uso aiuta a connettere le diverse basi di conoscenza.

7 – Apprendimento Supervisionato

L'**apprendimento** è la capacità di sfruttare l'*esperienza* per il migliorare il *comportamento* nello svolgimento di specifici compiti:

- L'*estensione* delle abilità/dei comportamenti possibili: fare di più;
- Il miglioramento dell'*accuratezza* nei compiti svolti: far meglio;
- Il miglioramento dell'*efficienza*: fare prima.

Informalmente si può anche dire che l'obiettivo dell'apprendimento è comprendere il mondo attraverso i **dati**. Occorre modellare non tanti i dati quanto il mondo che li *genera* per consentire il miglioramento delle decisioni/azioni del sistema intelligente.

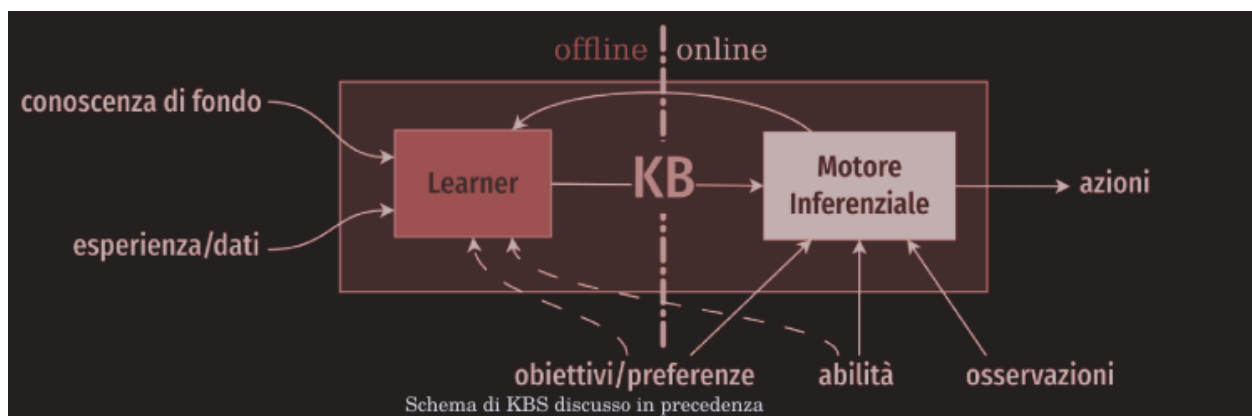
La conoscenza del passato non è fine a sé stessa, ma dev'essere utile per il futuro.

Apprendimento: Problematiche

In un *problema di apprendimento automatico* si distinguono:

- Il **task**, ovvero il *comportamento o compito* nel quale migliorare;
- I **dati**, ossia le esperienze usate per migliorare le prestazioni nello svolgimento del compito, di solito disponibili come sequenza di *esempi*;
- La **misura del miglioramento**: occorre stabilire *come* e *cosa* misurare, come ad esempio l'incremento di abilità, l'efficacia o l'efficienza nella soluzione del problema.

Ai fini della predizione, si considereranno la *conoscenza di fondo* (*background knowledge*, BK) e i *dati*, ossia l'esperienza pregressa, tipicamente in forma di esempi, ossia coppie input-output, per creare una *rappresentazione interna*, un **modello**, che potrà costruire parte della KB, da usare per future *predizioni* (su azioni/decisioni):



Nel seguito sono elencate le diverse problematiche legate alla soluzione dei problemi di apprendimento automatico:

- **Task:** il compito per cui si devono acquisire dati/esperienza.
 - **Apprendimento supervisionato:** abbiamo esempi etichettati (es. “questa foto è un gatto”). Può essere di **classificazione** (categorie discrete) o **regressione** (valori continuo, come il prezzo di una casa);
 - **Apprendimento non supervisionato:** la macchina cerca schemi da sola, senza etichette (es. raggruppare clienti simili);
 - **Apprendimento per rinforzo:** si impara compiendo azioni e ricevendo premi o punizioni (es. cane che impara un trucco).
- **Feedback:** è l’informazione che il sistema riceve per capire se sta andando bene. Nel supervisionato è il valore corretto dato dall’esperto, nel non supervisionato non è specificato un output desiderato, mentre nel rinforzo è il “premio”;
- **Misura delle prestazioni,** ossia il miglioramento dei modelli su *nuovi* dati. Tipicamente nella *valutazione* si dividono i dati in due gruppi, *training* e *test*: costruito un *modello (ipotesi)* usando gli esempi di training, se ne misura l’*accuratezza predittiva* su nuovi esempi, quelli di test. Si ottiene una misura *approssimata* delle prestazioni, dato l’insieme limitato di dati, che va ripetuta al variare delle diverse suddivisioni del campione di dati. Nel caso supervisionato, le ipotesi indotte da *campioni* della popolazione devono tendere a *generalizzare* il training set, senza farsi fuorviare da sue specificità, altrimenti risulterebbero *scarsamente predittive* rispetto a nuovi dati. Ad esempio, in un *problema di classificazione binario*, in cui si decide l’appartenenza a una classe, si potrebbe formulare facilmente un’ipotesi h_p che decide sempre la predizione positiva, tranne che per i soli negativi del training set e una ipotesi h_N che decida sempre per la predizione negativa, tranne che per i soli positivi del training set; tali ipotesi sono modelli *estremi*: per entrambe si avrebbe il *100% di accuratezza* sui dati di training ma risulterebbero in *totale disaccordo* sul resto degli esempi. Le ipotesi vanno sempre valutate su nuovi dati per determinare la loro generalità;
- **Bias:** tendenza a preferire certe ipotesi rispetto ad altre. Senza bias, come si può stabilire che un’ipotesi sia migliore di un’altra? Nell’esempio precedente le ipotesi h_N o h_p sono entrambe accurate al 100% sui dati di training: quale preferire? Serve un *fattore esterno* che prescindendo anche dai dati, in grado di ottimizzare le predizioni su nuovi esempi (h_p e h_N saranno in disaccordo su tutti gli esempi futuri). La *scelta* del bias è un problema empirico da risolvere

attraverso la pratica. Tipicamente, ci si basa su misure di *complessità* delle ipotesi;

- **Rappresentazione**, ossia la *codifica interna* dell'esperienza, la forma dell'ipotesi/del modello. La più semplice è la forma *grezza*, in cui si memorizzano le esperienze stesse, gli esempi. Una forma *compatta* è più comune perché permette la generalizzazione dei dati. La scelta fra le rappresentazioni possibili dei modelli viene detta **representation bias** ed è correlata alla scelta fra i diversi modelli, o **preference bias**. Come *linea-guida*, si sa che optando per rappresentazioni più ricche sarà più facile esprimere ipotesi generali (più predittive) ma che sono più difficili da apprendere: servono più dati per scegliere fra le più numerose ipotesi coerenti con i dati stessi. Occorre trovare dei compromessi e/o ricorrere a metodi di apprendimento della rappresentazione ottimale;
- **Ricerca**: scelta di rappresentazione e bias, l'apprendimento può essere visto come un *problema di ricerca* da risolvere attraversando lo *spazio delle ipotesi* (rappresentazioni possibili del modello), dato un **search bias**, per trovare quella che meglio si adatta agli esempi. Tipicamente gli spazi di ricerca sono molto estesi, spesso infiniti; quindi, risulta impossibile una ricerca sistematica e si deve ricorrere a forme di *ricerca locale*.
Per la definizione dell'*algoritmo*, quindi, si richiede la definizione di uno *spazio di ricerca*, di una *funzione di valutazione*, e di un *metodo di ricerca*;
- **Dati imperfetti** nelle situazioni reali, a seguito di diverse cause quali:
 - *Rumore*: feature osservate che risultano inadeguate a predire la classificazione;
 - *Dati mancanti*: per alcuni esempi, non sono disponibili i valori di alcune feature;
 - *Errori*: i valori di alcune feature risultano errati.
- **Ordine** naturale del dominio delle feature (ad esempio nello spazio, nel tempo). L'ordine permette di distinguere problemi di **interpolazione**, in cui si devono fare predizioni per casi *ricompresi* nelle regioni dei dati osservati (training), da quelli di **estrapolazione**, in cui le predizioni riguardano casi distanti dagli esempi di training. I modelli per tali problemi risultano in genere meno accurati (si dice che è "*più facile predire il passato che il futuro*") in quanto determinare il valore incognito di una feature in base a valori di input già osservati subito prima o dopo, risulta molto più facile che predire valori futuri. Per questo, ipotesi con parametri ottimizzati ai fini dell'*interpolazione* possono rivelarsi pessime su successivi problemi di *estrapolazione*;

- **Dimensionalità:** essendo gli esempi descritti in termini di feature (una *dimensione* per ciascuna) il numero di esempi possibili cresce esponenzialmente aumentando le dimensioni (*curse of dimensionality*). Essendo improbabile trovare esempi molto vicini a quelli già osservati, si determina la *sparsità* dello spazio degli esempi. Al crescere della dimensionalità, le singole dimensioni diventano poco importanti: in problemi dell'ambito medico, se gli esempi sono costituiti da immagini e altri attributi (dimensioni) la modifica di un singolo pixel non dovrebbe influenzare la diagnosi;
- **Acquisizione offline/online** degli esempi: nel primo caso tutti gli esempi sono disponibili prima dell'azione/decisione, nel secondo di essi si rendono disponibili nel tempo, nel corso del processo di learning. La rappresentazione degli esempi (conoscenza pregressa) potrebbe essere determinata prima di averli visti tutti (cosa non possibile, in genere) per cui per nuovi esempi si rende necessario un aggiornamento della rappresentazione. Nell'*active learning*, il sistema ragiona e decide sugli esempi *utili* da acquisire per svolgere il suo compito.

Fondamenti

Una **feature** (*caratteristica*) è una funzione F definita sugli *esempi*, ossia sullo spazio della loro rappresentazione:

- $F(e)$ denota il *valore* di F per l'esempio e (es. $lunghezza(e) = 123$ parole);
- Il *dominio* della feature è l'insieme dei valori che essa può assumere, ossia il codominio della funzione F (es. $dom(lunghezza) = \mathbb{N}$, o anche $\mathbb{R}$).

Il **problema** di apprendimento **supervisionato** si può descrivere come segue:

- **Dati:**
 - Le **feature di input** $X_1, \dots, X_m$;
 - Le **feature-obiettivo** (o **target**) $Y_1, \dots, Y_k$;
 - Un (multi)insieme di **esempi** $e = (x_e, y_e)$ con
 $x_e = (X_1(e), \dots, X_m(e))$ e $y_e = (Y_1(e), \dots, Y_k(e))$.
- **Determinare un predittore**, ossia una funzione atta a predire i valori delle Y_j , dati i valori delle X_i , per esempi non osservati, ossia *nuovi* rispetto a quelli dati.

Come accennato in precedenza, vi sono diverse varianti dei problemi di apprendimento supervisionato:

- p. di **regressione**: predire un valore target continuo, sottoinsieme dei numeri reali;
- p. di **classificazione**: predire un valore target ricompreso in un insieme finito prefissato, ad esempio binario/booleano $\{false, true\}$, o a valori multipli, come le misure $\{XS, S, M, L, XL, \dots\}$, anche non ordinati;
- p. **relazionale**: predire relazioni, come ad esempio chi sia la madre di una data persona; in tal caso i nuovi esempi per la valutazione saranno tratti da un'altra popolazione rispetto a quelli usati nell'addestramento;
- p. **strutturale**: predire una struttura, es. la forma di una molecola.

Esempi di applicazioni dei modelli supervisionati:

- Predire *attività* (es. camminare, correre ecc...) in base alla frequenza cardiaca e dal movimento così come rilevati da uno *smart-watch*;
- Predire la *probabilità* di inondazione di un sito nella prossima decade in base ai dati topografici, climatici, geologici, sull'uso del suolo (a fini assicurativi o della concessione di permessi edilizi);
- Predire *parole scritte a mano* su smartphone/tablet in base alla grafia (ordine del tratto);
- Predire *testo tradotto* in Indonesiano corrispondente a uno in Swahili.

Dato il *predittore* $\hat{Y}$ per la feature target Y , $\hat{Y}(e)$ denoterà una **stima puntuale** del valore di Y per l'esempio $e \in Es$, ossia la predizione del vero valore $Y(e)$ detto *ground truth*; il predittore $\hat{Y}$, appreso sulla base degli esempi di training, potrà essere applicato in seguito a qualsiasi altro esempio.

Si dice **loss** (o *errore*) di $\hat{Y}(e)$ una misura della *distanza* della stima $\hat{Y}(e)$ dal vero valore $Y(e)$. Essa viene definita come funzione $loss(p, a)$ su $\mathbb{R}_+$: errore della predizione di p quando il valore effettivo (*actual*) è a . Tipicamente se ne considera la media sull'insieme Es di esempi, o **mean loss**:

$$\frac{1}{|Es|} \sum_{e \in Es} loss(\hat{Y}(e), Y(e))$$

o, in alternativa, la semplice sommatoria (senza $1/|Es|$) che ha lo stesso punto di minimo. Si preferisce la mean loss perché può essere interpretata senza conoscere il numero di esempi $|Es|$.

In caso di feature con valori *totalmente ordinati*, le differenze sono significative (anche nel caso di valori interi). Ad esempio, altezze in cm, voti, numero bambini, taglie (mappate su interi).

Le funzioni di *loss* più utilizzate in problemi di *regressione*, ossia con $a, p \in \mathbb{R}$, sono elencate di seguito:

- **Errore 0-1 o loss L_0 :**

$$\text{loss}(p, a) = \begin{cases} 1 & p \neq a \\ 0 & p = a \end{cases}$$

anche con la funzione $1(p \neq a)$ che vale 1 se l'argomento è vero e 0 altrimenti. La *loss media* su un dataset sarà il numero medio di predizioni sbagliate, che non tiene conto di *quanto* siano errati. L'**accuratezza** ne costituisce il complemento a 1, ossia il numero medio di predizioni corrette e viene massimizzata minimizzando la *loss*. Il test d'uguaglianza non è affidabile nel confronto fra numeri reali, quindi, non è utile calcolare la *loss media*, mentre potrebbe esserlo il calcolo di moda o mediana;

- **Errore assoluto o loss L_1 :**

$$\text{loss}(p, a) = |p - a|$$

È sempre non negativa e per un dataset risulta *nulla* se tutte le predizioni sono corrette; a differenza della precedente, tiene conto della *precisione* delle predizioni;

- **Errore quadratico o loss L_2 :**

$$\text{loss}(p, a) = (p - a)^2$$

Si tratta di un errore amplificato. Ai fini della minimizzazione, si considera la *loss media quadratica* su Es o, equivalentemente, la **radice dell'errore quadratico medio** (*root-mean-square error*, RMSE) che ha gli stessi punti di minimo delle sommatorie sul dataset; spesso si antepone il fattore $\frac{1}{2}$ per semplificare le derivate;

- **Errore nel caso pessimo o loss L_∞ su Es :**

$$\text{loss}(Es) = \max_{e \in Es} |\hat{Y}(e) - Y(e)|$$

Rappresenta una stima pessimistica basata sull'errore massimo del predittore quindi non c

Una feature **categorica** Y ha un dominio finito $D = \{v_1, \dots, v_k\}$. Ad esempio, in base alla rappresentazione dei dati potrebbe rappresentare l'argomento principale di un articolo, la specie di un animale, il paese d'origine di una persona, il numero rappresentato nell'immagine di una cifra scritta a mano.

La tipica *stima puntuale* per tale tipo di feature è la **predizione certa** di uno fra i possibili valori $v_1, \dots, v_k$, ovvero una tupla binaria con un solo 1 nella posizione corrispondente (*one-hot encoding*). In alternativa, è possibile una **predizione probabilistica**, tramite una funzione p (o dizionario) che associ D a numeri reali non negativi in modo tale che $\sum_{v \in D} p[v] = 1$, dove $p[v]$ è valore della predizione di p per v ; la predizione certa può essere definita equivalente se $\exists j : p[v_j] = 1$ e p è nulla per gli altri valori.

Al caso categorico possono essere applicate le stesse loss del caso continuo.

Per $Y(e) = a$:

$$\text{loss}(p[a], 1)$$

essendo $p[a]$ la probabilità associata al caso corretto.

Ad esempio, l'errore quadratico per la predizione probabilistica sarà definito: $(1 - p[a])^2$.

Una tipica misura delle prestazioni è l'**accuratezza media**:

- Predizione *certa*: 1 per predizione corrette, nulla altrimenti;
- Predizione *probabilistica*: 1 quando c'è un'unica moda in corrispondenza al valore effettivo (v con $\max p[v]$ associata). Dipendendo solo dalla moda, l'accuratezza risulta una misura approssimativa per questo tipo di predizione.

La distribuzione p può essere ottimizzata anche usando la **log-loss (categorica)** o *entropia incrociata*:

$$\text{logloss}(p, a) = -\log p[a]$$

misura sempre non negativa, nulla se $p[a] = 1$ con le altre a 0. Dato il dataset Es , la **log-loss media** del predittore $\hat{Y}$ sarà:

$$-\frac{1}{|Es|} \sum_{e \in Es} \log \hat{Y}(e)[Y(e)]$$

dove $\hat{Y}(e)[Y(e)]$ è il valore della predizione $p = \hat{Y}(e)$ per l'effettivo $a = Y(e)$, ovvero è la probabilità p assegnata dal modello su a .

Il miglior predittore è un modello per il quale si massimizza la probabilità dei dati osservati, ossia la *verosimiglianza* degli esempi E_s dato $\hat{Y}$:

$$\prod_{e \in E_s} \hat{Y}(e)[Y(e)]$$

assumendo che gli esempi fra loro indipendenti. Passando al logaritmo, si ottiene la cosiddetta **log-likelihood**:

$$\sum_{e \in E_s} \log \hat{Y}(e)[Y(e)]$$

Quindi la *log-loss media* si ottiene cambiando il segno alla log-likelihood e dividendo per il numero di esempi. Essa sarà minima quando la log-likelihood (o la likelihood) è massima.

La log-loss media risulta appropriata per predizioni nel ragionamento sotto incertezza: ad esempio, sotto loss quadratica, i valori 10^{-7} e 10^{-6} sono molto vicini: una predizione di 10^{-7} avrà un errore *simile* a quello di 10^{-6} . Invece, in termini di probabilità, rappresenterebbero casi molto *diversi*. Tale differenza può essere apprezzata adottando la log-loss ma non con le loss basate sulle differenze fra valori predetto ed effettivo.

Essendo $\log 0$ indefinito, la log-loss per la predizione di 0 è trattata come fosse infinita. Dovrebbe perciò essere scoraggiata qualsiasi predizione di probabilità nulla per un valore che invece possa essere osservato per qualche esempio futuro.

La log-loss è correlata con l'*entropia*: numero di simboli (*bit*) per codificare i dati, con un *codice* (binario) basato su $\hat{Y}$ come distribuzione di probabilità (passando a logaritmi in base e si parla di *nat*).

Si consideri il caso in cui la feature target Y abbia il dominio $\{0,1\}$ (o anche $\{false, true\}$); tale valore potrebbe indicare, ad esempio, uno degli argomenti di un articolo di giornale, usando una variabile booleana Y_k per ciascun argomento possibile.

In tal caso la predizione può essere data da un valore reale: $p = \hat{Y}(e) \in [0,1]$, il che è equivalente alla predizione categorica di p per il valore 1 (ovvero di $1 - p$ per 0).

Si definisce la **log-loss binaria**, o *cross-entropy loss binaria*, nel seguente modo:

$$\text{logloss}(p, a) = -[a \log p + (1 - a) \log(1 - p)]$$

dove p è la predizione e a il valore effettivo, ricavabile dalla verosimiglianza. Per convenzione $\text{logloss}(1,1) = \text{logloss}(0,0)$, anche $\log 0$ non è definito.

La log-loss sarà pari a $\log p$ se $a = 1$, altrimenti sarà $\log(1 - p)$, come la log-loss categorica per la quale la predizione per 1 è p e per 0 è $1 - p$. Il termine *log-loss* le ricomprende entrambe.

Si consideri il *caso* in cui la predizione sia basata solo sui valori della feature-obiettivo Y osservata per i diversi esempi del dataset, senza usare le feature di input X_i .

Si avrà un modello *costante* (ossia una **baseline** naive) per il quale si predice *uno stesso valore* $v = \hat{Y}(e)$ per tutti gli esempi e . Il valore ottimale da predire si sceglie in base al criterio da ottimizzare (la loss) e potrà essere calcolato e ottimizzato *analiticamente* sulla base degli n esempi di training.

Nel caso di Y a valori reali, il modello fornisce la stima puntuale $p = \hat{Y}(e) \quad \forall e$:

Criterio	Errore per p	Predizione ottimale
loss 0-1 media	$\frac{1}{n} \sum_e \mathbf{1}(Y(e) \neq p)$	moda
loss assoluta media	$\frac{1}{n} \sum_e Y(e) - p $	mediana
loss quadratica media	$\frac{1}{n} \sum_e (Y(e) - p)^2$	media
loss caso pessimo	$\max_e Y(e) - p $	$(\min + \max)/2$

Per una feature target Y *binaria*, i dati possono essere riassunti da due semplici statistiche $n_1 = \sum_e Y(e)$ e $n_0 = n - n_1$ (dove n è il numero totale di esempi a disposizione, n_1 rappresenta quante volte è successo “1” mentre n_0 quante volte è successo lo “0”). La media o **frequenza empirica**, ossia la proporzione di 1 nel training set, è $\frac{n_1}{n}$.

Essa costituisce una *stima di massima verosimiglianza*, quindi anche la predizione con la minima log-loss. Gli errori 0-1 ed assoluto sono minimizzati considerando il valore binario più frequente.

Criterio	Errore per p	Predizione ottimale
log-loss media	$-\left[\frac{n_1}{n} \log p + \frac{n_0}{n} \log(1 - p)\right]$	$\frac{n_1}{n}$

Nel caso di una Y *categorica*, con un dominio $\{v_1, \dots, v_k\}$, il dataset può essere sintetizzato dalle frequenze $n_1, \dots, n_k$. La log-loss sarà minimizzata dalla predizione della frequenza empirica sul training set. L'errore 0-1 viene minimizzato dalla predizione della moda, che porta anche a massimizzare l'accuratezza.

Criterio	Errore per p	Predizione ottimale
log-loss media	$-\sum_i \frac{n_i}{n} \log p_i$	$p[v_i] = \frac{n_i}{n}$

Quindi la predizione preferibile dipende dalla rappresentazione di Y e dalla misura da ottimizzare. Per esempi non ancora osservati la frequenza empirica può risultare problematica se si massimizza la verosimiglianza o minimizza la log-loss. Ad esempio, se $n_i = 0$ allora il valore v_i sarà da considerarsi mai osservato.

Un nuovo esempio di test con un valore v_i comporterebbe una verosimiglianza nulla (valore minimo) e una log-loss infinita ($\log 0$), costituendo un caso di *sovradattamento*.

riguardare – capire – vedere – rivedere meglio questa parte alla fine ↑

Gli errori non sono tutti uguali, alcuni tipi possono essere peggiori di altri in base alle loro conseguenze.

Per un esperimento si può compilare una **matrice di confusione**. Nel caso più semplice si supponga che Y sia booleana. La valutazione della predizione sarà indipendente dalla decisione.

	<i>ap</i> actual positive	<i>an</i> actual negative
<i>pp</i> predict positive	<i>tp</i> true positive	<i>fp</i> false positive
<i>pn</i> predict negative	<i>fn</i> false negative	<i>tn</i> true negative

Errori:

- *Falsi positivi* o errore di *I tipo*: predizione positive sbagliate (*true* invece dell'effettivo valore *false*);
- *Falsi negativi* o errore di *II tipo*: predizione negative sbagliate (*false* invece dell'effettivo valore *true*).

Le tipiche misure definite per un **modello predittivo binario** sono le seguenti:

- **Precisione** (*precision*): $\frac{tp}{tp+fp}$
proporzione di veri positivi (*tp*) rispetto ai casi predetti positivi (*pp*);
- **Richiamo** (*recall* o *true-positive rate*, TPR): $\frac{tp}{tp+fn}$
proporzione di veri positivi (*tp*) rispetto a tutti i positivi effettivi (*ap*);
- Analogamente *false-positive rate* (FPR): $\frac{fp}{fp+tn}$
proporzione di falsi positivi (*fp*) su tutti i negativi (*an*).

Nella scelta del modello predittivo occorrerebbe massimizzare precisione e richiamo e minimizzare il FPR, ma nella pratica tali obiettivi risultano *incompatibili*: si può massimizzare la precisione e minimizzare il FPR facendo predizioni positive solo se si

è sicuri, ma ciò abbassa il richiamo; per massimizzare il richiamo si possono ammettere previsioni più azzardate ma questo tende ad abbassare la precisione e a incrementare il FPR.

Si supponga, per esempio, che i falsi positivi rappresentino casi c volte peggiori dei falsi negativi:

$c < 1$	$c = 1$	$c > 1$
FP preferibili ai FN	parità	FP peggiori dei FN

Si tratta di costi *additivi*, quindi il **costo** di una predizione sarebbe proporzionale a $c * fp + fn$. Si dovrebbe scegliere il predittore con il costo più basso.

Nel confronto di modelli, si dice che un predittore A **domina** B se A ha più TPR e meno FPR di B . In tal caso A avrà un costo inferiore (sarà migliore) di B per tutte le funzioni che dipendono solo dal numero di FP e FN, assumendo un costi additivi non negativi.

Indipendentemente dai costi, si possono considerare lo **spazio ROC** (*receiver operating characteristic*), ossia il grafico del FPR contro il TPR e lo **spazio P-R**, ossia il grafico della precisione contro il richiamo. Ogni modello produce un punto in ciascuno spazio.

Per alcuni modelli, è possibile individuare un *iperparametro* che, variato, porta a diversi valori di TPR e FPR. Un modello che restituisce un valore reale potrebbe predire true o false in base al superamento di una *soglia* (parametro).

Tale modello può essere rappresentato con una curva dello spazio ROC al variare di tale parametro e si potrà dire che domina un altro modello se la curva è al di sopra e più a sinistra.

Se la situazione è più complessa si può ricorrere a funzioni di costo che dipendono dai due tipi di errore oppure si può adottare l'**area sotto la curva** ROC (AUROC o AUC), una misura per confrontare modello nello spazio dei parametri (sotto una particolare funzione di costo). Risulterà migliore il modello con l'AUC maggiore.

Modelli-Base

I **modelli di predizione** da apprendere sono funzioni. Schematicamente, l'obiettivo di un *algoritmo supervisionato* può essere sintetizzato così:

- **Dato** un insieme di esempi descritti in termini delle feature di input e target

- **Restituire** la rappresentazione compatta di una funzione utile a predire il valore della feature target di nuovi esempi, per i quali siano disponibili solo valori delle feature di input.

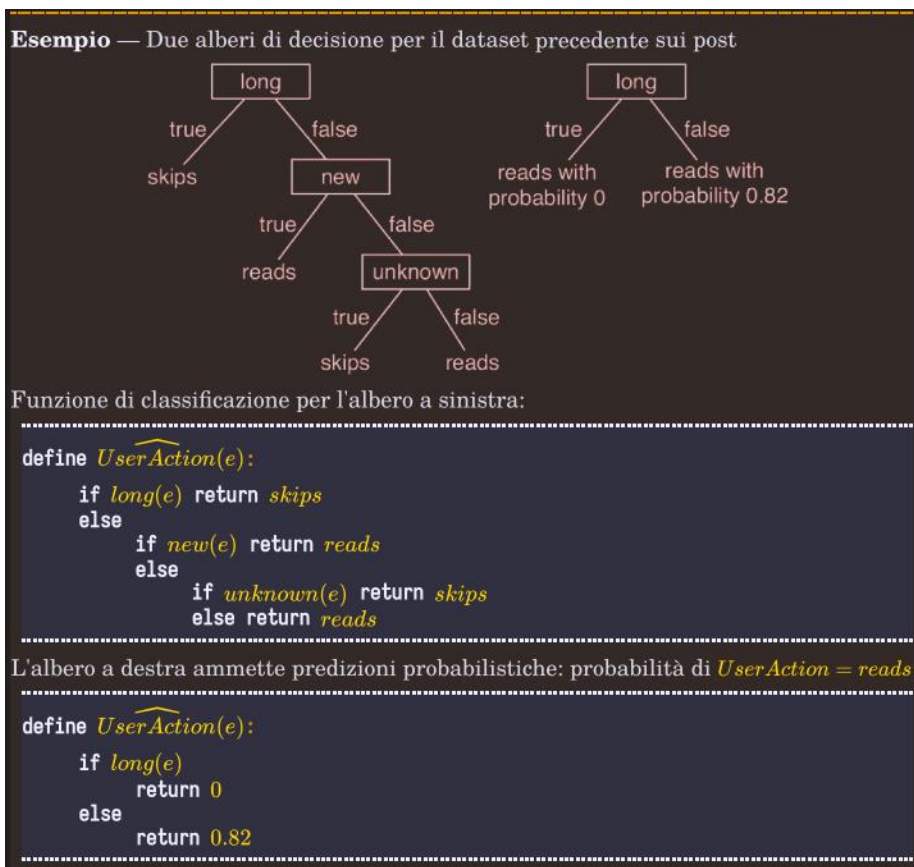
Le differenze fra i diversi algoritmi dipenderanno dalla scelta fra le diverse possibili rappresentazioni delle funzioni.

Un **albero di decisione** è un albero binario costituito da:

- Nodi *interni* (non-foglia) etichettati con *condizioni*, ossia test booleani basati sui valori delle feature negli esempi e connessi con due *figli*, radici di sotto-alberi, attraverso archi tipicamente etichettati con true o false;
- Nodi-*foglia* etichettati con una stima puntuale per la feature obiettivo: la classe in un *albero di classificazione* oppure un valore reale in un *albero di regressione*.

Tale struttura rappresenta una *funzione* definita *per parti* ovvero un *programma* fatto di strutture *condizionali* annidate.

Per la *predizione*, dato un esempio con i valori delle variabili di input: partendo dalla radice, ogni condizione incontrata viene valutata per tale esempio seguendo l'arco corrispondente al risultato di ogni test; raggiunta una foglia, si restituisce la stima puntuale associata.



Le due questioni principali nell'apprendimento di alberi di decisione sono le seguenti. Dati gli esempi di training:

- QUALE albero generare? Dovrebbe essere possibile rappresentare *qualsiasi funzione* con feature X_i discrete; il *bias* è insito nel criterio di preferenza tra i diversi alberi: andrebbero favoriti alberi *più piccoli* (coerenti con i dati), ossia con la minima profondità o con meno nodi;
- COME generarlo? Si può effettuare una *ricerca* in uno spazio di alberi a partire dal più piccolo che si adatti ai dati, ma tale spazio è enorme, per cui una ricerca esaustiva non è proponibile; ci si affida, quindi, a un metodo di ricerca *greedy*, puntando a minimizzare l'errore.

Come detto si può considerare un albero di decisione come un *programma* che sappia determinare la predizione per qualunque esempio e arbitrario: ciascuna *foglia* conterrà una predizione del valore di Y (senza condizioni); i nodi *interni* conterranno test di condizioni seguiti da diramazioni come in `if  $c(e)$  then  $t_1(e)$  else  $t_0(e)$` , dove c è la condizione booleana da testare e t_1 e t_0 sono i suoi sotto-alberi, ovvero sotto-programmi, t_1 sul ramo true e t_0 sul ramo false.

L'algoritmo segue (top-down) la decomposizione *ricorsiva* della struttura dell'albero:

```

procedure DT_Learner( $Cs, Y, Es, \gamma$ )
  Input
     $Cs$ : insieme delle possibili condizioni
     $Y$ : feature-obiettivo
     $Es$ : insieme di esempi di training
     $\gamma$ : miglioramento minimo per l'espansione ( $\gamma \geq 0$ )
  Output
    funzione per predire il valore di  $Y$  per un esempio

   $c \leftarrow \text{select\_split}(Es, Cs, \gamma)$ 
  if  $c = \text{None}$  then // criterio di stop raggiunto
     $v \leftarrow \text{leaf\_value}(Es)$ 
    define  $T(e) = v$ 
    return  $T$ 
  else
     $\text{true\_examples} \leftarrow \{e \in Es : c(e)\}$ 
     $t_1 \leftarrow \text{DT\_learner}(Cs \setminus \{c\}, Y, \text{true\_examples}, \gamma)$ 
     $\text{false\_examples} \leftarrow \{e \in Es : \neg c(e)\}$ 
     $t_0 \leftarrow \text{DT\_learner}(Cs \setminus \{c\}, Y, \text{false\_examples}, \gamma)$ 
    define  $T(e) = \text{if } c(e) \text{ then } t_1(e) \text{ else } t_0(e)$ 
    return  $T$ 

  procedure select_split( $Es, Cs, \gamma$ )
     $\text{best\_val} \leftarrow \text{sum\_loss}(Es) - \gamma$ 
     $\text{best\_split} \leftarrow \text{None}$ 
    for-each  $c \in Cs$  do
       $\text{val} \leftarrow \text{sum\_loss}(\{e \in Es \mid c(e)\}) + \text{sum\_loss}(\{e \in Es \mid \neg c(e)\})$ 
      if  $\text{val} < \text{best\_val}$  then
         $\text{best\_val} \leftarrow \text{val}$ 
         $\text{best\_split} \leftarrow c$ 
    return  $\text{best\_split}$  //  $\text{best\_split} = \text{None} \rightarrow$  criterio di stop raggiunto

```

Nell'algoritmo si effettua una scelta miope della condizione c ottimale – **greedy optimal split** – basata sulle somme delle loss e dopo il partizionamento e una soglia γ : l'aggiunta di un livello/partizione aumenta le dimensioni dell'albero di un'unità mentre γ penalizza tale incremento: se positiva, essa è utile a evitare che si verifichi $\text{val} < \text{best_val}$ per meri errori di arrotondamento.

Il *criterio di stop* viene determinato dall'esaurimento delle condizioni, ovvero da esempi che risultano tutti di una stessa classe, il che porta ad avere una loss nulla. L'*output* (nelle foglie) rappresenta una stima *puntuale*. Come stima puntuale si sceglie, in genere, il valore più *probabile*, la moda, oppure la mediana o la media; come visto in precedenza, in alternativa si può optare per la restituzione di una distribuzione di probabilità delle classi per gli esempi finiti in quella foglia, che corrisponderà ad una regione ristretta dello spazio degli esempi.

Quando si calcola la log-loss utilizzando il logaritmo in base 2, il valore medio della loss sul dataset (Es) non è solo un numero astratto, ma corrisponde all'**entropia** della distribuzione dei dati. L'entropia misura l'incertezza in bit; se il modello ha una log-loss media alta, significa che c'è molta incertezza, e quindi disordine, nei dati.

La variabile *val* è il numero totale di bit necessari per descrivere Es dopo aver applicato il test c . Se la dividiamo per il numero totale di esempi ($|Es|$), otteniamo l'entropia media residua *dopo* la divisione. Questa rappresenta l'incertezza che rimane anche dopo aver separato i dati.

Dunque, definiamo l'**Information Gain** (IG) come la differenza fra le due entropie, misura spesso usata anche assieme ad altre da ottimizzare.

Se le feature sono booleane è facile definire le condizioni per i test. In caso di feature non booleane ci sono diverse possibilità:

- Se X è categorica, con dominio $\{v_1, \dots, v_k\}$, si può associare una **variabile indicatrice** binaria X_i a ciascun valore: $X_i(e) = 1$ se $X(e) = v_i$ e $X_i(e) = 0$ altrimenti: per ogni esempio e , una sola fra $X_1(e), \dots, X_k(e)$ vale 1 (le altre saranno nulle);
- Se X è **ordinale** (a valori reali o discreti) allora essa ha un dominio *totalmente ordinato* (ad esempio le taglie, il grado di istruzione); in tal caso, per ciascun valore v , si può introdurre una **variabile** booleana di **taglio/cut** che varrà 1 se $X > v$ e 0 altrimenti; da una combinazione di tagli si possono definire intervalli, es. il caso in cui la variabile per $X > 9$ sia vera e quella per $X > 17$ sia falsa corrisponde a $9 < X \leq 17$;
- **Binning**: dato un insieme di soglie si crea una feature per ogni intervallo/bin: date le soglie $\alpha_1 < \alpha_2 < \dots < \alpha_k$ si ottengono $k + 1$ feature booleane, una vera per X tale che $X \leq \alpha_1$, una per $\alpha_k < X$ e una per $\alpha_i < X \leq \alpha_i + 1$ per ogni $1 \leq i < k$; si noti che il bin $\alpha_i < X \leq \alpha_i + 1$ richiede 2 partizionamenti tramite tagli; i valori α_i possono essere predefiniti, ad esempio, si possono adottare i *percentili* rilevati sui dati di training, oppure scelti in base a quelli della Y ;

- Per feature X categoriche, si possono definire condizioni $X \in S$ con $S \subset \text{dom}(x)$ dato l'insieme di valori $|S| \geq 1$; se Y è booleana per scegliere gli S , avendo ordinato $\text{dom}(X)$ in base alle proporzioni di Y veri, si trova la sezione migliore, *greedy optimal split*, della lista ordinata;
- Si può pensare anche a un'estensione che preveda la possibilità di conduzioni n -arie (*multiway split*) quindi per variabili con più valori nel dominio si potrà considerare un figlio per ciascun valore: si ottiene un *albero n -ario*, con struttura più complessa di quelli con test della form if-then-else, più difficili da apprendere in quanto per alcuni valori delle condizioni potrebbe non essere disponibile alcun esempio di training; con euristiche greedy, come IG, una condizione su variabile con domini più grandi porta ad avere più figli e ad un miglior adattamento ai dati a scapito della compattezza del modello.

Quando *select_split* restituisce *None* non si procede a ulteriore partizionamento. Ciò può accadere a seguito dell'esaurimento degli esempi o delle condizioni, oppure quando gli esempi hanno tutti lo stesso valore per ciascuna condizione, o hanno tutti la stessa classe.

Si può avere un miglioramento con l'aggiunta di una condizione con una soglia $< \gamma$. Altri criteri di stop anticipato che evitino ulteriori partizionamenti possono essere la *minimum child size*, allorché un nodo-figlio abbia meno esempi rispetto a una data soglia, o la *maximum depth*, che segnala il raggiungimento di una profondità (altezza) massima.

È possibile che una condizione funzioni bene solo in congiunzione con altre ma un metodo greedy potrebbe non funzionare quando questo si verifica. Si tratta di un *caso difficile*: ad esempio, si consideri di dover trovare un albero che approssimi una funzione di parità di k variabili booleane, vera se un numero dispari (o pari) di esse sono vere; conoscere i valori di meno di k variabili non dà alcuna informazione sul valore della funzione; si possono approssimare funzioni di parità più semplici ($k = 2$) come lo XOR o l'equivalenza, ma in generale gli alberi di decisione possono essere molto complessi. Quando dal partizionamento greedy risultano alberi troppo complicati conviene procedere alla loro *semplificazione* (ad esempio con tecniche di *pruning*).

Si prenderanno in considerazione algoritmi che apprendono *modelli lineari* per risolvere problemi di regressione (target continuo) e classificazione (target discreto).

Con la **regressione lineare** si mira all'adattamento di una funzione *lineare* a un training set con feature *numeriche*. Si supponga che le feature di input siano $X_1, \dots, X_m \in \mathbb{R}$ e che la feature target sia $Y \in \mathbb{R}$.

Il modello è una **funzione lineare** delle feature di input:

$$\hat{Y}_{\bar{w}}(e) = w_0 + w_1 * X_1(e) + \dots + w_m * X_m(e) = \sum_{i=0}^m w_i * X_i(e)$$

dove $\bar{w} = \langle w_0, w_1, \dots, w_m \rangle$ è n vettore di **pesi** che costituiscono i *parametri* da determinare. Si ottiene una forma più compatta assumendo la feature aggiuntiva costante $X_0 = 1$.

↑ Spiegazione: forniamo un esempio, immaginiamo di avere i dati su una casa (es. metri quadri, vicinanza al centro, ecc...). Tutti i dati che abbiamo sono le varie $X_i(e)$, mentre $Y(e)$ è il prezzo e $\hat{Y}(e)$ è la nostra stima. Tramite le varie w_i diamo un peso ad ogni feature di input, quindi i metri quadri ecc...

Ma come scegliamo i pesi w giusti? Dobbiamo misurare quanto la nostra retta è “sbagliata”, e lo facciamo così ↓

Per definire il problema di ottimizzazione, dato l'insieme di esempi Es , si considera l'errore quadratico medio su Es per Y :

$$\begin{aligned} \text{error}(Es, \bar{w}) &= \frac{1}{|Es|} \sum_{e \in Es} \left(Y(e) - \hat{Y}_{\bar{w}}(e) \right)^2 \\ &= \frac{1}{|Es|} \sum_{e \in Es} \left(Y(e) - \sum_{i=0}^m w_i * X_i(e) \right)^2 \end{aligned}$$

Questa funzione ha un unico minimo.

↑ Spiegazione: come abbiamo visto in precedenza, l'errore quadratico medio ci serve per due motivi: 1) per questioni di calcolo, così l'errore sarà sempre positivo e 2) perché vogliamo punire molto severamente i grossi errori.

Facciamo la somma degli errori di tutte le case del dataset (per esempio) e si fa la media. Continuo ↓

Considerate le derivate parziali rispetto ai pesi

$$\frac{\partial}{\partial w_i} \text{error}(Es, \bar{w}) = \frac{1}{|Es|} \sum_{e \in Es} 2\delta(e) * X_i(e)$$

dove $\delta(e) = \hat{Y}_{\bar{w}}(e) - Y(e)$ è una funzione lineare sui pesi. Il minimo può essere calcolato *analiticamente* risolvendo le equazioni lineari ottenute annullando tali derivate.

↑ Spiegazione: in quest'ultima parte vogliamo capire come trovare i pesi w che rendono questo errore il più piccolo possibile. Per trovare il fondo, matematicamente, utilizziamo le **derivate**. Quando la derivata (la pendenza) è **zero**, significa che siamo sul fondo.

La formula finale ci dice come calcolare la pendenza rispetto a ogni peso w . $\delta(e)$ è semplicemente l'errore compiuto sull'esempio e (come ci dice il testo, $(e) = \hat{Y}_w(e) - Y(e)$).

Tutto ciò significa che non dobbiamo andare per tentativi. In questo modo avremo i valori perfetti di $w_0, w_1, \dots, w_m$ che ci danno la retta migliore in assoluto.
(Fine spiegazione)

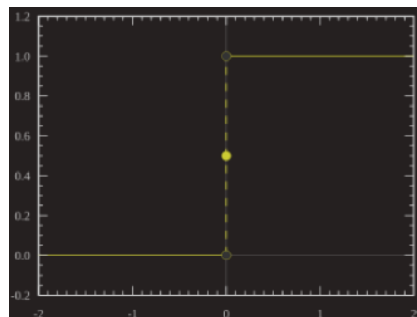
Per la *classificazione binaria* si ha una feature target Y con dominio $\{0,1\}$. Come modello non basta una funzione lineare: le predizioni non possono superare 1 o essere minori di 0. Si considera allora una **funzione lineare compressa** (*squashed*):

$$\begin{aligned}\hat{Y}_w(e) &= \phi(w_0 + w_1 * X_1(e) + \dots + w_m * X_m(e)) \\ &= \phi\left(\sum_i w_i * X_i(e)\right)\end{aligned}$$

dove ϕ è detta anche **funzione di attivazione** da $\mathbb{R}$ (retta reale) a $R \subset \mathbb{R}$, tipicamente si considera $R = [0,1]$. Un predittore basato su tale funzione si dice **classificatore lineare**.

Lo **step** o *gradino di Heaviside* è una funzione di attivazione semplice definita come segue:

$$\text{step}_0 = \begin{cases} 0 & x < 0 \\ 1 & x \geq 0 \end{cases}$$

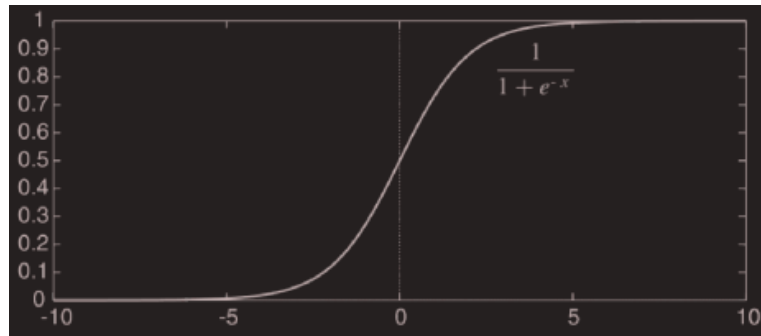


Essa viene impiegata in unità alla base dei **percettroni**. Tuttavia, risulta difficile adattare la discesa di gradiente a funzioni non ovunque differenziabili: con una funzione quasi ovunque differenziabile come step, l'ampiezza dello scalino dovrebbe tendere a zero per garantire la convergenza.

Allora, si considerano in alternativa funzioni come la sigmoide (un tipo di funzione *logistica*):

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

il cui grafico è rappresentato come segue:



Essa comprime le predizioni nell'intervallo $]0,1[$, appropriato ai fini della classificazione, e risulta differenziabile, con derivata:

$$\frac{d}{dx} \text{sigmoid}(x) = \text{sigmoid}(x) * (1 - \text{sigmoid}(x))$$

Nella **regressione logistica** il modello di *classificazione* è costituito da una funzione lineare compressa dalla sigmoide:

$$\hat{Y}(e) = \text{sigmoid} \left(\sum_{i=0}^m w_i * X_i(e) \right)$$

Per trovare i pesi $\bar{w}$ si minimizza la *log-loss* media:

$$LL(Es, \bar{w}) = -\frac{1}{|Es|} \left[\sum_{e \in Es} Y(e) \log \hat{Y}(e) + (1 - Y(e)) \log (1 - \hat{Y}(e)) \right]$$

con derivate parziali:

$$\frac{\partial}{\partial w_i} LL(Es, \bar{w}) = \frac{1}{|Es|} \sum_{e \in Es} \delta(e) * X_i(e)$$

dove $\delta(e) = \hat{Y}_{\bar{w}}(e) - Y(e)$.

Tale funzione risulta difficile da minimizzare analiticamente, essendo $\hat{Y}_{\bar{w}}$ non lineare nei parametri per cui si ricorre a una soluzione algoritmica.

↑ Spiegazione: nella regressione lineare classica, la predizione $\hat{Y}$ è una retta che può andare verso l'alto all'infinito $(+\infty)$ o verso il basso $(-\infty)$.

Problema: nella classificazione binaria, il target Y deve essere tassativamente 0 o 1. Se il modello predice un numero diverso da 0 o 1, non ha senso.

Per risolvere questo problema, non buttiamo via la somma pesata ($\sum w * X$), ma la “incartiamo” dentro una nuova funzione chiamata funzione di attivazione (ϕ). Il ruolo di ϕ è quello di prendere il risultato della somma e comprimerlo in un intervallo controllato, tipicamente $[0,1]$.

La funzione di attivazione più semplice è quella a “interruttore”, ovvero step. In base al punteggio (la somma pesata) calcolato, scatta il valore 0 o 1. Come possiamo vedere c'è un problema, ovvero che la **derivata è inutile**: dove è piatto (ovunque tranne lo zero), la pendenza è 0 (il modello non impara), mentre nel salto (nello zero), la derivata non è definita (è infinita).

Perché oggi non usiamo più il gradino? Per imparare (ovvero per correggere i propri errori), l'algoritmo deve calcolare la derivata (la pendenza della curva). Come detto precedentemente, questa pendenza o è zero o è infinita.

Per uscire da questo vicolo cieco, si introduce la **sigmoide**. Essa risolve tutto poiché “ammorbidisce” il gradino: invece di un salto verticale, ha una curva, ed essendo tale, ha una pendenza (derivata) diversa da zero quasi ovunque. Questo permette alla discesa del gradiente di “scivolare” lungo la curva e trovare il punto di minimo errore.

Calcolare la pendenza tramite la sigmoide è semplicissimo (derivata scritta prima). Infatti, ci basta conoscere il valore della predizione stessa, rendendo i calcoli al computer velocissimi. Dunque, per ogni esempio, l'algoritmo fa la somma pesata, passa il risultato nella sigmoide e ottiene una probabilità tra 0 e 1.

Poiché ora l'output è una probabilità e non un numero qualsiasi, non usiamo più l'errore quadrato, ma la **log-loss**. La formula agisce come un interruttore: se l'esempio è positivo ($Y = 1$), si attiva solo la parte sinistra ($\log \hat{Y}$), quindi se il modello ha predetto 0.9 (giusto), l'errore è basso, altrimenti con 0.1 avremmo il logaritmo che schizza all'infinito (punizione severa). Altrimenti, se l'esempio è negativo ($Y = 0$), si attiva solo la parte destra ($\log(1 - \hat{Y})$).

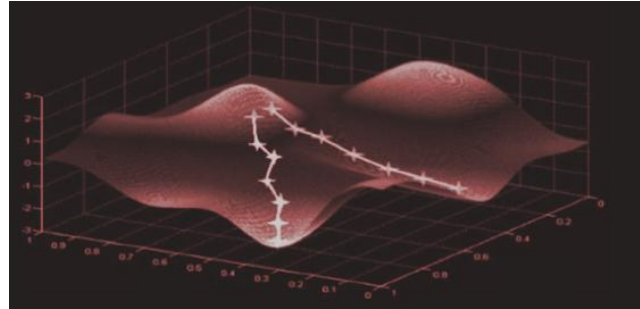
L'ultima parte mostra la derivata della log-loss rispetto ai pesi. A differenza della regressione lineare, dove potevano risolvere l'equazione analiticamente (con carta e penna) per trovare il minimo perfetto subito, qui non si può.

Essendo la $\hat{Y}$ dentro una sigmoide (funzione non lineare), l'equazione è troppo complessa da risolvere direttamente.

Dunque, siamo costretti ad usare una **soluzione algoritmica** (iterativa), ovvero la **discesa del gradiente**, che procede a piccoli passi verso il minimo.

La **discesa di gradiente** è un metodo *iterativo* per la ricerca dei minimi di funzioni d'errore o *loss*.

Un esempio di utilizzo dell'algoritmo è riportato in figura:



Dopo averi inizializzato $\bar{w}$ in maniera casuale, a ogni iterata, ogni w_i verrà decrementato in proporzione alla sua derivata parziale

$$w_i \leftarrow w_i - \eta * \frac{\partial}{\partial w_i} \text{error}(Es, \bar{w})$$

dove η è detta *passo* o **learning rate**, che funge da moltiplicatore alla derivata parziale del peso. Le derivate misurano l'influenza sull'errore di piccole variazioni dei pesi.

Per un modello lineare, l'errore quadratico risulta una funzione *convessa* con un *unico* minimo locale, quindi anche globale.

Con un passo sufficientemente piccolo, l'algoritmo converge al minimo locale. Le derivate parziali di $\text{error}(Es, \bar{w})$ o log-loss media si possono scrivere come segue:

$$\frac{\partial}{\partial w_i} \text{error}(Es, \bar{w}) = \frac{1}{|Es|} \sum_{e \in Es} \delta(e) * X_i(e)$$

con $\delta(e) = \hat{Y}_{\bar{w}}(e) - Y(e)$ e costante 2 assorbita da η .

Normalmente l'aggiornamento dei pesi avviene *dopo* aver iterato su *tutti* gli esempi; in alternativa, si può ricorrere alla **discesa di gradiente stocastica** (SGD) nella quale si prevede un aggiornamento dopo aver visto un *campione* di b esempi, detto **batch** o **minibatch**; in tal caso, un passaggio completo sul dataset si chiama **epoca**, costituita da $\lceil |Es|/b \rceil$ batch.

L'algoritmo di apprendimento del modello è descritto come segue:

```

procedure Linear_learner( $X_s, Y, E_s, \eta, b$ )
  Input
     $X_s$ : insieme di feature input,  $X_s = \{X_1, \dots, X_m\}$ ,  $X_0 = 1$ 
     $Y$ : feature target
     $E_s$ : insieme di esempi di training
     $\eta$ : learning rate
     $b$ : dimensione del batch
  Output
    funzione di predizione su esempi
  Local
     $w_0, \dots, w_m$ : reali
     $d_0, \dots, d_m$ : reali
    inizializzare casualmente  $w_0, \dots, w_m$ 
    define  $pred(e) = \phi(\sum_i w_i \cdot X_i(e))$ 
    repeat
      for each  $i \in [0, m]$  do  $d_i \leftarrow 0$ 
      selezionare il batch  $B \subseteq E_s$  di dim.  $b$ 
      for each esempio  $e \in B$  do
         $error \leftarrow pred(e) - Y(e)$ 
        for each  $i \in [0, m]$  do  $d_i \leftarrow d_i + error \cdot X_i(e)$ 
      for each  $i \in [0, m]$  do
         $w_i \leftarrow w_i - \eta \cdot d_i / b$ 
    until terminazione
    return  $pred$ 

```

Le possibili *condizioni* di terminazione per l'algoritmo sono:

1. Un numero massimo di passi (utile a evitare di divergere);
2. Un errore vicino a 0;
3. Ridotte variazioni di $\bar{w}$ fra iterate successive.

Esempio — Classificatore per il problema dei post:

Classificatore corretto:

$$\widehat{Reads}(e) = \text{sigmoid}(-8 + 7 \cdot Short(e) + 3 \cdot New(e) + 3 \cdot Known(e))$$

trovato dopo circa 3000 iterate dell'algoritmo con $\eta = 0.05$.

Si può leggere il modello come: $\widehat{Reads}(e)$ è vero (ossia il valore predetto per e risulta più vicino a 1 che a 0) sse $Short(e)$ è vero ed è anche vero $New(e)$ o $Known(e)$.

Tale funzione è analoga a quella appresa attraverso gli alberi di decisione.

Sono possibili alcune *varianti*: regolando le dimensioni dei lotti/batch, con b più piccole l'apprendimento risulta più veloce dato che si richiedono meno esempi per l'aggiornamento (ma si potrebbe compromettere la convergenza verso il minimo locale); considerando, invece, tutti gli esempi, si termina quando tutte le d_i sono nulle (quindi si è al minimo locale): tipicamente si parte con batch piccoli e si aumenta b fino alla convergenza (o al raggiungimento di prestazioni soddisfacenti).

Nella **discesa di gradiente incrementale** (online GD) l'aggiornamento dei pesi va fatto *dopo ogni esempio*, quindi, in tal caso, $b = 1$. Non si devono memorizzare i d_i , i pesi sono aggiornati direttamente. Ciò torna utile per esempi acquisiti in *streaming*. Nel caso di esempi non selezionati casualmente, si corre il rischio della cosiddetta **dimenticanza catastrofica**, ossia dell'adattamento ai dati più nuovi, dimenticando quelli meno recenti.

In uno spazio m -dimensionale, ovvero con m feature (di input), si definisce **iperpiano** l'insieme di punti che soddisfano un vincolo espresso come equazione *lineare* sulle variabili. Esso determina uno spazio di $(m - 1)$ dimensioni (es. 2 dimensioni = retta, 3 = piano ecc...).

Un **problema** di classificazione binaria si dice **linearmente separabile** quando esiste un iperpiano che separi gli esempi delle diverse classi. Un problema del genere può essere risolto con la regressione logistica (usando un algoritmo di GD stocastica): l'errore è riducibile arbitrariamente se e solo se il problema è linearmente separabile.

L'iperpiano separatore sarà l'insieme di punti per i quali $\sum_i w_i * X_i = 0$, dove $\bar{w}$ contiene i coefficienti appresi. Dato un nuovo esempio, per la predizione della classe il modello avrà un valore maggiore di 0.5 da un lato e minore dall'altro.

Quando Y è categorica con più di due classi, essa può essere rappresentata usando variabili indicatrici binarie da imparare separatamente. Essendoci solo un valore di Y per ogni esempio, il modello dovrebbe essere capace di predire probabilità per ciascuna variabile indicatrice, con somma unitaria.

È possibile apprendere modelli per tutte tranne una e ricavare la probabilità del suo valore dalle altre. Ciò introduce una forma di *asimmetria* che può risultare problematica con predizione debole del valore (a volte persino con probabilità negative) per l'accumularsi di errori sulle altre variabili (in pratica se predicono, ad esempio, 0.6 e 0.5, ci troviamo con $1 - 0.6 - 0.5$ che fa -0.1).

In alternativa si può considerare, per ogni valore, l'esponenziale di una funzione lineare, da normalizzare. Si avranno così più parametri del necessario nel modello (*sovra-parametrizzazione*), ma anche uno stesso trattamento per tutti i valori.

Data Y categorica con dominio rappresentato dalle tuple $\langle v_1, \dots, v_k \rangle$, si definisce la **funzione softmax** $\langle \alpha_1, \dots, \alpha_k \rangle \in \mathbb{R}^k \mapsto \mathbb{R}^k$ che calcola un vettore la cui i -esima componente sarà:

$$\text{softmax}(\langle \alpha_1, \dots, \alpha_k \rangle) = \frac{\exp(\alpha_i)}{\sum_{j=1}^k \exp(\alpha_j)}$$

Tale funzione assicura la positività e la somma unitaria potendo così definire una distribuzione di probabilità. Essa è correlata con la sigmoide, essendo:

$$\text{sigmoid}(x) = \frac{1}{\exp(-x) + 1} = (*) \frac{\exp(x)}{\exp(0) + \exp(x)} = \text{softmax}(\langle 0, x \rangle)_2$$

la coppia $\langle 0, x \rangle$ corrisponde ai valori (*false*, *true*); $\text{softmax}(\langle 0, x \rangle)_2$ è la seconda componente della coppia risultante dalla softmax; si ha l'uguaglianza (*) moltiplicando numeratore e denominatore per $\exp(x)$ quindi equivale al caso in cui la prima componente (per *false*) sia fissata a 0. Si noti come la softmax, come la sigmoide, *non* possa rappresentare probabilità nulle.

La **regressione softmax** è una generalizzazione della regressione logistica (detta anche *regressione logistica/logit multinomiale*): si ha un'equazione lineare per ogni valore di Y , dove $\text{dom}(Y) = \{v_1, \dots, v_k\}$.

Per la predizione per un esempio e e $\text{softmax}(\langle u_1(e), \dots, u_k(e) \rangle)$ calcola una tupla di k valori con j -esima componente corrispondente alla predizione per $Y = v_j$ e $u_j(e) = w_{0,j} + X_1(e) * w_{1,j} + \dots + X_m(e) * w_{m,j}$.

Il modello viene tipicamente ottimizzato usando la log-loss categorica: sia w_{ij} per X_i e v_j , e sia $Y(e) = v_q$:

$$\begin{aligned} \frac{\partial}{\partial w_{ij}} \text{logloss}(\text{softmax}(\langle u_1(e), \dots, u_k(e) \rangle), v_q) &= \frac{\partial}{\partial w_{ij}} - \log \left(\frac{\exp(u_q(e))}{\sum_j \exp(u_j(e))} \right) \\ &= \frac{\partial}{\partial w_{ij}} \left(\log \left(\sum_j \exp(u_j(e)) \right) - u_q(e) \right) \\ &= \left[\left(\hat{Y}(e) \right)_j - 1(j = q) \right] * X_i \end{aligned}$$

con $1(j = q) = 1$ se j è l'indice del valore osservato v_q e $(\hat{Y}(e))_j$ è il j -esimo componente della predizione, valore predetto meno quello effettivo.

↑ Spiegazione: **softmax** serve per la classificazione con **più di 2 classi**. Trasforma punteggi qualsiasi in probabilità, aventi somma unitaria. Se le classi sono solo 2, la softmax coincide con la sigmoide. Dunque, per ogni classe (es. v_j) il modello ha un set di pesi diverso:

1. Calcola il punteggio u per la classe 1, classe 2, classe 3... usando la solita formula lineare ($\sum w * X$);
2. Passa tutti questi punteggi dentro la funzione softmax;
3. Ottiene le probabilità finali (es. Gatto 70%, Cane 20%, Topo 10%).

Dopo tutti i passaggi complicati con logaritmi e somme, la formula finale ha: $(\hat{Y}(e))_j$ che rappresenta la **predizione** del modello che l'esempio e sia di classe j , $1(j = q)$ che vale 1 se quella è la **classe giusta**, e ci serve per bilanciare i pesi (se la differenza è negativa allora siamo sotto di un certo numero, altrimenti se positiva siamo sopra, così bilanciamo) 0 altrimenti, e X_i è l'**input**.

Nell'implementazione del modello vi sono problematiche connesse con la rappresentazione dei numeri:

- L'esponenziale di numeri grandi potrebbe non essere rappresentabile portando a un caso di *overflow*, ad esempio $\exp(800)$;
- L'esponenziale di numeri grandi negativi tende a 0, comportando possibili casi di $\exp(-800)$.

Per questi casi, conviene sottrarre il massimo da ciascun valore per cui ci sarà sempre uno 0 e il resto dei valori risulteranno negativi.

Per grandi numeri di valori possibili il calcolo del denominatore diventa costoso. Si può ricorrere a una modalità di predizione in termini di un albero binario di valori, definita **softmax gerarchica**, simile alla softmax, ma più efficiente per grandi domini.

Nella regressione lineare e in quella logistica, si considerano feature di input numeriche. In caso di altri tipi si devono fare delle trasformazioni:

- Feature *categoriche*: vanno convertite in feature con dominio $\{0,1\}$, tramite **one-hot encoding**, usando variabili-indicatrici;
- Feature a *valori reali*: si possono usare direttamente in modelli lineari per Y , una volta aggiustate le altre feature di input; spesso si adottano anche trasformazioni per crearne di nuove, dotate di proprietà desiderabili;
- Feature *ordinali*, comprese quelle a valori reali: si possono considerare dei *tagli* per definire feature booleane, ad esempio da x , scegliendo un valore v del suo dominio, si considera una nuova feature, vera se $x > v$, oppure $x - v > 0$; nel *binning* si sceglie un insieme di soglie, $\alpha_1 < \alpha_2 < \dots < \alpha_k$, ricavando una feature booleana per ogni intervallo fra α_i e α_{i+1} , come per una funzione costante a pezzi; una feature può essere derivata da $\max(x - v, 0)$, funzione alla base delle *rectified linear unit*.

La progettazione di feature appropriate ai diversi problemi e relativi modelli è il tema principale della cosiddetta **feature engineering**. Spesso risulta difficile progettare buone feature.

Sovradattamento

Il **sovradattamento** (*overfitting*) è il problema che si manifesta con l'apprendimento di modelli basati su *regolarità apparenti* presenti nei dati di training, ma non in quelli di test o, in generale, nel mondo da cui essi sono tratti. Il fenomeno è tipico nella ricerca di *segnali* in dati distribuiti casualmente: si tratta di correlazioni *spurie* nei

dati che non si riflettono sull'intero dominio del problema; si verifica un *eccesso di fiducia* nelle predizioni del modello rispetto a quanto autorizzino i dati a disposizione.

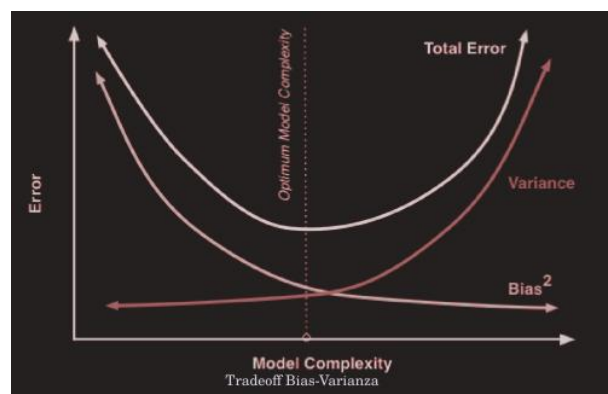
Previsioni estreme possono diventare problematiche con la disponibilità di nuovi dati. Questa forma di sovradattamento è legata alla cosiddetta **regressione verso la media**. I valori delle osservazioni (ad esempio i giudizi sui ristoranti) sono determinati da *qualità* e *caso*: più dati abbassano l'impatto della casualità.

Il sovradattamento può dipendere anche legato dalla *complessità del modello* scelto: un modello più complesso, con più parametri, risulta *più facile* da adattare ai dati di training rispetto a uno più semplice. Si può considerare il *caso estremo* in cui si preveda un modello con un parametro per ciascun esempio. Tuttavia, esso andrà giudicato sulla base di nuovi esempi di test, non utilizzati per l'addestramento.

Errore sul Test Set, dalla statistica:

$$Errore_2(Test) = Bias^2 + Varianza + Errore_irriducibile$$

- **Bias**: errore dovuto all'*imperfezione* del modello appreso, risulta basso quando esso ha una complessità vicina a quella ideale (**ground truth**) del *processo reale* che governa la generazione dei dati. Può essere scomposto in *bias di rappresentazione*, la differenza tra tipo di modello scelto e *ground truth*, e *bias di ricerca*, influenza dell'algoritmo che non cerca esaustivamente nello spazio di ipotesi;



- **Varianza**: errore dovuto all'insufficienza di dati disponibili; un modello più complesso (con più parametri) richiede più dati per cui, se il quantitativo di dati è fisso, si deve trovare un **compromesso** fra *bias* e *varianza*. Per un modello complesso, se non è disponibile il quantitativo di dati necessario allora il bias sarà basso e la varianza alta; per un modello semplice, se i dati disponibili sono sufficienti alla stima dei parametri si ha una bassa varianza ma un bias alto;

- **Rumore:** errore *intrinseco* dovuto a dati che dipendono da *feature non considerate* nel modello e alla *naturale aleatorietà* nel processo di generazione dei dati.

Come visto in precedenza, nella predizione il calcolo della *media* costituisce spesso una scelta ottimale rispetto a molte misure basate su dati di training. Ma la *media empirica* su Es non è sempre valida per stime riguardanti nuove istanze.

Ma, in mancanza di dati a sufficienza, si può verificare il caso di un valore fra quelli del dominio mai osservato in Es avrebbe media nulla, così come la sua frequenza relativa se si deve stimare la probabilità.

Ad esempio, si supponga di dover predire il voto di uno studente. La media risulta *appropriata* solo se sono stati già sostenuti diversi esami, mentre risulterebbe *non appropriata* se fosse basata su un solo esame o addirittura *indefinita* in assenza di esami.

Per porre rimedio a tali problemi si possono considerare *pseudo-esempi* aggiuntivi come *conoscenza pregressa* addizionale da usarsi per calcolare i cosiddetti **pseudoconteggi** (*prior counts*) di partenza. Essi saranno non negativi ma non necessariamente interi.

Disponendo dei valori osservati $v_1, \dots, v_n$, per predire il successivo $\hat{v}$, si assume di usare una predizione media a_0 , ottenuta da conoscenza pregressa (il parere dell'esperto), prima ancora di avere dati a disposizione, ossia quando $n = 0$; per i casi successivi si stimerà la media come segue:

$$\hat{v} = \frac{c * a_0 + \sum_i v_i}{c + n}$$

con $c \in \mathbb{R}_+$ numero di dati fittizi assunti, *pseudo-esempi*, ciascuno con valore a_0 (non necessariamente intero). Quando $c = 0$ la predizione corrisponde a quella della media empirica, non utilizzabile senza dati osservati a disposizione ($n = 0$). Il valore di c regola l'importanza relativa fra ipotesi iniziale (*a priori*) e dati osservati.

Come visto in precedenza, adottando errore quadratico o log-loss, la media empirica fornisce una stima ottimale: $\hat{p} = \frac{n_1}{n_0 + n_1}$. Tuttavia, pur minimizzando errore/loss sul training set, non ci sono garanzie sul test set. Tipicamente, se uno dei valori non occorre mai nel training set, si registrerebbe una log-loss infinita.

In tali casi, si può ricorrere allo **smoothing additivo di Laplace**:

$$\hat{p} = \frac{n_1 + 1}{n_0 + n_1 + 2} = \frac{n_1 + 1}{n + 2}$$

Fra tutti gli stimatori, minimizza log-loss ed errore quadratico sul test set. Si osservi come l'aggiustamento equivalga all'aggiunta di due pseudo-esempi, uno con $Y = 0$ e uno con $Y = 1$.

Si dovrebbero preferire modelli semplici a quelli complessi.

A tale scopo, si può considerare l'uso di un **regolarizzatore**, ossia un termine da aggiungere alla misura di adattamento del modello ai dati (tipicamente un errore, una loss) in modo da *penalizzare* la complessità dell'ipotesi da apprendere.

Si dovrà pertanto scegliere un'ipotesi h che minimizzi

$$\left[\sum_e \text{loss}(\hat{Y}(e), Y(e)) \right] + \lambda * \text{regularizer}(\hat{Y})$$

dove:

- $\text{loss}(\hat{Y}(e), Y(e))$ misura la qualità dell'adattamento su e ;
- $\text{regularizer}(\hat{Y})$ penalizza la complessità del modello;
- il **parametro di regolarizzazione** λ cerca di fare una media tra adattamento e semplicità; esso può servire anche a sommare quantità su scale diverse e può essere determinato in base a conoscenza pregressa o tramite *cross-validation*.

Si osservi che con *molti* esempi la somma a sinistra tende a dominare sul regolarizzatore, mentre con *pochi* esempi il regolarizzatore ha maggiore effetto.

Il parametro di regolarizzazione è un esempio di iperparametro del modello.

Formalmente un **iperparametro** è un parametro usato per definire cosa/come si ottimizza e viene scelto in base a conoscenza pregressa, esperienza con problemi simili, o anche via **tuning** tramite *cross-validation*.

Ad esempio, nel caso degli *alberi* di decisione (binari), una misura della *complessità* è il numero di test, il quale dipende a sua volta dal numero delle foglie.

Nella minimizzazione della loss si può aggiungere una funzione delle dimensioni dell'albero T :

$$\left(\sum_{e \in Es} \text{loss}(\hat{Y}(e), Y(e)) \right) + \gamma * |T|$$

dove $|T|$ è la misura sopra indicata: un nuovo nodo-test risulta *utile* se riduce l'errore di γ .

Nel caso di *modelli con parametri* a valori reali, un regolarizzatore L_2 penalizza la somma dei quadrati dei parametri.

Ad esempio, nella cosiddetta **ridge regression**, esso viene sommato all'errore quadratico:

$$\left[\sum_{e \in Es} \left(Y(e) - \sum_{i=0}^m w_i * X_i(e) \right)^2 \right] + \lambda \sum_{i=0}^m w_i^2$$

Anche nella *regressione logistica*, si adotta un regolarizzatore L_2 puntando a minimizzare la seguente *log-loss*:

$$- \left[\sum_{e \in Es} \left(Y(e) \log \hat{Y}(e) + (1 - Y(e)) \log (1 - \hat{Y}(e)) \right) \right] + \lambda \left(\sum_{i=0}^m w_i^2 \right)$$

dove $\hat{Y}(e) = \text{sigmoid}(\sum_{i=0}^m w_i * X_i(e))$.

Nell'implementazione della *regolarizzazione* L_2 , si introduce il *passo*:

$$w_i \leftarrow w_i - \eta * \lambda * (b/|Es|) * w_i$$

con $b/|Es|$ perché λ è relativo all'intero dataset ma l'aggiornamento va operato su *ciascun batch* di dimensione b ; è anche possibile regolarizzare *dopo ogni iterata* su tutti gli esempi.

Si noti che il fattore $\eta * \lambda * (b/|Es|)$ cambia di rado per cui può essere pre-calcolato.

Un *regolarizzatore* L_1 si basa sulla somma dei valori assoluti dei parametri.

Ad esempio, nel modello detto LASSO si aggiunge $\lambda(\sum_i |w_i|)$ all'errore quadratico.

Usando la *log-loss*:

$$- \left[\sum_{e \in Es} \left(Y(e) \log \hat{Y}(e) + (1 - Y(e)) \log (1 - \hat{Y}(e)) \right) \right] + \lambda \left(\sum_{i=0}^m |w_i| \right)$$

con:

- Derivata parziale del termine $\text{sign}(w_i) = w_i/|w_i| \in \{-1, +1\}$: non definita in 0, ma non serve spostarsi da 0 se è già sul valore minimo;
- Si implementa facendo tendere a 0 ogni parametro di una costante: diventa nullo nel caso si scambi il segno del parametro;
- Nell'**iterative soft-thresholding**, GD per la regressione logistica, si ha:

$$w_i \leftarrow \text{sign}(w_i) * \max(0, |w_i| - \eta * \lambda * b/|Es|)$$

In caso di feature numerose, il regolarizzatore L_1 tende ad annullare molti pesi, facendo ignorare le relative feature, per cui diventa un meccanismo implicito per la **selezione delle feature**.

↑ Spiegazione di tutto l'ambaradan: fino ad ora, l'obiettivo del modello era solo minimizzare l'errore sui dati (Loss).

Ora l'obiettivo cambia. Vogliamo minimizzare una somma di due cose:

$$\text{Obiettivo} = \text{Errore sui Dati} + \lambda * \text{Complessità del Modello}$$

- **Errore** ($\sum \text{loss}$): “Quanto bene ho imparato i dati?”;
- **Regolarizzatore** (regularizer): “Quanto sono complicati i miei pesi?”;
- λ : è la manopola del volume:
 - Se λ è alto: mi importa solo della semplicità (rischio Underfitting);
 - Se λ è zero: mi importa solo dei dati (rischio Overfitting, torniamo alla situazione di prima);
 - Questo è un **iperparametro** che scegliamo noi (spesso con la cross-validation).

La tecnica chiamata **Ridge Regression** dice che la “complessità” è data dalla **somma dei quadrati dei pesi** (w_i^2). Aggiungendo questo valore, si ha un effetto di punizione molto alta per i pesi grandi, trovandoci così con dei pesi “schiacciati” verso lo zero, diventando molto piccoli, ma mai zero (il modello diventa più stabile e meno sensibile ai singoli dati rumorosi, “lisciando” la curva).

Tramite il LASSO, decidiamo che adesso la “complessità” è data dalla **somma dei valori assoluti dei pesi** ($|w_i|$). A differenza della L_2 che rimpicciolisce tutto, la L_1 ha la capacità matematica di **azzerare completamente alcuni pesi** (come? Tramite il passo $w_i \leftarrow \text{sign}(w_i) * \max(0, |w_i| - \eta * \text{lambda} * b/|Es|)$).

Se il modello azzerava il peso di una feature, significa che sta letteralmente cancellando quella colonna dai dati, dicendo “Questa informazione è inutile”, quindi fa automaticamente una **feature selection**.

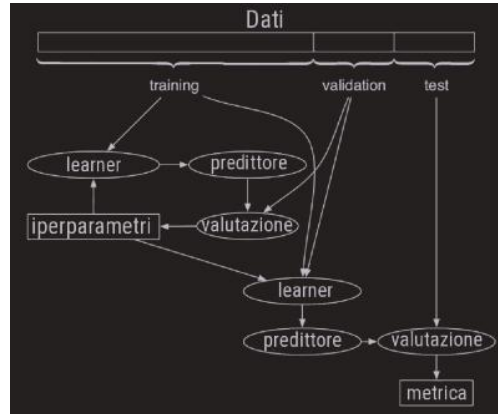
Riguardo il passo della L_1 , questo è come lo si calcola:

1. Si prende il peso w_i ;
2. Si sottrae un pezzettino fisso (la penalità);
3. Se il risultato scende sotto zero (o cambia segno), lo si **tronca a zero**.

Questo conferma matematicamente perché la LASSO tende a creare molti zeri (modelli “sparsi”).

Sintesi: la regolarizzazione serve a evitare l'overfitting, penalizzando i modelli complessi (con pesi alti), la L_2 usa i quadrati (w_i^2) rendendo i pesi piccoli, buono per stabilità generale, mentre la L_1 usa i valori assoluti ($|w_i|$) rendendo i pesi zero, buono per eliminare le feature inutili. Più λ è alto, più il modello è semplice (pesi piccoli/nulli).

Risulta difficile determinare la giusta complessità dei modelli da impiegare *prima* di aver visto i dati: essa dipende dal *numero* di parametri e/o da altri *iperparametri* dell'algoritmo. A tale scopo, si può adottare uno *schema generale* di apprendimento e valutazione, sintetizzato nella figura seguente:



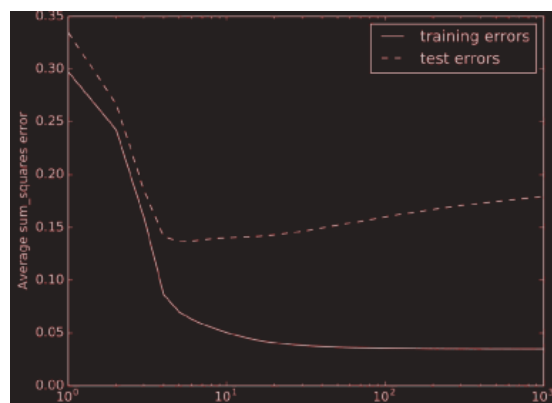
Si noti come il test set venga messo da parte per essere usato solo per la valutazione finale. Dev'essere composto da esempi *mai visti*, non utilizzati nella fase di addestramento, da impiegare come surrogato di nuovi esempi futuri.

Nella **Cross Validation** (CV) si trattiene *parte* dei dati di training per valutare un modello appreso sulla base del resto degli esempi di training.

Si dividono i dati *non di test* in due parti: insieme di *training*, per l'apprendimento dei modelli e un insieme di **validazione** per valutare diversi modelli (o diverse configurazioni degli iperparametri). Trovato il modello migliore, si userà il dataset (escluso il test set) per apprendere un modello.

La CV va ripetuta più volte scambiando i ruoli degli esempi per ottenere modelli diversi. Alla fine, si sceglierà il modello migliore ossia quello con la migliore configurazione degli iperparametri.

Uso dell'errore di validazione: considerando il grafico precedente, l'errore sul training set decresce al crescere delle dimensioni dell'albero mentre l'errore sul validation set (e sul test set) si abbassa e poi prende a risalire.

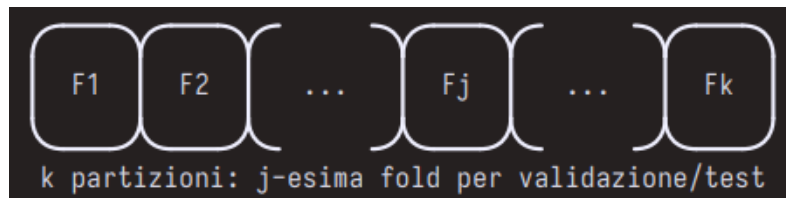


La CV aiuta a scegliere la configurazione corrispondente al minimo dell'errore sul validation set. L'*ipotesi* è che si abbia lo stesso punto di minimo anche per l'errore calcolato sul test set.

Si può considerare come iperparametro da regolare anche il numero di passi di training prima di restituire un predittore, una forma di **early stopping**.

Tipicamente l'addestramento ha bisogno di molti esempi per avere modelli migliori, ma ciò non deve andare a scapito del set per la validazione.

Disponendo solo di dataset limitati, si può puntare al riutilizzo degli esempi per il training e la validazione per regolare i parametri (quindi alla complessità) del modello appreso, scambiandone i ruoli:



Nella **k-Fold Cross Validation** i passi per valutare una scelta di parametri sono:

- Si suddivide casualmente il dataset in k parti di uguali dimensioni, dette **fold**;
- Si valutano i modelli con le diverse impostazioni: addestrando il modello k volte, ognuna con una distinta fold usata per la validazione;
- Si determinano e restituiscono le impostazioni ottimali.

Ad esempio, ripetendo per $k = 10$ volte, si addestrano modelli sul 90% e si valida sul 10% dei dati considerati.

In tal modo la validazione coinvolge ogni esempio esattamente in *una* fold.

Nel caso estremo di un numero ancor più limitato di esempi, si può adottare una k -fold CV con k pari al numero di esempi di training, detta **Leave-One-Out Cross Validation**.

Avendo n esempi di training, si apprendono n modelli: per ogni esempio e , si usano tutti gli altri per addestrare un modello da valutare infine su e .

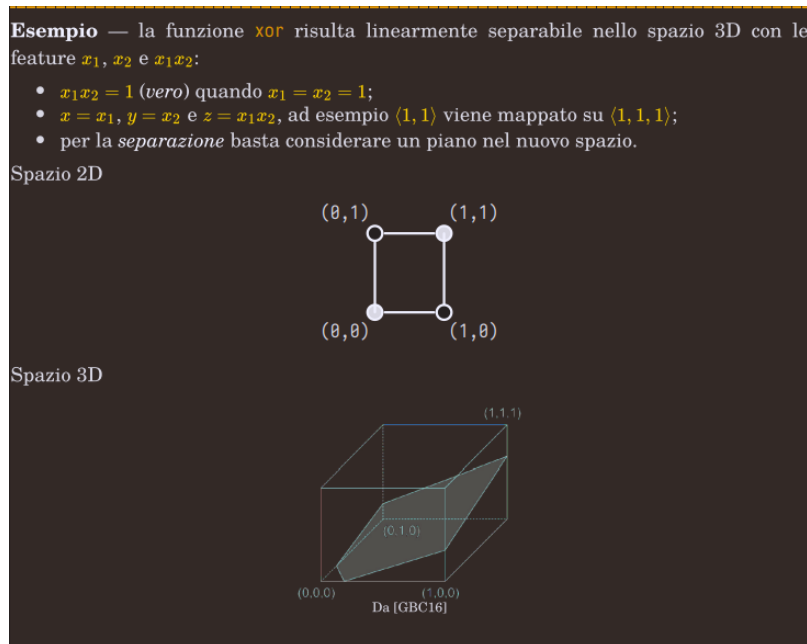
La complessità cresce col numero degli esempi di training: il metodo diventa *poco pratico* con modelli prodotti in fasi di training *indipendenti* fra loro, mentre risulta adeguato se il modello creato in una esecuzione può essere *modificato* in modo efficiente nel successivo, sostituendo un solo esempio con un altro.

Modelli Compositi

I modelli d'apprendimento visti fin qui forniscono le basi per altre tecniche di apprendimento. Sebbene qualsiasi funzione discreta possa essere rappresentata attraverso alberi di decisione, alcune funzioni anche semplici, richiedono alberi molto complessi.

Anche i modelli lineari possono risultare spesso limitati, a meno che non si introducano meccanismi di costruzione di *nuove feature*.

Una possibile estensione considera l'uso di funzioni *non lineari* degli input originari in modo da creare nuovi input per i modelli lineari. La costruzione di nuove feature aumenta la *dimensionalità* dello spazio e funzioni che non erano lineari in origine possono diventarlo nel nuovo spazio o almeno rendere i problemi *linearmente separabili*.



In questo esempio rappresentiamo lo XOR in 3 dimensioni, inserendo la nuova coordinata come la moltiplicazione delle due variabili. In questo modo riusciremo, tramite un piano, a dividere lo spazio in 3D e ad avere le soluzioni accettate fuori da quelle non accettate (il problema è diventato linearmente separabile).

Si possono definire modelli lineari su nuovi spazi di feature multidimensionali.

Tali metodi si basano sulla trasformazione in nuovi problemi di ottimizzazione la cui soluzione dipende da una funzione **kernel** definita come prodotto nel nuovo spazio vettoriale:

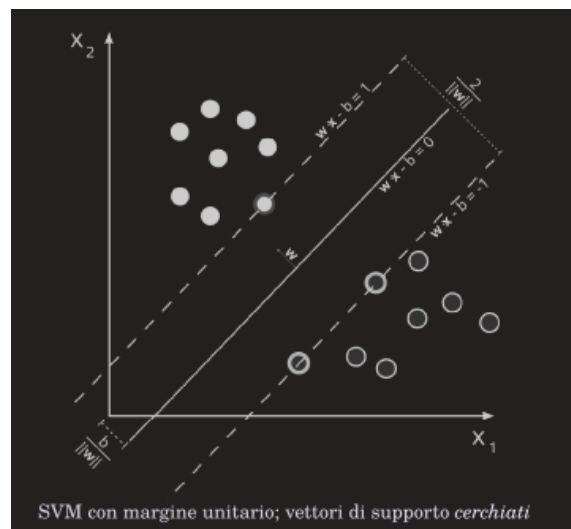
$$\mathcal{K}(\mathbf{x}, \mathbf{z}) = \langle \phi(\mathbf{x}), \phi(\mathbf{z}) \rangle$$

con $\phi: \mathbf{X} \rightarrow \mathbf{E_s}$ che rappresenta la trasformazione implicita/esplicita verso $\mathbf{E_s} = \mathbb{R}^n$, $n > m$, detto *embedding space*.

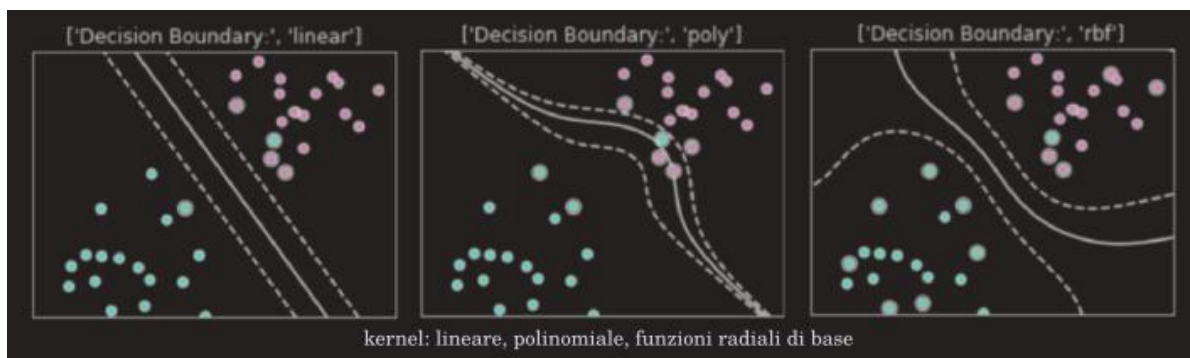
Tale approccio risulta utile in casi di non separabilità lineare nello spazio originario delle $\mathbf{X}$. Il kernel generalizza il concetto di *prodotto vettoriale*, interpretabile come misura di *similarità* basata sulle nuove feature, ad esempio, prodotto di feature aggiuntive/sostitutive rispetto a quelle iniziali.

Un SVM è un classificatore basato su *kernel*: esso definisce un iperpiano, *superficie di decisione* ottimale, che divide gli esempi positivi da quelli negativi. In particolare, si considera il *margin*, ovvero la distanza minima dagli esempi dell'iperpiano: la SVM mira a trovare l'iperpiano di *massimo margin*. Gli esempi sul margin, che determinano tale iperpiano, si dicono vettori di *supporto*.

La soluzione (parametri dell'iperpiano) sarà calcolata sulla base di prodotti applicati a tali vettori nello spazio di embedding (ossia tramite kernel). Il modello è *sparso* perché determinato da pochi vettori *di supporto*, i più vicini all'iperpiano, per cui, pur aumentando la dimensionalità, può evitare il sovradattamento.



Kernel diversi (ovvero diverse funzioni ϕ) determinano diversi separatori:



È possibile usare modelli simili anche per risolvere problemi di regressione.

Reti neurali ammettono input alla funzione lineare di appiattimento che sono a loro volta frutto di funzioni lineari compresse con coefficienti da determinare: con diversi layer fatti in questo modo si possono rappresentare *funzioni più complesse*.

Gli **alberi di regressione** sono alberi con funzioni alle foglie, che possono costituire modelli lineari di regressione, potendo così rappresentare *funzioni lineari per parti*. Delle estensioni possono considerare foglie contenenti reti neurali o altri modelli non lineari.

In fase di apprendimento, la scelta delle condizioni può essere fatta adottando un criterio di minimizzazione della varianza (l'errore numerico). Per la predizione dell'output per un nuovo esempio, a partire dalla radice, lo si instrada giù per l'albero fino a individuare una foglia; quindi, si userà il modello di regressione in essa contenuto.

In generale, nell'**ensemble learning** si *combinano* le predizioni di un dato numero di modelli (semplici), ciascuno appreso da un learner *di base*, a volte anche da parte del dataset. Ad esempio, nelle *foreste* di alberi di decisione, le predizioni finali si otterranno attraverso medie, ovvero meccanismi di voto.

Vi sono diversi approcci di *ensemble learning*, accomunati dall'essere costituiti da un certo numero di modelli-base addestrati sui dati e poi combinati attraverso meccanismi di voto o medie di tipo diverso.

La **random forest** è un modello composto da più alberi di decisione. L'*idea* è quella di addestrare un certo numero di alberi su parti differenti del dataset. Per ogni esempio da classificare ognuno darà una predizione; le predizioni verranno aggregate per formare la predizione finale per l'esempio dato.

L'efficacia dipende dalla *diversità* degli alberi generati in modo da ottenere da ciascuno predizioni anche diverse. A tale scopo, per ogni albero si potrebbe considerare un sottoinsieme *casuale* di feature (ad esempio $1/m$ delle feature).

In alternativa, le feature per i test nei nodi possono essere scelte in un *insieme ristretto*, variabile per ciascun albero (o nodo addirittura).

Nel **bagging** si considerano diversi sottoinsiemi degli esempi di training: dati m esempi di training, ogni modello-base ne usa un piccolo numero. Ad esempio, si possono considerare sottoinsiemi di m esempi estratti casualmente con rimpiazzo: in ognuno, risulteranno esempi assenti oppure anche duplicati, il che porta a circa il 63.2% degli esempi originali per ciascun modello-base. Nella *predizione* finale si dovrà mediare; ad esempio, si può considerare la *classe* di maggioranza sulle

predizioni, ossia un meccanismo di *voto* basato sulla *moda*, che predica la classe più probabile/predetta da più modelli-base.

Per foreste di alberi di regressione sono previste medie di tipo diverso .

Nel **boosting** i modelli sono appresi in *sequenza* imparando dagli errori dei precedenti: si usa, in genere, lo stesso tipo di *learner di base* (ma si possono anche differenziare) quali, ad esempio, gli alberi di decisione o funzioni lineari compresse. Essi vanno via via adattati agli esempi sui quali il modello precedente commette errori: è come se gli esempi fossero *ordinati* in base agli errori.

Come nel caso precedente, la predizione *finale* si ottiene aggregando le predizioni dei modelli prodotti nelle diverse iterati: somma, media pesata, moda...

Si osservi come i modelli-base possano anche essere *deboli*, non necessariamente ottimali, si chiamano infatti anche **weak learner**: essi devono giusto superare le prestazioni di un predittore casuale. L'ensemble risultante avrà prestazioni migliori garantite rispetto a ciascuno di essi. Fra i più noti algoritmi troviamo ADABOOST.

Il **functional gradient boosting** prevede un iperparametro K , che corrisponde al numero di iterate da ripetere e quindi di modelli-base di regressione da apprendere. La *predizione finale* viene operata in funzione degli input nel modo seguente:

$$p_0 + d_1(X) + \dots + d_K(X)$$

dove p_0 rappresenta la predizione iniziale, ad esempio la media, e d_i è la differenza dalla predizione precedente.

L' i -esima predizione si può scrivere:

$$p_i(X) = p_0 + d_1(X) + \dots + d_i(X) = p_{i-1}(X) + d_i(X)$$

dove d_i viene costruito dal weak learner minimizzando l'errore di p_i (fissato p_{i-1}):

$$\sum_e \text{loss}(p_i(e), Y(e)) = \sum_e \text{loss}(\hat{d}_i(e), Y(e) - p_{i-1}(e))$$

Usando una qualsiasi loss, anche una per target continui, ma non la log-loss; $\hat{d}_i(e)$ deve ottimizzare $Y_i(e) - p_{i-1}(e)$.

Si può operare come con un dataset con output modificati sottraendo le predizioni precedenti: ogni modello tende a correggere gli errori dei precedenti.

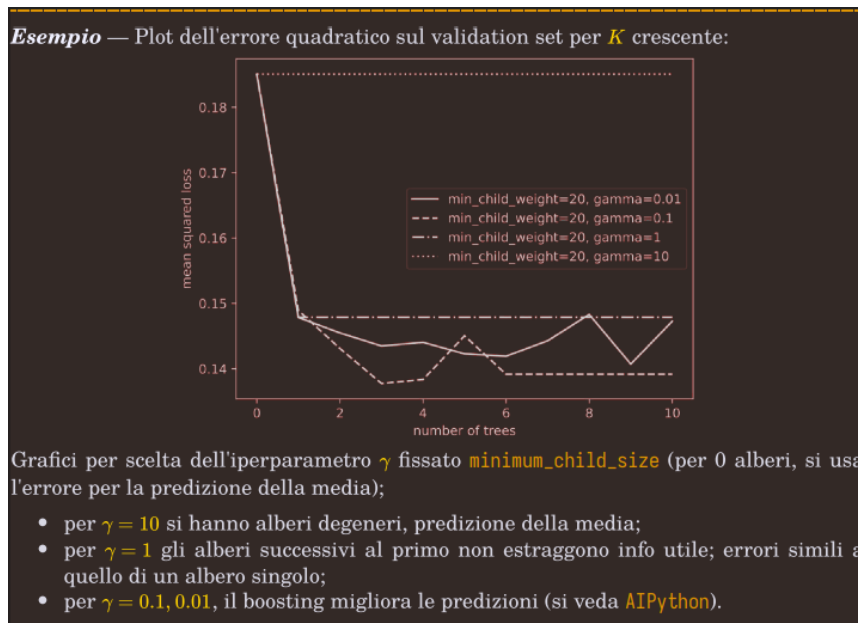
```

=====
procedure Boosting_learner( $Xs, Y, Es, L, K$ )
  Input
     $Xs$ : insieme di feature di input
     $Y$ : feature obiettivo
     $Es$ : insieme di esempi di training
     $L$ : learner di base
     $K$ : numero di componenti dell'ensemble
  Output
    funzione per fare predizioni sugli esempi
   $media \leftarrow \sum_{e \in Es} \frac{Y(e)}{|Es|}$ 
  define  $p_0(e) = media$ 
  for each  $i = 1$  to  $K$  do
     $E_i \leftarrow \{ \langle Xs(e), Y(e) - p_{i-1}(e) \rangle \mid e \in Es \}$ 
     $d_i \leftarrow L(E_i)$ 
    define  $p_i(e) = p_{i-1}(e) + d_i(e)$ 
  return  $p_K$ 
=====

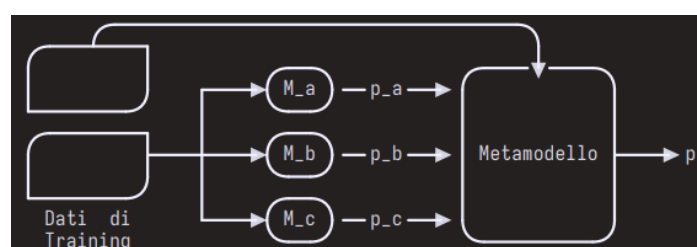
```

con:

- p_i è la funzione di predizione sugli esempi;
- E_i è un nuovo insieme di esempi, generabile anche on demand: per ogni $e \in Es$, l'ultima predizione, $p_{i-1}(e)$, viene sottratta da $Y(e)$;
- d_i si calcola applicando il modello-base a E_i .



Lo **stacking**, o *stacked generalization*, è un modello ensemble costituito da *modelli-base* di tipi diversi, come ad esempio alberi, modelli lineari, SVM, ecc... e un *meta-learner* che apprende come aggregare al meglio le decisioni. Uno schema generale è rappresentato in figura:



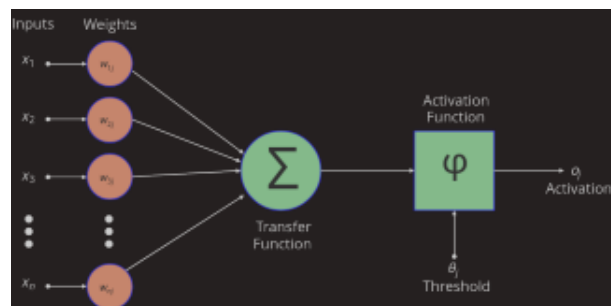
8 – Reti Neurali e Apprendimento Profondo

Introduzione

Attualmente ci si basa sull'apprendimento delle feature, specie se si disponga di ingenti quantitativi di dati. I metodi per tali scopi apprendono feature utili per il problema da risolvere anche se spesso queste si rivelano poco interpretabili.

Un altro vantaggio dell'apprendimento di feature è che possono anche essere reimpiegate nella soluzione di problemi affini.

Imparare le feature è un obiettivo perseguito attraverso il cosiddetto *feature representation learning* ed è in genere svolto attraverso **reti neurali artificiali** (ANN) che sono ispirate alla struttura dei *neuroni* del cervello umano. Le **unità** artificiali risultano diverse ai corrispettivi biologici in quanto molto più semplici e meno numerose di quelle nei cervelli umani.



Tali modelli sono impiegati per risolvere problemi che comportano un ragionamento di *bassi livello*, con molti dati disponibili, come l'*interpretazione delle immagini* e così via, risultando molto *flessibili* e capaci di *inventare* nuove feature.

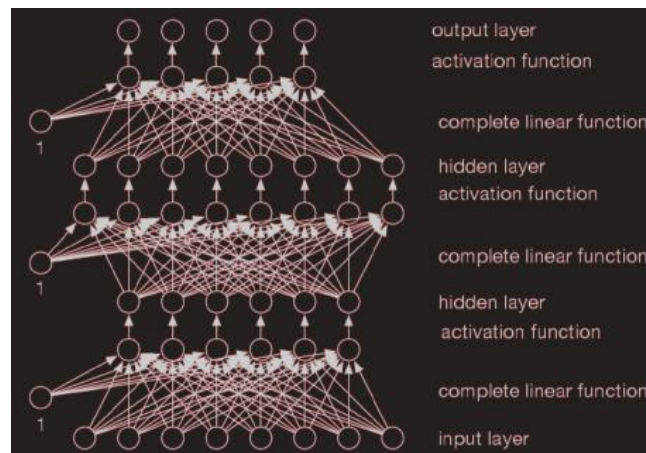
Reti Neurali Artificiali Feed Forward

Essenzialmente, una *rete neurale feed-forward* implementa una funzione di predizione come segue: dato l'input (vettoriale) x :

$$f(x) = f_n \left(f_{n-1} \left(\cdots f_2(f_1(x)) \right) \right)$$

- Ogni **layer/strato** è costituito da una funzione f_i che mappa un vettore (array o lista) di input su un vettore di output;
- Ciascuna componente del vettore di output di un layer costituisce un'*unità*;
- La *profondità* della rete è n , con l'ultimo strato, f_n , detto di *output* della rete mentre gli altri layer sono detti *nascosti/hidden*, tranne quello di *input*, con x .

In genere le ANN hanno più feature di input ma anche più feature-target, tipicamente a valori *reali*; feature discrete possono essere trasformate in variabili-indicatrici o feature ordinali.



Ogni layer di unità opera in funzione dei precedenti: il layer di *input* ha un'unità per feature di input e viene avvalorato da ciascun esempio: un layer *nascosti* prevede la composizione di una funzione *lineare* f_i con una funzione ϕ di *attivazione* non-lineare, da cui si calcola l'output $out[j]$ in funzione dei valori in input $in[k]$ allo strato

$$out[j] = \phi \left(\sum_k in[k] * w[k, j] \right)$$

si noti l'input aggiuntivo costante $i[0] = 1$ per il *bias* (o *termine noto*); occorrerà apprendere ciascun peso $w[k, j]$, uno per ogni (arco) coppia input-output del layer. Si noti che layer lineari *consecutivi* sarebbero inutili perché trasformabili in unico layer lineare.

Le funzioni di attivazione ϕ , tipicamente, sono (quasi ovunque) differenziabili. Ad esempio, si può usare un *rettificatore* o *Rectified Linear Unit* (ReLU) definito

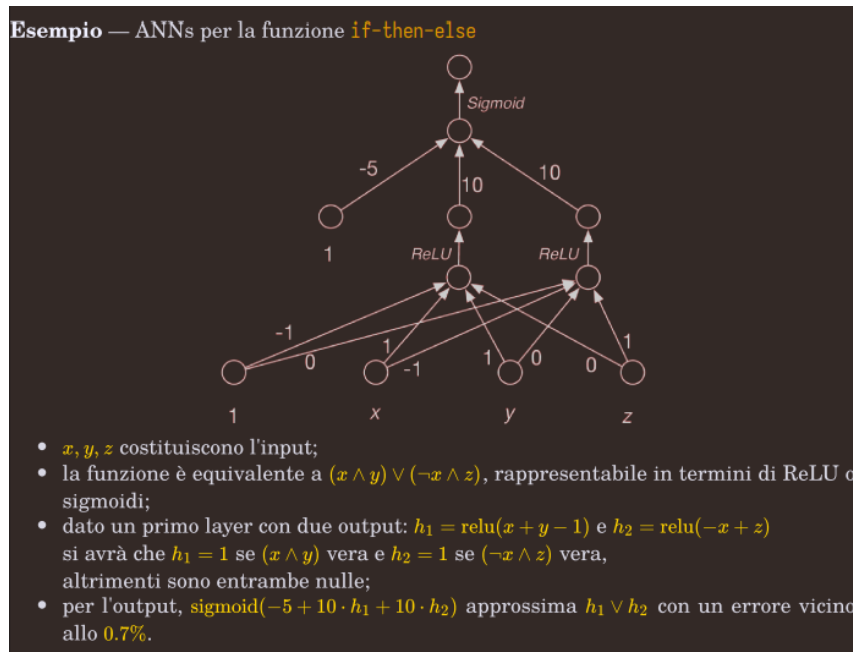
$$\phi(x) = \max(0, x)$$

assumendo che la derivata sia nulla in $x = 0$.

La scelta di ϕ per il layer di output può variare in base al tipo di y e alla loss a ottimizzare:

- Per y reale e loss quadratica media si può usare $\phi(x) = x$, funzione *identità*;
- Per y booleana e log loss binaria, tipicamente si usa la *sigmoide* $\phi(x) = 1/(1 + e^{-x})$, giustificabile anche in termini probabilistici;
- Per y categorica e log loss categorica si usa invece una *softmax*, considerando un *output layer* con una unità per ciascun valore di y .

Queste scelte comportano derivate semplici dell'errore proporzionali a $\hat{Y}(e) - Y(e)$.



L'ampiezza di un layer è pari alle dimensioni del vettore di output. L'ampiezza della rete è pari all'ampiezza massima fra i suoi layer. La sua *profondità* è pari al numero di layer (scelta di progetto). Le dimensioni dell'input dipendono dal problema.

Si può dimostrare che una rete con un solo layer nascosto, purché sufficientemente ampio, può approssimare qualsiasi funzione continua su un intervalli di numeri reali o discreta con input/output discreti. I layer a livelli bassi costituiscono feature utili a quelli dei livelli più alti, cosa che li rende più semplici.

L'algoritmo **back-propagation** (BP) implementa la DG stocastica per determinare i pesi del modello: per ogni esempio e del batch, si aggiorna ogni peso w in ragione di $-\frac{\partial}{\partial w} \text{error}(e)$ usando le proprietà delle derivate.

La predizione per e , con feature di input x_e , sarà:

$$f(e) = f_n \left(f_{n-1} \left(\cdots f_2(f_1(x_e)) \right) \right)$$

con le f_i parametrizzate dai pesi. Supponendo che $v_i = f_i \left(f_{i-1} \left(\cdots f_2(f_1(x_e)) \right) \right)$, ossia $v_i = f_i(v_{i-1})$, posto $v_0 = x_e$, i valori v_i per $1 \leq i \leq n$ possono essere calcolati in una passata attraverso le funzioni annidate.

Definire le derivate per gli aggiornamenti è più difficile perché è più difficile calcolare la variazione dell'errore determinata dai pesi di livelli inferiori a quello di output.

Sia w uno dei pesi nella definizione di una certa f_j (si noti che f_i non dipende da w , per $i \neq j$). La derivata dell'errore rispetto al peso è data da:

$$\begin{aligned}
 \frac{\partial}{\partial w} \text{error}(f(e)) &= \text{error}'(v_n) * \frac{\partial}{\partial w} f_n(v_{n-1}) \\
 &= \text{error}'(v_n) * \frac{\partial}{\partial w} f_n(f_{n-1}(v_{n-2})) \\
 &= \text{error}'(v_n) * f'_n(v_{n-1}) * \frac{\partial}{\partial w} (f_{n-1}(v_{n-2})) \\
 &= \text{error}'(v_n) * f'_n(v_{n-1}) * f'_{n-1}(v_{n-2}) * \frac{\partial}{\partial w} (f_{n-2}(v_{n-3})) \\
 &= \text{error}'(v_n) * f'_n(v_{n-1}) * f'_{n-1}(v_{n-2}) * \dots * \frac{\partial}{\partial w} (f_j(v_{j-1}))
 \end{aligned}$$

con f'_i derivata di f_i rispetto ai suoi input. L'espansione prosegue fino a f_j .

L'ultima derivata parziale non è istanza della regola di concatenazione non essendo v_{j-1} funzione di w , quindi si usa la regola di decomposizione lineare e w viene moltiplicato per la componente di v_{j-1} appropriata.

BP determina l'aggiornamento per ogni peso in due passate per ogni esempio:

- *Predizione*: per ogni layer, dati i valori degli input, si calcola il valore degli output, v_i (nell'algoritmo che segue, ogni v_i nell'array *values* associato al layer);
- *Retro-propagazione*: andando a ritroso attraverso i layer, si determina l'aggiornamento di ciascun peso (dei layer lineari); si calcolano (a partire da 0) $\text{error}'(v_n) * \prod_{i=0}^k f'_{n-i}(v_{n-i-1})$ per essere passati ai layer inferiori; tale input costituisce il *termine d'errore* per un layer, combinato con i valori calcolati al passo di predizione per aggiornare tutti i pesi del layer e calcolare il termine d'errore per il layer inferiore.

BP combina la SGD con la *programmazione dinamica* memorizzando i risultati intermedi.

Un *layer* viene costruito come modulo indipendente che implementa le due passate, predizione e retro-propagazione, con l'aggiornamento dei pesi dopo ogni *batch* di esempi.

La *classe Dense* implementa una funzione lineare *densa* in cui ogni unità di output è connessa a tutte quelle di input. Dato l'array w dei pesi, $w[i, j]$ è il peso per la connessione dell'unità di input i all'output j e $w[n_i, j]$ costituisce il bias per l'output j

(associato all'input implicito 1). I metodi output e Backprop vengono chiamati su ogni esempio.

- *output(in)* restituisce i valori di output per i valori di input del vettore *in*;
- *Backprop(erore)* restituisce il vettore degli errori in input e aggiorna i gradienti;
- *update()* aggiorna i pesi per un batch di esempi.

ogni classe memorizza *in* e *out* se utili a *Backprop*.

```

procedure Neural_network_learner(Xs,Ys,Es,functions, $\eta$ ,batch_size)
  Input
    Xs: feature di input,  $Xs = (X_1, \dots, X_n)$ 
    Ys: feature target
    Es: insieme di esempi di training
    functions: sequenza di funzioni che definiscono la rete
    batch_size: n. di esempi in ogni batch
     $\eta$ : learning rate (passo della GD)
  repeat
    batch  $\leftarrow$  campione casuale di batch_size esempi
    for each esempio e  $\in$  batch do
      for each unità di input i do values[i]  $\leftarrow X_i(e)$ 
      for each fun  $\in$  functions in ordine ascendente do
        values  $\leftarrow$  fun.output(values)
        for each unità di output j do error[j]  $:= \phi_o(\text{values}[j]) - Ys[j]$ 
        for each fun  $\in$  functions in ordine discendente do
          error  $:=$  fun.Backprop(error)
      for each fun  $\in$  functions che contiene pesi do
        fun.update()
  until terminazione
  
```

- *functions* è una sequenza di funzioni (parametriche) che definisce la rete, tipicamente, per ciascun layer, una lineare e una di attivazione, implementata da una classe che implementi le due passate e memorizzi l'informazione necessaria;
- La prima funzione del layer più basso ha input corrispondenti alle feature di input; per le successive, il numero di input è pari al numero di output della funzione precedente, mentre il numero output della funzione finale è pari al numero di feature target, o al numero di valori per una feature di output categorica (non binaria);
- Si assume che il tipo di output della funzione di attivazione sia utile alla funzione di errore (loss), in modo che la derivata sia proporzionale al valore predetto meno il valore effettivo, questo vale per l'identità, la sigmoideale e la softmax: ϕ_0 è la funzione di attivazione finale, usata nell'algoritmo.

```
class ReLU()
    method output(in)
        for each  $i : 0 \leq i < n_i$  do  $out[i] := \max(0, in[i])$ 
        return out
    method Backprop(error)
        for each  $i : 0 \leq i < n_i$  do
             $ieror[i] := error[i]$  if  $(in[i] > 0)$  else 0
        return ierror

class sigmoid()
    method output(in)
        for each  $i : 0 \leq i < n_i$  do  $out[i] := 1/(1 + \exp(-in[i]))$ 
        return out
    method Backprop(error)
        for each  $i : 0 \leq i < n_i$  do  $ieror[i] := out[i] * (1 - out[i]) * error[i]$ 
        return ierror
```

Esempio — Rete vista in precedenza per l'if-then-else con i dati visti nel cap.7

- si invoca `Neural_network_learner` assegnando a `functions` la lista `[Dense(3,2),ReLU(),Dense(2,1)]`
- un run con 10000 epoche, learning rate = 0.01 e batch di 8 esempi produce i pesi seguenti (3 cifre significative):
 - $h_1 = \text{relu}(2.47 * x + 0.0000236 * y - 2.74 * z + 2.74)$;
 - $h_2 = \text{relu}(3.62 * x + 4.01 * y + 0.228 * z - 3.84)$;
 - $output = \text{sigmoid}(-4.42 * h_1 + 6.64 * h_2 + 4.95)$;
- la funzione risultante approssima con un errore nel caso pessimo dell'1%;
 - predizione con la massima log loss 0.99 per l'esempio $\neg x \wedge \neg y \wedge z$, per il quale doveva essere 1
- al variare dei run si ottengono diversi pesi
- in tale caso semplice si ottiene un target interpretabile (approssimativamente):
 - h_1 positiva a meno che $x = 0, z = 1$
 - h_2 grande solo quando $x = 1, y = 1$

Nell'apprendimento *sub-simbolico* si ha un *contrasto apparente* con l'ipotesi di sistema di simboli fisico intrinseco: la rete codifica un significato che si può solo intravedere attraverso i valori delle unità nascoste, ma a tali unità non è associato un *significato preciso*. Solo in alcuni casi si può interpretare cosa rappresentino ed esprimerlo in forma concisa.

9 – Ragionamento su Modelli di Conoscenza Incerta

Probabilità

Nel mondo reale spesso si devono prendere decisioni in base a informazioni che sono però *incomplete*. In tali casi è difficile capire l'*esatto* stato del mondo.

Per prendere decisioni un KBS deve cercare di sfruttare *tutta* la conoscenza disponibile. Essendo tale conoscenza limitata, il sistema dovrà essere in grado di ragionare sotto **incertezza**, facendo ipotesi e valutandone l'attendibilità.

Se non si può assumere una *conoscenza completa* del mondo, per prendere decisioni spesso si formulano diverse ipotesi.

L'**incertezza** può essere caratterizzata come **ontologica** o **epistemologica**. Nel primo caso essa riguarda il mondo in sé, mentre nel secondo l'incertezza concerne le credenze sullo stato del mondo: essendo spesso le situazioni con completamente conosciuti, ci si possono come fare idee/valutazioni (*belief*) soggettive.

Il problema è: come aggiornare tale *credito* in presenza di nuova informazioni disponibile? Si devono considerare valutazioni nei due momenti successivi: a **priori**, ossia prima di qualsiasi osservazione, e a **posteriori**, ossia dopo la scoperta d'informazione, tipicamente da osservazioni, il che porta all'aggiornamento delle credenze sullo stato del mondo.

La misura della credibilità rientra nella prospettiva che si chiama **bayesiana** o **soggettiva**: lo studio di come la conoscenza impatti sulle credenze del soggetto. Soggetti diversi potranno assumere diverse probabilità. Tale prospettiva è diversa da quella *oggettiva* detta anche *frequentista*.

La *credibilità* di una proposizione α viene misurata con un valore in $[0,1]$: si ha probabilità *nulla* se si crede che α sia del tutto falsa e che nessuna nuova evidenza possa cambiarla; si avrà una probabilità pari a 1 se α si ritiene assolutamente vera; altrimenti la probabilità sarà un valore in $]0,1[$ che misura l'incertezza sulla credibilità della sua verità; ciò non significa che α sia solo parzialmente vera, bensì che la sua verità sia sconosciuta.

Indipendenza

Per definire distribuzioni su molte variabili serve molta conoscenza. Gli assiomi della probabilità risultano deboli per certi scopi: pongono pochi vincoli sulle probabilità condizionate da sfruttare.

Ad esempio, con n variabili binarie, ci saranno $2^n - 1$ probabilità da conoscere per avere una completa distribuzione dalla quale derivare quelle condizionate. Serve un DB molto grande per determinarle anche in forma approssimata.

Per limitare il quantitativo d'informazione richiesta, si può assumere che una variabile dipenda direttamente *solo* da alcune altre. Altre assunzioni dovrebbero far sì che si richiedano meno dati per specificare un modello: con strutture più semplici il ragionamento sarà svolto in modo più efficiente.

In generale $P(h | e) \in]0,1[$ non determina vincoli sui valori di $P(h | f \wedge e)$, se non in casi-limite: $P(h | f \wedge e) = 1$ quando f implica h oppure $P(h | f \wedge e) = 0$ quando f implica $\neg h$.

Spesso è però disponibile un tipo comune di *conoscenza qualitativa* riassumibile con:

$$P(h | e) = P(h | f \wedge e)$$

cioè f risulta irrilevante per la probabilità di h data e .

Estendendo l'idea a un dato insieme di variabili Zs : X si dice *condizionatamente indipendente* da Y dato Zs se e solo se:

$$P(X | Y, Zs) = P(X | Zs)$$

cioè $\forall x \in \text{dom}(X) \forall y \in \text{dom}(Y) \forall z \in \text{dom}(Zs)$, se $P(Y = y \wedge Zs = z) > 0$ allora

$$P(X = x | Y = y \wedge Zs = z) = P(X = x | Zs = z)$$

ossia: il noto valore di ogni variabile in Zs , conoscere il valore di Y non influenza la credibilità dell'assegnazione di un valore a X . L'indipendenza condizionata è spesso *facile* da accertare e torna utile nell'*inferenza*. È, invece, più raro disporre di una tabella delle probabilità dei mondi da cui verificare l'indipendenza per via numerica.

Proposizione – Se le probabilità condizionate sono ben definite, i seguenti enunciati sono equivalenti:

1. X è condizionatamente indipendente da Y data Z ;
2. Y è condizionatamente indipendente da X data Z ;
3. $\forall x, y, y', z$:

$$P(X = x | Y = y \wedge Z = z) = P(X = x | Y = y' \wedge Z = z)$$

cioè: noto il valore di Z , cambiare Y non influenza la credibilità del valore di X ;

4. $P(X, Y | Z) = P(X | Z) * P(Y | Z)$.

X e Y incondizionatamente **indipendenti** se

$$P(X, Y) = P(X) * P(Y)$$

Ossia condizionatamente indipendenti ma senza osservazioni date: ciò non implica che siano pure condizionatamente indipendenti, data qualche altra evidenza Z . Raramente tale proprietà può essere determinata per via numerica da dati disponibili.

Belief Network

Poter fare assunzioni di indipendenza condizionata consente una rappresentazione concisa dei domini.

Data una variabile X , solo alcune variabili influenzano *direttamente* il suo valore l'insieme di variabili che la influenzano localmente si chiama *Markov blanket* e tale località può essere sfruttata: date queste variabili, X sarà condizionatamente indipendente dalle altre. Le assunzioni di *indipendenza* condizionata inducono un *ordinamento* delle variabili che può essere rappresentato attraverso un *grafo orientato*.

Una *rete bayesiana* è un modello (a grafo) che evidenzia la dipendenza condizionata fra variabili. Definito un *ordinamento totale* sull'insieme delle sue variabili $\{X_1, \dots, X_n\}$, la *distribuzione* di probabilità *congiunta* potrà essere decomposta in termini di probabilità condizionate, tramite la *chain rule*:

$$P(X_1 = v_1 \wedge X_2 = v_2 \wedge \dots \wedge X_n = v_n) = \prod_{i=1}^n P(X_i | X_1, \dots, X_{i-1})$$

Ossia:

$$P(X_1, X_2, \dots, X_n) = \prod_{i=1}^n P(X_i | X_1, \dots, X_{i-1})$$

I **genitori** di ciascuna variabile X_i saranno costituiti dal sottinsieme minimale di *predecessori* di X_i nell'ordinamento totale, $parents(X_i) \subseteq \{X_1, \dots, X_{i-1}\}$, tale che gli altri predecessori di X_i siano condizionatamente indipendenti da X_i dato $parents(X_i)$. Così X_i dipende dai genitori, ma è indipendente dagli altri predecessori:

$$P(X_i | X_1, \dots, X_{i-1}) = P(X_i | parents(X_i))$$

Quando più insiemi minimali soddisfano tale condizione, allora si può operare una scelta casuale. Alcuni dei predecessori possono dipendere deterministicamente da altri.

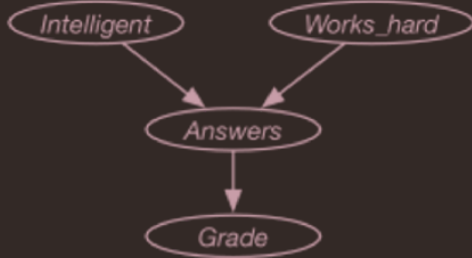
Riscrivendo la *chain rule*, la **fattorizzazione** della *congiunta* sarà:

$$P(X_1, X_2, \dots, X_n) = \prod_{i=1}^n P(X_i \mid \text{parents}(X_i))$$

Formalmente, una **belief network** o **rete bayesiana** (BN) è rappresentata da un grafo aciclico orientato (DAG) con una variabile, X , (con un suo dominio) per ciascun nodo, archi verso X da ciascuno dei nodi relativi ai suoi genitori $\text{parents}(X)$ e quindi una distribuzione condizionata di probabilità (CPD) $P(X \mid \text{parents}(X))$ (a priori in caso di X senza genitori).

Il grafo è *aciclico* per costruzione con una fattorizzazione che dipende dall'ordinamento (i genitori vanno scelti tra i predecessori): a diversi ordinamenti corrisponderanno diverse BN (alcuni ordinamenti portano a reti con meno archi). Una BN codifica relazioni di *indipendenza* condizionata fra variabili: dati i genitori, ogni variabile è indipendente da tutte le variabili che non sono sue discendenti.

Esempio — Dato l'ordinamento *Intelligent, Works_hard, Answers, Grade* delle variabili in un esempio precedente:



```

graph TD
    Intelligent --> Answers
    Works_hard --> Answers
    Answers --> Grade
  
```

- *Intelligent* non ha predecessori, quindi nemmeno genitori;
- *Works_hard* è indipendente da *Intelligent* e $\text{parents}(\text{Works_hard}) = \emptyset$;
- *Answers* dipende da *Intelligent* e *Works_hard*:
 $\text{parents}(\text{Answers}) = \{\text{Intelligent}, \text{Works_hard}\}$;
- *Grade* è indipendente da *Intelligent* e *Works_hard* data *Answers*:
 $\text{parents}(\text{Grade}) = \{\text{Answers}\}$;

- la fattorizzazione della distribuzione congiunta è

$$P(\text{Intelligent}, \text{Works_hard}, \text{Answers}, \text{Grade}) =$$

$$P(\text{Intelligent}) \cdot P(\text{Works_hard})$$

$$\cdot P(\text{Answers} \mid \text{Intelligent}, \text{Works_hard})$$

$$\cdot P(\text{Grade} \mid \text{Answers});$$
- per le risposte si possono considerare domini semplificati, come ad esempio $\text{Answers} = \{\text{insightful}, \text{clear}, \text{superficial}, \text{vacuous}\}$, oppure anche il testo stesso delle risposte.

Data una BN che specifica una distribuzione congiunta, il problema di *inferenza probabilistica* più comune è il calcolo della *distribuzione a posteriori* di (alcune) **variabili di query**, data l'*evidenza*, ossia la congiunzione di assegnazioni di valori osservate riguardanti altre variabili.

Esempio — Prima delle osservazioni, $P(\text{Intelligent})$ fornita dalla BN, dopo si può usare l'inferenza per determinare $P(\text{Grade})$:

- query $P(\text{Intelligent} \mid \text{Grade} = A)$: distribuzione a posteriori, ossia saputo che il voto conseguito è A (un valore di Grade);
- query $P(\text{Intelligent} \mid \text{Grade} = A \wedge \text{Works_hard} = \text{false})$: distribuzione a posteriori, noto pure che $\text{Works_hard} = \text{false}$;
- Intelligent e Works_hard sono indipendenti quando non siano date altre informazioni ma, se si conosce il voto, esse diventano *dipendenti*; ciò spiega perché ci si vanta se si hanno buoni voti senza studiare molto: aumenta la probabilità di risultare intelligenti.

Per modellare un dominio attraverso una BN occorre:

- Individuare le *variabili rilevanti*: cosa può essere osservato del dominio, ogni feature osservata costituirà una *variabile osservata* utile a poter poi condizionare le query sulla base dei valori assegnati; le feature che costituiranno le *variabili di query* ossia le informazioni di cui interessi determinare la probabilità a posteriori; potrebbero essere individuate anche altre variabili *nascoste* o **latenti**, né osservate né di query, utili a considerare dipendenze indirette e a semplificare il modello, perché vi saranno meno probabilità condizionate da specificare;
- Decidere i *domini di valori*, ossia il livello di dettaglio del ragionamento che porterà a rispondere alle query. Per ogni variabile, si specifica il significato di ogni valore in osservazione del **principio di chiarezza** che richiede l'onniscienza del sistema riguardo il valore di ogni variabile, ossia sappia cosa deve accadere perché un variabile (non latente) assuma un dato valore; va quindi documentato il significato delle variabili e dei loro valori (escludendo quelle latenti i cui valori potranno in seguito essere *appresi* dai dati);
- Specificare le *relazioni* di dipendenza diretta tra le variabili, ossia gli archi da considerare per definire la funzione *parent*;
- Definire le CPD per ognuna variabile in dipendenza *dai genitori*, solitamente in forma di tabelle.

Inferenza Probabilistica

Lo scopo dell'inferenza è quello di rispondere a query su un dato modello probabilistico, essenzialmente attraverso il calcolo di distribuzioni (possibilmente in modo efficiente). Un tipico task è quello di calcolare la **distribuzione a posteriori** di una o più variabili di query, data l'evidenza disponibile. Il problema risulta complesso.

Gli *approcci* principali all'*inferenza probabilistica* con le BN si caratterizzano in termini di inferenza *esatta* ed *approssimata*.

L'inferenza esatta richiede che le probabilità vengano calcolate *precisamente*. Fondamentalmente gli approcci relativi a tale forma si dividono in due modalità principali:

1. *Enumerazione* dei mondi esistenti (coerenti) con l'evidenza;
2. Sfruttamento della struttura della rete: ad esempio l'*algoritmo di eliminazione delle variabili*, basato sulla *programmazione dinamica*, sfrutta le relazioni di indipendenza condizionata.

Per l'inferenza approssimata, invece, esistono diversi metodi per stimare le probabilità in base al tipo di approssimazione:

- Metodi che forniscono **limiti garantiti** $[l, u]$ di variazione entro i quali ricada la probabilità esatta p , ad esempio un algoritmo *anytime* garantisce che l e u tendano ad avvicinarsi reciprocamente col passare del tempo (o con più spazio a disposizione);
- Metodi che forniscono **limiti probabilistici** sull'errore garantendo un errore contenuto (ad esempio 0.1) in un'alta percentuale di casi (ad esempio 95%), ovvero stime di probabilità che convergono nel tempo verso quella esatta e ad una certa velocità di convergenza;
- Metodi di **inferenza variazionale** capaci di fornire generalmente buone *approssimazione*: scelta una *classe* di rappresentazioni più *semplici* (in termini di complessità), ad esempio BN non connesse (senza archi), si trova in tale classe il modello *più vicino* al problema originario ossia una distribuzione a posteriori, *facile* da calcolare, più vicina a quella cercata in modo da trasformare il problema in uno di minimizzazione dell'errore, seguito da quello di inferenza vera e propria.

Modelli Probabilistici Sequenziali

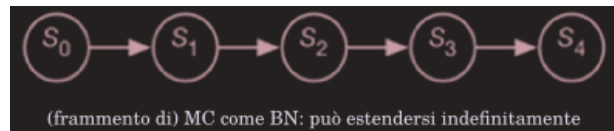
Si considerino BN speciali con una struttura che *si ripete*. Essi rendono possibile il ragionamento su *sequenze* non solo *temporali*, ma anche nello spazio, come le parole in una frase. È possibile estendere le strutture anche a un numero *illimitato* di variabili casuali.

Nel seguito si richiameranno brevemente alcuni modelli fra i più usati.

Una **catena di Markov** (MC) è una BN con nodi/variabili disposti in una *sequenza*; quindi, ogni variabile dipenderà direttamente solo dalla precedente.

Tramite queste strutture è possibile rappresentare sequenze di (valori o) *stati* (trattati da uno spazio finito o almeno enumerabile).

Ad esempio, si possono considerare sequenza di stati in un sistema dinamico o di parole in un testo. Ogni *posizione* nella sequenza sarà indicato anche come **stage**.



Le sequenze sono spesso intese come successioni *nel tempo*. Essendo BN in cui ogni nodo, tranne il primo, dipende solo dal genitore, si dice che viene fatta una **assunzione di Markov** (mancanza di memoria, o *memorylessness*) che può essere descritta dalla seguente proprietà della distribuzione

$$P(S_{i+1} | S_0, \dots, S_i) = P(S_{i+1} | S_i)$$

dove S_i rappresenta lo **stato** al tempo t : intuitivamente, S_t porta con sé tutte le informazioni *storiche* che possono influenzare gli stati futuri.

Una MC è un **modello stazionario** (ovvero *omogeneo nel tempo*) se tutte le variabili condividono un unico dominio e stesse probabilità di transizione per ogni stage:

$$\forall i \geq 0: P(S_{i+1} | S_i) = P(S_1 | S_0)$$

Il che la rende definibile attraverso due sole distribuzioni:

- $P(S_0)$ che specifica le condizioni iniziali;
- $P(S_{i+1} | S_i)$ che specifica le *dinamiche*, le stesse per ogni $i \geq 0$ (la dinamica del mondo non cambia nel tempo).

Quindi con pochi parametri si specifica una struttura potenzialmente *non limitata*. Sarà poi possibile porre domande su punti arbitrari nel futuro o nel passato.

Per determinare le distribuzioni:

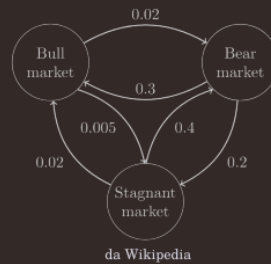
- Per $P(S_i)$: si può usare l'*eliminazione di variabili* sommando su tutte le precedenti (successive irrilevanti);
- Per $P(S_i | S_k)$: se $i > k$, basta considerare solo le variabili tra S_i e S_k , altrimenti solo quelle di indice inferiore a k .

La **distribuzione** di una MC si dice **stazionaria** se ciò che vale una volta varrà anche per la successiva: P stazionaria se per ogni stato s si ha che $P(S_{i+1} = s) = P(S_i = s)$, per cui:

$$P(S_i = s) = \sum_{S_i} P(S_{i+1} = s | S_i) * P(S_i)$$

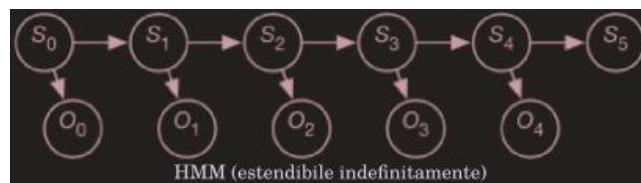
Esempio — Catene generate da un *processo markoviano* relativo al mercato finanziario

- spazio stati: {Bull, Bear, Stagnant};
- probabilità delle transizioni nel diagramma che segue (ovvero anche attraverso matrice):



distribuzione stazionaria: $\pi = (0.885, 0.071, 0.044)$.

Un **modello di Markov nascosto** o *Hidden Markov Model* (HMM) è costituito da una catena di Markov estesa con l'aggiunta di nodi per le *osservazioni* per ciascuno stage, come in figura:



Oltre alle transizioni di stato, per ogni istante di tempo t c'è l'osservazione O_t (variabile) che dipende da S_t (e da t) con *dominio* costituito dall'insieme delle possibili osservazioni; tali osservazioni sono *parziali*, cioè stati diversi possono essere mappati su una stessa osservazione e *rumorose*, ossia in uno stesso stato ma in momenti diversi si possono in maniera casuale diverse osservazioni.

Sono necessarie anche le seguenti assunzioni di base:

- Lo stato al tempo $t + 1$ dipende direttamente solo dallo stato al tempo t per $t \geq 0$, come nelle catene di Markov;
- L'osservazione a tempo t dipende direttamente solo dallo stato al tempo t .

Un HMM *stazionario* è quindi definito attraverso le seguenti distribuzioni:

- $P(S_0)$ è la specifica delle condizioni iniziali;
- $P(S_{t+1} | S_t)$ è la specifica della dinamica;
- $P(O_t | S_t)$ è la specifica del modello del sensore.

Sono possibili diverse inferenze su tali modelli, come ad esempio ricostruire la sequenza di stati data una sequenza di osservazioni (ovvero la sua probabilità).

Simulazione Stocastica

Spesso i problemi di inferenza sono troppo complessi per un'efficiente inferenza esatta: ci sono molte variabili e molte dipendenze in gioco. In tali casi, conviene ricorrere all'*inferenza approssimata* con metodi basati sulla generazione di *campioni* casuali della distribuzione (a posteriori) specificata dalla rete.

Nella **simulazione stocastica** si costruisce un insieme di campioni mappati *su* e *da* probabilità. Per l'*inferenza* si va dalle probabilità ai campioni e viceversa.

Ad esempio, la probabilità $P(a) = 0.25$ indica che su N campioni, per circa un quarto di essi la variabile booleana A risulterà vera.

I problemi tipicamente correlati con la simulazione stocastica sono i seguenti:

1. Come *generare* i campioni;
2. Come *inferire* probabilità dai campioni;
3. Come *incorporare* nel meccanismo le osservazioni a disposizione.

I metodi **Monte Carlo** forniscono le basi per soluzioni che usano il campionamento per calcolare la distribuzione a posteriori di una variabile in una BN.

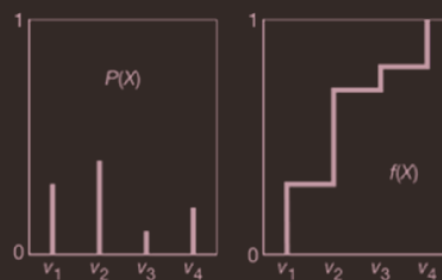
Per la *generazioni di campioni* dalla distribuzione di una sola variabile X occorre ordinare i valori del dominio per definire la *distribuzione cumulativa* f in funzione di x come segue:

$$f(x) = P(X \leq x)$$

Quindi, per ottenere un *campione casuale* v per X , si genera y da una distribuzione uniforme su $[0,1]$ (numeri pseudo-casuali) e si ricava il valore $v \in \text{dom}(X)$ che abbia y come immagine nella cumulativa, ossia tale che $f(v) = y$, ovvero $v = f^{-1}(y)$.

Esempio — Si consideri la variabile X con dominio $\{v_1, v_2, v_3, v_4\}$ ordinato (i.e. $v_1 < v_2 < v_3 < v_4$) e distribuzione $P(X)$ definita come segue:

- $P(X = v_1) = 0.3, P(X = v_2) = 0.4, P(X = v_3) = 0.1, P(X = v_4) = 0.2$



Cumulativa da istogramma

Definita la cumulativa $f(v_i) = P(X \leq v_i)$:

- 30% del codominio di f ha v_1 come contro-immagine, ossia possibilità di essere campionato quando risulta $y \in [0, 0.3]$; analogamente, 40% per v_2 , ecc. ...

Avendo a disposizione un insieme di campioni, per la *stima delle probabilità* di tale insieme basta ricorrere alla *media campionaria*: data una proposizione α sulle variabili, sia s la proporzione di campioni (assegnazioni) per i quali α è vera rispetto al totale n . Per la *legge dei grandi numeri*, s si avvicina asintoticamente alla probabilità esatta $p = P(\alpha)$ al crescere di n .

Per una *stima dell'errore* ϵ si può utilizzare la *disuguaglianza di Hoeffding*:

$$P(|p - s| > \epsilon) \leq 2e^{-2n\epsilon^2}$$

da cui si può ricavare il numero di campioni n che garantisce una stima *probabilmente approssimativamente corretta* della probabilità.

Campionando tutte le variabili di una BN, ogni campione sarà generato in proporzione alla sua probabilità. Ciò consente di *stimare* le *probabilità* a priori di qualsiasi variabile. Dato un ordinamento $X_1, \dots, X_n$ delle variabili, tale che ognuna sia preceduta dai propri genitori, per estrarre un campione/tupla dalla distribuzione congiunta:

- Si parte campionando X_1 usando la cumulativa;
- Per ogni i da 2 a n :
 - Si campiona un valore per X_i da $P(X_i \mid \text{parents}(X_i))$, distribuzione condizionata di X_i (una CPD della BN), dati i valori già campionati per i genitori.

La stima della distribuzione di una variabile di query si ottiene considerando la proporzione di campioni assegnati a ogni valore di tale variabile.

Esempio — Per la BN vista in precedenza, ordinamento come in tabella



Si deve ripetere la costruzione di un campione, con un valore per variabile, nell'ordine:

1. campionando *Tampering* con la cumulativa si ottiene, ad esempio, *Tampering = false*;
2. campionando *Fire* analogamente, si ha ad esempio *Fire = true*;
3. campionando *Alarm* da $P(\text{Alarm} \mid \text{Tampering} = \text{false}, \text{Fire} = \text{true})$ si potrebbe avere, ad esempio *Alarm = true*;
4. poi si campiona *Smoke* usando $P(\text{Smoke} \mid \text{Fire} = \text{true})$;
5. ecc...

Ciò va ripetuto fino a ottenere il numero desiderato di campioni/tuple $\{s_1, \dots, s_n\}$.

campione	<i>Tampering</i>	<i>Fire</i>	<i>Alarm</i>	<i>Smoke</i>	<i>Leaving</i>	<i>Report</i>
s_1	false	true	true	true	false	false
s_2	false	false	false	false	false	false
s_3	false	true	true	true	true	true
s_4	false	false	false	false	false	true
s_5	false	false	false	false	false	false
s_6	false	false	false	false	false	false
s_7	true	false	false	true	true	true
s_8	true	false	false	false	false	true
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
s_{1000}	true	false	true	true	false	false

Ad esempio, per la stima di $P(\text{Report} = \text{true})$ andrà considerata la proporzione dei campioni per i quali *Report* è vera sui 1000 campioni.

Si considera ora un metodo che prescinde dall'ordine di campionamento.

La distribuzione *stazionaria* di una catena di Markov è la distribuzione delle sue variabili, che non cambia in base alla funzione di transizione della catena: se la catena mescola sufficientemente, c'è un'unica distribuzione stazionaria. Tale distribuzione può essere approssimata facendo *girare* il meccanismo di campionamento sufficientemente a lungo.

Nei metodi detti **Markov Chain Monte Carlo** (MCMC) per generare campioni da una distribuzione si costruisce una *catena di Markov* di campioni avente, a lungo andare, la distribuzione desiderata come sua (sola) distribuzione stazionaria; quindi, si campiona da tale catena perché i suoi campioni saranno distribuiti secondo la distribuzione-obiettivo.

Può occorrere una fase iniziale di *riscaldamento/burn-in*: andrebbero scartati i primi campioni, probabilmente lontani dalla distribuzione stazionaria, che sarà ottenuta più avanti.

In particolare, nel **Gibbs sampling** si crea una catena di campioni da una BN, fissando i valori per le variabili osservate (condizionamento) e campionando le altre. Si genera un nuovo campione a partire dal precedente, modificando i valori per le variabili non fissate.

Si può anche considerare una sola variabile alla volta: la singola variabile sarà campionata in base alla distribuzione condizionata ai valori correnti delle altre variabili. Si noti che una variabile X in una BN dipende solo dai valori delle variabili nel suo **Markov blanket** $mb(X)$ contenute i suoi genitori, i suoi figli e gli altri genitori dei suoi figli, come si vede in figura:



La distribuzione dei campioni si avvicina alla distribuzione congiunta reale se non ci sono probabilità *nulle*. La *velocità* dipende da quella del rimescolamento delle probabilità (ovvero da quanta parte dello spazio delle probabilità viene esplorato) che a sua volta dipende da quanto esse risultino *estreme*. In genere il metodo funziona bene per probabilità non estreme.

10 – Apprendimento e Incertezza

Apprendimento Probabilistico

Le forme di apprendimento viste in precedenza possono essere estese con l'uso esplicito del ragionamento probabilistico: i dati forniscono l'evidenza fattuale sulla quale si può effettuare il condizionamento. Ciò è alla base dell'apprendimento automatico supervisionato e non.

Dato il *dataset* Es , per determinato un modello ottimale, m , per i dati a disposizione, Es , si può utilizzare il *teorema di Bayes*:

$$P(m | Es) = \frac{P(Es | m) * P(m)}{P(Es)}$$

dove:

- $P(Es | m)$ rappresenta la **verosimiglianza** o *likelihood* del fatto che Es sia stato generato da m : è alta se m fornisce una buona approssimazione della distribuzione dei dati reale (sconosciuta);
- $P(m)$ è la **probabilità a priori** che codifica il *learning bias*, indicando i modelli più verosimili a priori e può essere usata per la *regolarizzazione* inducendo a scegliere modelli *più semplici* attribuendo loro una maggiore probabilità;
- $P(Es)$, detta **funzione di partizione**, viene usata ai fini della *normalizzazione*.

Nella selezione di un singolo modello, il modello **MAP** (*maximum a posteriori*) è quello che massimizza la probabilità a posteriori, determinabile in base al numeratore

$$P(Es | m) * P(m)$$

essendo trascurabile il denominatore in quanto non dipende da m . Si basa, quindi, su likelihood (adattamento ai dati) e distribuzione a priori dei modelli, utile come *learning bias* per codificare una presenza verso modelli più semplici (rasoio di Occam).

In alternativa, si può puntare a determinare il modello di **massima verosimiglianza** (ML) massimizzando solo

$$P(Es | m)$$

Quando lo spazio dei *modelli* è molto ricco, sorge il *problema* di avere spesso più di un modello m per cui $P(Es | m) = 1$; ad esempio quando si adottano modelli come gli alberi di decisione.

Pertanto, anche un modello molto poco probabile a priori non andrebbe escluso: potrebbe essere quello corretto e, se i dati sono sufficienti, risultare anche il migliore.

Si noti che, per la regola di Bayes, se $P(m)$ è *uniforme*, il modello ML coincide con il modello MAP. Altrimenti conviene preferire m più semplici, adottando distribuzioni a priori che li rendano più probabili, quanto meno soggetti a *sovradattamento*.

Apprendimento Bayesiano

L'**apprendimento Bayesiano** offre un'*alternativa* alla scelta di un *singolo* modello più probabile (MAP o ML): si sfrutta la distribuzione a posteriori di diversi modelli.

Dati gli esempi di training Es , per predire Y per un nuovo esempio $\mathbf{x}$, si deve determinare la probabilità a posteriori:

$$P(Y | \mathbf{x} \wedge Es)$$

Lo scopo di una modello è quello di *generare* esempi (più complesso di *discriminare*). Dato un insieme esaustivo M di modelli disgiunti, si può scrivere (ragionando *per casi* e sfruttando la definizione della *probabilità condizionata*):

$$\begin{aligned} P(Y | \mathbf{x} \wedge Es) &= \sum_{m \in M} P(Y \wedge m | \mathbf{x} \wedge Es) \\ &= \sum_{m \in M} P(Y | m \wedge \mathbf{x} \wedge Es) * P(m | \mathbf{x} \wedge Es) \\ &= \sum_{m \in M} P(Y | m \wedge \mathbf{x}) * P(m | Es) \end{aligned}$$

Qui l'ultimo passaggio dipende da due assunzioni sul modello m :

1. m racchiude tutte le informazioni su Es necessarie a una predizione, ovvero $P(Y | m \wedge \mathbf{x} \wedge Es) = P(Y | m \wedge \mathbf{x})$;
2. m non cambia per un solo nuovo esempio, ossia $P(m | \mathbf{x} \wedge Es) = P(m | Es)$.

Invece di scegliere il modello migliore, nel **model averaging** si mediano le predizioni di tutti i modelli, ponderando con $P(m | Es)$:

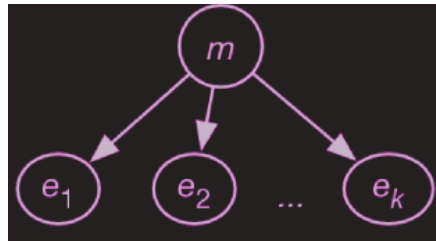
$$P(Y | \mathbf{x} \wedge Es) = \sum_{m \in M} P(Y | m \wedge \mathbf{x}) * \underline{P(m | Es)}$$

dove $P(m | Es)$, calcolabile con la regola di Bayes, dipende dalla capacità di generare i dati osservati e dalla probabilità a priori m , mentre $P(Es)$ serve solo alla normalizzazione e risulta difficile da calcolare se M contiene molti modelli, inutile se si deve solo massimizzare il numeratore.

Un'assunzione comune è che gli esempi $Es = \{e_1, \dots, e_k\}$ siano **indipendenti e identicamente distribuiti** (i.i.d.) dato m ossia che, per ogni coppia di esempi e_i, e_j , $i \neq j$, essi siano indipendenti dato m . In tal caso si può scrivere:

$$P(Es | m) = \prod_{i=1}^k P(e_i | m)$$

il che può essere rappresentato da una semplice rete bayesiana:



Nel ragionamento con questa rete si possono applicare metodi di inferenza esatta o approssimata, condizionando su ogni e_i osservato.

Tipicamente si fanno interrogazioni su un e_i non osservato, ossia predizioni su esempi non visti oppure anche su m (distribuzione sui modelli).

Un possibile *obiettivo* può essere quello di imparare le probabilità, ossia i parametri di un modello probabilistico in una delle diverse formulazioni possibili.

Nell'approccio bayesiano si trattano i parametri (ϕ) non come costanti ma come variabili aleatorie a valori reali.

Si consideri il caso più semplice: variabile Y booleana e nessun input a disposizione. Per tale Y , si può considerare ϕ come variabile su $[0,1]$ tale che $P(Y = true | \phi) = \phi$ e $P(Y = false | \phi) = 1 - \phi$. Quindi ϕ determina l'insieme M dei diversi modelli. Gli esempi in Es saranno del tipo $Y = true$ o $Y = false$.

Per $P(\phi)$ non c'è alcuna informazione pregressa (BK) su Y , oltre agli esempi in Es . In tal caso i valori sono equiprobabili ossia ϕ ha una *distribuzione uniforme* su $[0,1]$ (nella figura che segue corrisponde alla densità per $n_0 = 0, n_1 = 0$). $P(\phi | Es)$ può essere considerata come l'aggiornamento di $P(\phi)$ dopo aver osservato gli esempi in Es :

$$P(\phi | Es) = \frac{P(Es | \phi) * P(\phi)}{P(Es)} \propto \phi^{n_1} * (1 - \phi)^{n_0} * P(\phi)$$

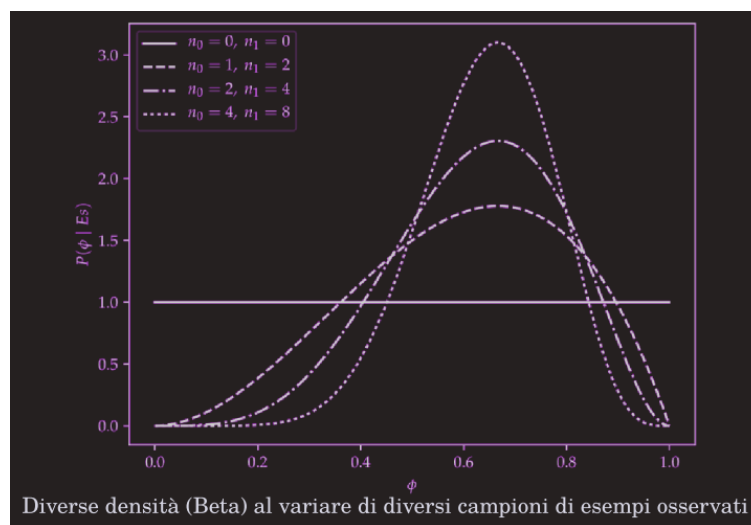
Dove n_0 è il numero di esempi con $Y = false$ e n_1 quello di esempi con $Y = true$.

Sia ϕ il parametro della distribuzione di una variabile Y binaria (ϕ probabilità di $Y = true$). Si definisce la *distribuzione Beta* nel modo seguente:

$$Beta_{\alpha_0, \alpha_1}(\phi) = \frac{1}{Z} * \phi^{\alpha_1-1} * (1 - \phi)^{\alpha_0-1}$$

dove

- I parametri α_0 e α_1 sono conteggi iniziali, convenzionalmente si considerano $\alpha_i = n_i + 1$ con $i \in \{0,1\}$ e Z è una costante di normalizzazione: si noti che la distribuzione uniforme corrisponde a una $Beta_{1,1}$;
- La sua *moda* (valore più probabile) risulta $\phi = \frac{\alpha_1-1}{\alpha_0+\alpha_1-2}$;
- Il suo *valore atteso* è $\phi = \frac{\alpha_1}{\alpha_0+\alpha_1}$, il che giustifica lo *smoothing di Laplace*.



La distribuzione di $P(\phi)$ non è necessariamente uniforme. Tipicamente si ha che: $P(\phi) = \phi^{c_1-1} * (1 - \phi)^{c_0-1}$, come nel caso in cui si considerino c_1 *pseudo-esempi* con $Y = true$, e c_0 con $Y = false$.

Quindi, la distribuzione a posteriori è $P(\phi | Es) \propto \phi^{c_1+n_1-1} * (1 - \phi)^{c_0+n_0-1}$ ossia mantiene la stessa forma di quella a priori (si dice che le distribuzioni sono *coniugate*).

La stima MAP per ϕ è $\hat{\phi}_{MAP} = \frac{c_1+n_1-1}{c_0+n_0+c_1+n_1-2}$ e il valore atteso è

$$E[\phi] = \frac{c_1+n_1}{c_0+n_0+c_1+n_1}.$$

In caso si disponesse solo delle statistiche n_0 e n_1 , che possono corrispondere a sequenze diverse di esempi, si ricorre alla binomiale.

Nel caso di Y categorica, parametro con k valori, si considera la *distribuzione Dirichlet*, un'estensione della Beta a più dimensioni:

$$Dirichlet_{\alpha_1, \dots, \alpha_k}(p_1, \dots, p_k) = \frac{1}{Z} * \prod_{j=1}^k p_j^{\alpha_j - 1}$$

dove gli $\alpha_j \geq 0$ sono i conteggi iniziali dei diversi valori $j = 1, \dots, k$, e si considera in genere $\alpha_j = n_j + 1$ come un conteggio, p_j è la probabilità del j -esimo valore ($Y = y_j$) e Z è la costante di normalizzazione. Il *valore atteso* per il risultato i -esimo (mediando su tutti i p_j) sarà $\alpha_i / \sum_j \alpha_j$, con gli α_j tutti non-negativi e non tutti nulli.

Per predire un valore per Y , partendo da *pseudo-conteggi* $c_j > 0$ per ogni y_j (invece che una distribuzione a priori uniforme) ottenuti prima di osservare dati (ossia $\forall j : n_j = 0$) considerata una Dirichlet anche quella a posteriori sarà Dirichlet e avendo osservato poi n_j esempi con $Y = y_j$, la probabilità di Y può essere stimata usando il *valore atteso*:

$$P(Y = y_j) = \frac{c_j + n_j}{\sum_h c_h + n_h}$$

Notare che con pseudo-conteggi tutti unitari si ottiene il cosiddetto *smoothing di Laplace*.

Gli pseudo-conteggi si possono ottenere *integrando* l'opinione di esperti con i dati a disposizione. Questa integrazione comporta le seguenti problematiche:

- La difficoltà nel dare *valori esatti* di probabilità non modificabili nel seguito;
- La rappresentazione dell'incertezza d'una *stima* della probabilità;
- La *combinazione* di stime fatte da più esperti;
- L'*integrazione* tra *opinione* degli esperti e *dati* effettivi.

Una possibile *soluzione* è quella in cui le probabilità (numeri reali) provengano da conteggi (interi): invece di $P(A)$, l'esperto si limiterà a fornire $\langle n, m \rangle$ ossia il numero n occorrenze di A sulle m prove effettuate, una stima (non sempre credibile) delle *dimensioni* dei dati su cui fonda il parere.

Conteggi da diversi esperti *combinati* opportunamente per ricavare gli pseudo-conteggi del sistema: mentre la proporzione riflette la *probabilità*, i diversi gradi di *confidenza* si riflettono nei valori assoluti.

Ad esempio, ci sono diversi modi per rappresentare la probabilità di $3/7$ che corrispondono a diversi livelli di confidenza:

- Con $< 3, 7 >$ si ha una bassa confidenza: la stima sarà facilmente soppiantata da dati/stime di altri esperti;
- Con $< 30, 70 >$ si ha maggiore confidenza: pochi esempi non la modificherebbero di molto, non così se ce ne fossero decine;
- Con $< 3000, 7000 >$ si ha un'alta confidenza: anche centinaia di esempi non modificherebbero molto i conteggi a priori ricevuti dagli esperti, solo molte migliaia di dati ne ridurrebbero l'influenza.

Un **classificatore Bayesiano** è un modello probabilistico di classificazione per il quale la distribuzione *a posteriori* viene calcolata mediante la regola di Bayes.

L'idea di fondo è che la *classe* influenzi/predica i valori delle feature di input per le istanze che vi appartengono: gli esempi sono raggruppati in classi perché hanno valori comuni delle feature quindi imparando le *dipendenze* delle feature dalla classe, si userà il modello risultante per predire la classificazione di nuove istanze.

Nel classificatore **Naive Bayes** (NB) si fa l'assunzione di *indipendenza* condizionata reciproca tra le feature X_i nota la classe Y . Questo caso può essere descritto da una semplice rete bayesiana con una nodo genitore relativo alla classe e nodi figli per ciascuna delle feature di input X_i . tale rete richiede la specifica delle distribuzioni $P(Y)$ per Y e $P(X_i | Y)$ per ogni X_i . Per predire la classe per un nuovo esempio si dovrà calcolare $P(Y | Xs)$, condizionando sui suoi valori osservati dalle diverse X_i .

Con più variabili-obiettivo Y_j si possono apprendere modelli separati.



Dato un esempio con *input* $X_1 = v_1, \dots, X_k = v_k$, per il calcolo della distribuzione a posteriori:

$$\begin{aligned}
 P(Y | X_1 = v_1, \dots, X_k = v_k) &= \frac{P(X_1 = v_1, \dots, X_k = v_k | Y) * P(Y)}{P(X_1 = v_1, \dots, X_k = v_k)} \\
 &= \frac{P(X_1 = v_1 | Y) * \dots * P(X_k = v_k | Y) * P(Y)}{P(X_1 = v_1, \dots, X_k = v_k)} \\
 &= \frac{P(Y) * \prod_{i=1}^k P(X_i = v_i | Y)}{\sum_Y P(Y \wedge (X_1 = v_1, \dots, X_k = v_k))} \\
 &= \frac{P(Y) * \prod_{i=1}^k P(X_i = v_i | Y)}{\sum_Y P(Y) * \prod_{i=1}^k P(X_i = v_i | Y)}
 \end{aligned}$$

Si noti come il denominatore funga da costante di normalizzazione.

Il modello NB è capace di trattare anche esempi con **dati mancanti** ossia con valori non osservati per alcune X_i : si condiziona sulle feature osservate, facendo un'assunzione detta MAR, *Missing at Random*. Il NB risulta *ottimale* con X_i unica feature osservata (senza assunzioni di indipendenza): al crescere del numero di X_i osservate, l'accuratezza delle predizioni dipende dalla reciproca indipendenza delle X_i data Y .

Nel caso in cui Y booleana e siano X_i tutte osservate, si ha un modello simile a quello di *regressione logistica* (REGLOG). Essi fanno le stesse predizioni quando per X_i il peso è $w_i = \log[P(X_i | h)/P(X_i | \neg h)]$. La differenza sta nel fatto che REGLOG usa la DG per minimizzare una log-loss; quindi, ha il *punto debole* di non essere applicabile in caso di valori mancanti, mentre NB apprende i parametri dalla stima MAP, valida anche in caso di *valori mancanti*, mentre NB apprende i parametri dalla stima MAP, valida anche in caso di *valori mancanti*, ma con il *punto debole* dell'assunzione di indipendenza condizionata tra X_i data Y , spesso non giustificata.

Per apprendere un classificatore, ossia la distribuzione a posteriori $P(Y | \mathbf{x})$, serve determinare dai dati le distribuzioni $P(Y)$ e $P(X_i | Y)$ per ogni i .

La stima di $P(X_i | Y)$ costituisce un *sotto-problema* di apprendimento, uno per ogni valore di Y .

Una soluzione semplice è quella data dalla *stima ML*, in cui si calcola $P(X_i = x_i | Y = y)$ come *proporzione empirica* osservata nei dati di training, il rapporto tra numero dei casi in cui $X_i = x_i \wedge Y = y$ e quello dei casi in cui $Y = y$.

Esempio — Dal dataset di un esempio precedente, dato il modello NB in figura, si devono determinare le distribuzioni richieste come frequenze empiriche:

- $P(\text{User_action} = \text{reads}) = 9/18 = 0.5$
- $P(\text{Author} = \text{known} | \text{User_action} = \text{reads}) = 2/3$
- $P(\text{Author} = \text{known} | \text{User_action} = \text{skips}) = 2/3$
- $P(\text{Thread} = \text{new} | \text{User_action} = \text{reads}) = 7/9$
- $P(\text{Thread} = \text{new} | \text{User_action} = \text{skips}) = 1/3$
- $P(\text{Length} = \text{long} | \text{User_action} = \text{reads}) = 0$
- $P(\text{Length} = \text{long} | \text{User_action} = \text{skips}) = 7/9$
- $P(\text{Where_read} = \text{home} | \text{User_action} = \text{reads}) = 4/9$
- $P(\text{Where_read} = \text{home} | \text{User_action} = \text{skips}) = 4/9$
 - *Author* e *Where_read* non hanno potere predittivo: conoscere i loro valori non cambia la probabilità di *reads*. Quindi saranno ignorate nel seguito.

Casi di *probabilità nulle* possono avere effetti indesiderati. Alcune feature diventano *molto predittive*: con un solo valore nullo si potrebbe escludere un'intera categoria, ma un insieme finito di dati potrebbe essere insufficiente a fornire l'evidenza necessaria per supportare una tale conclusione.

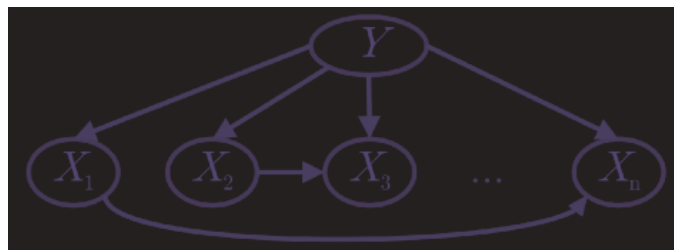
Alcune *combinazioni* di osservazioni possono risultare *impossibili*: nel classificatore si avrebbe una divisione per zero. Il *problema* nasce dall'uso delle frequenze empiriche come stime di probabilità. Vi si può ovviare incorporando pseudo-conteggi da scegliere accuratamente.

In conclusione, un classificatore NB risulta spesso efficace, pur essendo semplice e facile da implementare quindi è un buon metodo per prototipizzare rapidamente. Funziona bene quando l'assunzione di indipendenza è appropriata nel caso in esame. Ad esempio, nel caso di **tipi naturali** (spesso associati a *sostantivi*), classi che si sono evolute per *ché* utili a distinguere oggetti.

Il modello NB può essere ampliato ammettendo estensioni in alcune X_i possano essere anche in nodi-genitori, mentre le altre rimangono figlie di Y : la probabilità della classificazione dati i genitori può essere rappresentata attraverso alberi di decisione, funzioni lineari compresse oppure reti neurali.

In altre estensioni sono ammesse figlie di Y non necessariamente indipendenti:

- **Tree Augmented Naive-Bayes (TAN) network**: oltre alla classe possono avere un solo altro genitore (purché il grafo resti aciclico), un modello semplice ed efficiente delle dipendenze tra X_i ;



- **Latent Tree**: prevede una struttura più complessa nella Y , decomponibile in un numero di *variabili latenti* connesse ad albero, tale che ogni variabile osservata figlia di una variabile latente; si avrà così un modello della dipendenza tra variabili osservate.

Nell'apprendimento degli alberi di decisione serve un bias che tipicamente andrà a favorire alberi più piccoli e può essere imposto attraverso la scelta della distribuzione a priori (sui diversi alberi).

Più alberi potranno adattarsi perfettamente ai dati, a meno di *rumore*. Ad esempio, considerando esempi di training con gli *stessi* valori delle X_i ma valori *diversi* per la Y oppure esempi con *valori mancanti* per qualche X_i , per ogni *assegnazione non osservata*, si possono adattare perfettamente al training set alberi che si adattano perfettamente al training set potrà risultare ottimale.

Questa può essere vista come conseguenza del *No Free Lunch Theorem* per il quale modelli apparentemente efficaci possono sempre rivelarsi peggiori su problemi/dataset diversi. Conviene quindi tenere in considerazione anche alberi che si adattano meno perfettamente ai dati, sfruttando anche la *distribuzione a priori* dei modelli (stima MAP).

In base al criterio MAP, si deve massimizzare $P(Es | m) * P(m)$. Passando ai logaritmi e negando, la funzione da ottimizzare (minimizzare) risulta:

$$(-\log_2 P(Es | m)) + (-\log_2 P(m))$$

Questo obiettivo trova corrispondenza con la *Teoria dell'informazione*, in particolare con il criterio della **minima lunghezza di descrizione** (*minimum description length*, MDL): vanno preferiti i modelli che minimizzano la somma precedente dove, $-\log_2 P(Es | m)$ rappresenta il numero di bit per descrivere i dati Es , in base a m , e $-\log_2 P(m)$ è il numero di bit per descrivere m .

Come si vede, il *bit* è un'unità di misura che accomuna probabilità e complessità del modello.

Si può interpretare il modello ottimale per il *criterio MDL* come il più efficiente per la codifica e trasmissione dei dati: prima va comunicato il modello m e poi anche i dati, in termini di m . Essendo la funzione log monotona, il modello MAP *coincide* con quello MDL.

La definizione esatta di una codifica ottimale risulta in genere *difficile*: spesso si ricorre a una *misura approssimata* della complessità del modello.

Si possono considerare:

- $|t|$, il numero di *parametri probabilistici*:
 - Per un albero (con probabilità alle foglie): $|t|$ = numero delle foglie;
 - Per una funzione lineare o una rete neurale: $|t|$ = numero di parametri numerici.
- $|Es|$, numero di esempi (a partire da 0):
 - Ci sono al più $|Es| + 1$ diverse probabilità da distinguere nel modello, quindi, servono $\log_2(|Es| + 1) \approx \log_2(|Es|)$ bit.

Come approssimazione del MDL da minimizzare si può usare:

$$-\log_2 P(Es | m) + |m| * \log_2(|Es|)$$

detto indice **BIC** (*Bayesian information criterion*).

Apprendimento non Supervisionato

Nell'**apprendimento non supervisionato** il training set non contiene la feature-target. Si cercano o costruiscono pattern dei dati.

Ci sono diversi compiti di tipo non supervisionato. Un *obiettivo* può essere quello di (ri)costruire una classificazione naturale dei dati. Ciò significa che la feature-obiettivo può essere considerata come variabile *latente* (i suoi valori non sono osservati).

Il metodo generale per ricostruire i valori della classificazione naturale è il **clustering**. Si partizionano gli esempi in **cluster**, le *classi* naturali, in modo tale che ogni cluster predica i valori delle feature per gli esempi in esso contenuti.

Per ogni partizionamento dei dati (detto anche *clustering* o sistema di cluster) si può calcolare un *errore di predizione* associato; quindi, si deve trovare il clustering migliore minimizzando tale errore.

Esempi

- Nella *diagnosi medica* un raggruppamento dei trattamenti dovrebbe consentire di predire gli effetti desiderabili e indesiderabili di ciascun trattamento; si potrebbe poi arrivare a non dare un farmaco a un paziente poiché farmaci simili potrebbero avere effetti disastrosi su pazienti simili.
- In un sistema di *tutoring intelligente* si raggruppano dati sui comportamenti degli studenti in fase di apprendimento. Strategie d'insegnamento che funzionano per uno dei membri di un cluster potrebbero essere indicate per gli altri membri.

Si distinguono l'**hard clustering** in cui ogni esempio viene assegnato a un cluster preciso (per cui il cluster può essere usato per *predire* i valori delle feature dell'esempio) e il **soft clustering** che calcola una distribuzione di appartenenza degli esempi ai cluster: la predizione dei valori delle feature di un esempio si può ottenere come *media ponderata* delle predizioni per cluster, pesata attraverso la distribuzione trovata.

K-MEANS è un algoritmo per l'hard clustering. Prende in *input* gli esempi di training e il numero di cluster k da cercare. Si assumerà che il dominio di ogni feature sia *cardinale/numerico*, ossia con differenze tra valori significative. Esso produrrà come *output* i k cluster, ovvero l'assegnazione degli esempi ai cluster e, per ogni cluster, si avrà la predizione del valore (medio) per ogni feature (vettore medio, prototipi del cluster).

Dato il training set Es con $X_1, \dots, X_n$ feature di input, si indicherà con $X_j(e)$ valore osservato della feature X_j per l'esempio $e \in Es$ (si assume che tali valori siano *osservati* direttamente). A ogni cluster è associato un indice intero $c \in \{1, \dots, k\}$.

Essenzialmente K-MEANS costruisce una funzione $cluster : Es \rightarrow \{1, \dots, k\}$ tale che $cluster(e) = c$ indica che e appartiene al cluster c . Per fare questo si basa su una funzione $pred(j, c)$ che restituisce il valore di X_j per il cluster c : $pred(j, c)$ sarà il valore viene predetto per ogni e del cluster: $pred(j, cluster(e))$ è il valore di X_j .

L'obiettivo dell'algoritmo è quindi quello di trovare $cluster$ e $pred$ ottimali, ossia che minimizzino la somma di **loss quadratiche**:

$$\sum_{e \in Es} \sum_{j=1}^n [pred(j, cluster(e)) - X_j(e)]^2$$

La *predizione* per un cluster verrà calcolata come *media* delle predizioni dei suoi esempi.

Una soluzione ottimale ottenuta da una ricerca tra le possibili assegnazioni degli esempi ai cluster che minimizzino l'errore è NP-hard, proponibile solo in caso di pochi esempi. Con molti esempi, invece, ci saranno moltissime k -partizioni possibili e una ricerca esaustiva non risulta praticabile.

L'algoritmo K-MEANS abbassa *iterativamente* la loss quadratica e può essere schematizzato come segue:

- Inizializzare (casualmente) le assegnazioni degli esempi ai cluster;
- Ripetere fino a un'assegnazione stabile:
 - Per ogni feature X_j per ogni cluster c , definire $pred(j, c)$ con il valore medio di $X_j(e)$ per ogni e in c :

$$pred(j, c) \leftarrow \frac{\sum_{e: cluster(e)=c} X_j(e)}{|\{e : cluster(e) = c\}|}$$

dove il denominatore conta il numero di esempi nel cluster c ;

- Riassegnare ogni esempio e a un cluster c : scegliere c che minimizzi

$$\sum_{j=1}^n [pred(j, c) - X_j(e)]^2$$

(quadrato della distanza euclidea)

Esso *converge* quando non si hanno (sensibili) cambiamenti nelle assegnazioni.

Ottenute diverse assegnazioni stabili da più esecuzioni, si può scegliere il partizionamento miglior confrontandoli in termini dell'errore, ossia della distanza dai centroidi dei cluster assegnati ai diversi esempi. Si può pensare all'applicazione del RANDOM RESTART all'algoritmo con diverse più assegnazioni iniziali, per poi scegliere l'assegnazione stabile con minimo errore.

L'etichettatura precisa è poco importante: qualsiasi permutazione andrà bene.

È importante notare la sensibilità dell'errore (ossia della distanza euclidea) alle diverse *scale* delle feature (ad esempio, feature come altezza, età e genere).

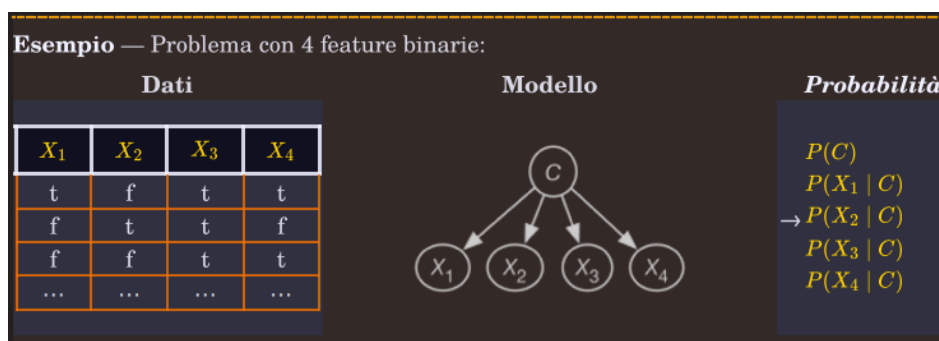
Pertanto, conviene lavorare su valori preventivamente *riscalati* (su scala comune, ad esempio standardizzati) per poter essere poi confrontati.

È importante anche la scelta del(l'iper-)parametro k che dovrebbe rappresentare un compromesso fra complessità e adattamento ai dati. È possibile usare il *BIC score*, la *ricerca del gomito/elbow* (o *ginocchio/knee*) nella *curva* dell'errore al variare dei diversi valori di k : curva decrescente al crescere di k . Il valore *naturale* per k sarebbe quello con maggiore *riduzione* dell'errore rispetto a $k - 1$. Si noti che il clustering ottimale per k può spesso risultare molto diverso da quello ottimale per $k + 1$.

I classificatori bayesiani possono costituire una base per l'*apprendimento non supervisionato* se si considera la classe come variabile *latente*, ossia non osservata (senza valori nel dataset).

L'algoritmo **EXPECTATION MAXIMIZATION** (EM) consente di apprendere modelli probabilistici con variabili latenti; quindi, può essere utilizzato per problemi di *soft clustering* se combinato con un classificatore probabilistico come NB. Il suo schema è analogo a quello di K-MEANS, con gli stessi argomenti in input, l'insieme di esempi Es e il numero di cluster k ma gli esempi saranno assegnati ai cluster in senso probabilistico.

Dato Es , EM costruisce un modello NB con un nodo genitore con la variabile *latente* C che indicherà la classe/cluster, quindi con *dominio* $\{1, \dots, k\}$ e come nodi figli uno per ogni feature X_i osservata nei dati in Es . Si devono apprendere le distribuzioni $P(C)$, $P(X_i | C)$ che si adattino meglio a Es .



Rispetto a K-MEANS si può considerare l'estensione (*augmentation*) della tabella-dati con due colonne (virtuali): una colonna C per la classe/cluster, ogni esempio originario replicato per k righe della tabella, una per classe/cluster; una colonna

Count per il *contatore* normalizzato, con le k assegnazioni dell'esempio alle classi di C , che insieme rappresentano la distribuzione $P(C | e)$.

Iterativamente EM ricalcola i conteggi dal modello e poi il modello dai conteggi.

Esempio — Problema con $k = 3$ classi:

X_1	X_2	X_3	X_4	C	$Count$
⋮	⋮	⋮	⋮	⋮	⋮
t	f	t	t	1	.4
t	f	t	t	2	.1
t	f	t	t	3	.5
⋮	⋮	⋮	⋮	⋮	⋮

← E

M →

$P(C)$
 $P(X_1 | C)$
 $P(X_2 | C)$
 $P(X_3 | C)$
 $P(X_4 | C)$

Passi (simili a quelli di K-MEANS):

- **EXPECTATION:** aggiorna *Count* in base alle distribuzioni correnti del modello:
 - Per ogni esempio originario $\langle X_1 = v_1, \dots, X_n = v_n \rangle$, il contatore associato a $\langle X_1 = v_1, \dots, X_n = v_n, C = c \rangle \quad \forall c = 1, \dots, k$ viene aggiornato con $P(C = c | X_1 = v_1, \dots, X_n = v_n)$;
 - Il *valore atteso* va calcolato tramite inferenza probabilistica;
- **MAXIMIZATION:** inferisce le probabilità del modello dagli esempi *estesi*:
 - In tabella: esempi completi con valori per *tutte* le variabili, come nell'apprendimento supervisionato di probabilità (con il NB);
 - Per la stima delle probabilità del modello si considera la *massimizzazione ML* (o MAP).

EM parte con distribuzioni o conteggi casuali e punta a convergere a un *massimo* locale della *likelihood* dei dati fornendo come output un *modello probabilistico*, utile anche a classificare nuovi esempi:

$$\begin{aligned}
 P(C = c | X_1 = v_1, \dots, X_n = v_n) &= \\
 &= \frac{P(C = c) * \prod_{i=1}^n P(X_i = v_i | C = c)}{\sum_{c'} P(C = c') * \prod_{i=1}^n P(X_i = v_i | C = c')}
 \end{aligned}$$

Per il calcolo delle probabilità tramite *statistiche sufficienti*:

- A ogni iterata, si calcolano dai dati:
 - *cc*: *contatori-classe*, vettore k -dimensionale, con $cc[c]$, somma dei contatori degli esempi per la classe c nella tabella dei dati estesa;
 - *fc*: *contatori-feature*, matrice *tridimensionale* (feature categoriche) in cui per i tra 1 e n , per ogni $v \in \text{dom}(X_i)$ e ogni classe/cluster c : $fc[i, v, c]$ è la somma dei contatori per ogni esempio t per c ;

- Le statistiche sono usate per derivare quelle dell'iterata successiva; si noti che cc può essere anche calcolato da fc , ma è più comodo tenere un contatore separato;
- La stima delle *probabilità* può essere fatta in base a cc e fc :
 $P(C = c) = cc[c]/|Es|$, con $|Es|$ numero di esempi nel dataset originario, o somma dei conteggi in quello esteso.
Inoltre, $P(X_i = v \mid C = c) = fc[i, v, c]/cc[c]$.

Un *problema* tipico (con $k > 1$) è che EM ha pressoché sempre massimi locali e globali *multipli*: con qualsiasi permutazione delle etichette della classi stesse probabilità. Per trovare il massimo globale si possono effettuare più *restart* e infine scegliere il modello con log-likelihood massima.

Apprendimento di Belief Network

Una BN definisce una distribuzione su un insieme di variabili. Dato che è difficile per un esperto fornire modelli accurati da codificare come rete bayesiana, si può cercare di *imparare* la rete da dati disponibili.

La complessità di tale problema dipende dalla disponibilità di conoscenza a priori sulla struttura e sulle variabili coinvolte e dalla completezza del dataset disponibile.

L'*obiettivo* dell'apprendimento della rete bayesiana varia dal caso *più semplice* in cui la struttura è nota e tutte le variabili note e *osservabili*, disponendo di un dataset completo, fino al caso *più difficile* in cui la struttura non è data, le variabili non siano tutte note e il dataset sia incompleto, ovvero ci siano dati mancanti (senza possibilità di fare l'assunzione che siano *missing at random*).

L'*obiettivo* si riduce a imparare, per ogni X_i , la distribuzione condizionata $P(X_i \mid \text{parents}(X_i))$. Questo si può vedere come un problema di apprendimento *supervisionato* con feature-target X_i e feature di input $\text{parents}(X_i)$.

Essendo, solitamente, i genitori in numero limitato ogni probabilità potrà essere appresa *separatamente* sfruttando gli esempi di training e altra conoscenza a priori comunicata dagli esperti, come ad esempio pseudo-conteggi che migliorino la stima delle probabilità.

Un caso un po' più difficile è quello in cui il modello contenga anche variabili *latenti* ossia variabili i cui valori non siano non osservabili per nessun esempio. Nella tabella dei dati non ci saranno colonne per tali variabili. Senza queste variabili nel modello aumentano i parametri per cui si hanno maggiori rischi di *sovradattamento*.

Ad esempio, escludendo la variabile E dal modello precedente occorrerebbe determinare le distribuzioni $P(A)$, $P(B)$, $P(C | A, B)$, $P(D | A, B, C)$, passando quindi da 10 a 14 parametri.

Esempio — BN con E variabile nascosta:

Modello



Dati

A	B	C	D
t	f	t	t
f	t	t	t
t	t	f	t
...	...	...	...

Probabilità

$P(A)$
 $P(B)$
 $\rightarrow P(E | A, B)$
 $P(C | E)$
 $P(D | E)$

L'obiettivo sarà quello di apprendere i parametri del modello, ossia 10 valori di probabilità nelle diverse distribuzioni.

Per imparare le probabilità di BN con variabili latenti si può usare EM:

- Nel *passo E*, tramite inferenza probabilistica, per ogni esempio, si deriva la distribuzione delle variabili latente, date quelle osservate;
- Nel *passo M*, si deriva la probabilità del modello dai dati estesi, come visto in precedenza.

Esempio — (cont.)

A	B	C	D	E	cont.
⋮	⋮	⋮	⋮	⋮	⋮
t	f	t	t	t	.71
t	f	t	t	f	.29
f	f	t	t	f	4.2
⋮	⋮	⋮	⋮	⋮	⋮
f	t	t	t	f	2.3

← E

M →

Probabilità
$P(A)$
$P(B)$
$P(E A, B)$
$P(C E)$
$P(D E)$

- **E** calcola $P(E | A, B, C, D)$ per ogni esempio;
- **M** impara le probabilità dai dati completi.

Oltre al caso delle variabili latenti, i dati potrebbero risultare *incompleti* per altre cause. Ci potrebbero essere *valori mancanti* per alcune variabili in qualche tupla. Serve cautela nel trattamento di tali esempi: la mancanza potrebbe non essere casuale ma risultare *correlata* con il fenomeno d'interesse.

Nel caso più semplice vi sono valori mancanti per variabili categoriche senza genitori da interrogare. Per poter interrogare altre variabili, la distribuzione potrebbe essere modellata aggiungendo il valore missing al dominio delle variabili con valori mancanti oppure considerando una variabile indicatrice binaria per ogni valore del dominio della variabile: con 0 a indicare un valore mancante nelle variabili indicatrici. I casi di dati mancanti non vanno ignorati.

Si supponga di conoscere le variabili della rete da costruire e che nessuna sia latente ma non si conosca la *struttura*, disponendo però di un dataset completo.

In tal caso gli approcci principali all'**apprendimento della struttura** sono i seguenti:

- Si definisce la struttura in termini di *indipendenza condizionata*: dato un *ordinamento totale* delle variabili, occorre determinare i sottoinsiemi *parents(*)* di predecessori in base a relazioni di *indipendenza condizionata* quindi occorre prima determinare il *miglior* ordinamento totale e in seguito verificare l'effettiva indipendenza, il che risulta difficile a causa della limitatezza dei dati disponibili;
- Si conduce una ricerca basata su una misura di *valutazione (score)* delle diverse reti con l'obiettivo di trovare una struttura che minimizzi l'errore, ad esempio, usando una misura basata sul criterio MAP, quindi facendo un compromesso tra adattamento ai dati e complessità.

Assumendo la disponibilità di un dataset di esempi *Es* completo, si punta a trovare un modello *m* che massimizzi:

$$P(m | Es) \propto P(Es | m) * P(m)$$

con $P(Es | m)$ calcolabile come prodotto delle probabilità per ogni esempio, ulteriormente fattorizzabili in ragione delle distribuzioni di ogni X_i dati i genitori $par(X_i, m)$ in m :

$$\begin{aligned} P(Es | m) * P(m) &= \left(\prod_{e \in Es} P(e | m) \right) * P(m) \\ &= \left(\prod_{e \in Es} \prod_{X_i} P_m^e(X_i | par(X_i, m)) \right) * P(m) \end{aligned}$$

dove $P_m^e(*)$ distribuzione relativa a e secondo m .

Passando ai logaritmi (stessi punti di massimo) si può scrivere:

$$\begin{aligned} \log P(Es | m) + \log P(m) &= \\ &= \left(\sum_{e \in Es} \sum_{X_i} \log P_m^e(X_i | par(X_i, m)) \right) + \log P(m) \\ &\stackrel{*}{=} \left(\sum_{e \in Es} \sum_{X_i} \log P_m^e(X_i | par(X_i, m)) \right) + \sum_{X_i} \log P(m(X_i)) \end{aligned}$$

$$\begin{aligned}
 &= \sum_{X_i} \left(\sum_{e \in Es} \log P_m^e(X_i \mid \text{par}(X_i, m)) \right) + \sum_{X_i} \log P(m(X_i)) \\
 &= \sum_{X_i} \left(\sum_{e \in Es} \log P_m^e(X_i \mid \text{par}(X_i, m)) + \log P(m(X_i)) \right)
 \end{aligned}$$

dove in * si assume che si possa fattorizzare $P(m) = \prod_i P(m(X_i))$ con $m(X_i)$ *modello locale* per X_i . L'obiettivo finale diventa:

$$= \sum_{X_i} \left(\sum_{e \in Es} \log P_m^e(X_i \mid \text{par}(X_i, m)) + \log P(m(X_i)) \right)$$

In cui ogni variabile può essere ottimizzata separatamente purché si mantenga la rete aciclica: va trovato un buon ordinamento totale sulle variabili tramite ricerca *locale* o *branch-and-bound*. Risultano diversi *problemi supervisionati* indipendenti da risolvere determinando per ogni variabile le distribuzioni $P_m^e(X_i \mid \text{par}(X_i, m))$ e una possibile approssimazione di $\log P(m(X_i))$, ad esempio usando il BIC.

Se la struttura non è data, alcune variabili sono latenti e vi sono dati mancanti, ci sono diversi problemi da risolvere: oltre a quello dei dati *mancanti* c'è la *complessità* computazione data dalla ricerca in uno spazio circoscritto ma troppo esteso per pensare di testare tutte le combinazioni possibili di ordinamenti e variabili latenti, anche laddove si considerino solo variabili latenti che semplifichino il modello. Lo *spazio* sarà finito ma è *vastissimo*.

Possibili *soluzioni* a tali problemi sono la selezione del miglior modello, ad esempio adottando il criterio MAP oppure effettuare predizioni *mediando* (*averaging*) su tutti i modelli (*approccio bayesiano*). Questa seconda soluzione fornisce in genere predizioni migliori ma risulta più difficile da spiegare/giustificare.

Nelle prossime pagine ci saranno degli screen di Gemini che spiega un po' l'ultimo capitolo.

"Le forme di apprendimento viste in precedenza possono essere estese con l'uso esplicito del ragionamento probabilistico: i dati forniscono l'evidenza fattuale sulla quale si può effettuare il condizionamento." $\varnothing$

Spiegazione: Qui l'autore sta creando un ponte con gli argomenti passati del corso. Gli algoritmi classici ("le forme di apprendimento viste in precedenza") spesso restituiscono risposte rigide e deterministiche (es. "questo paziente è malato" o "questo paziente è sano"). L'autore dice che possiamo "potenziare" ("estendere") questi algoritmi usando le probabilità. $\varnothing$

"L'evidenza fattuale": Nel mondo reale non abbiamo certezze assolute, ma abbiamo i **dati** che abbiamo raccolto. Questi dati sono fatti concreti, misurabili: sono la nostra "evidenza fattuale". $\varnothing$

"Effettuare il condizionamento": Questo è il cuore del pensiero bayesiano. In probabilità, "condizionare" significa *aggiornare le proprie convinzioni alla luce di nuove scoperte*. Se all'inizio pensi che piova al 50%, e poi guardi fuori (nuova evidenza) e vedi nuvole nere, "condizioni" la tua stima portandola al 90%. Nell'apprendimento automatico, il modello si "condiziona" sui dati di training per aggiornare i propri parametri interni. $\varnothing$

"Ciò è alla base dell'apprendimento automatico supervisionato e non." $\varnothing$

Spiegazione: L'autore sottolinea che questo ragionamento probabilistico non è solo un truccetto matematico per casi specifici, ma è le fondamenta (la "base") dell'intero Machine Learning moderno. Vale sia se stiamo insegnando al computer con esempi che hanno già la risposta corretta ("supervisionato", es. classificare email in spam/non-spam), sia se gli stiamo chiedendo di trovare pattern nascosti da solo ("non supervisionato", es. raggruppare i clienti in base ai loro acquisti). $\varnothing$

"Dato il dataset E_s , per determinare un modello ottimale, m , per i dati a disposizione, E_s , si può utilizzare il teorema di Bayes:" $\varnothing +1$

$$P(m|E_s) = \frac{P(E_s|m) \cdot P(m)}{P(E_s)}$$

$\varnothing +1$

Spiegazione: Formalizziamo il problema. Abbiamo i nostri dati, chiamati E_s (che sta per "Esempi" o "Evidenza"). Il nostro scopo è trovare la regola matematica perfetta (il "modello ottimale m ") che spieghi al meglio questi dati. Per trovare questo m ideale, la formula magica è il Teorema di Bayes. $\varnothing +1$

Cosa calcola la formula: Quello che c'è a sinistra dell'uguale, $P(m | E_s)$, è esattamente il nostro obiettivo. Si legge: "Qual è la probabilità che il modello m sia quello giusto, DATO CHE abbiamo osservato esattamente questo dataset E_s ?". $\varnothing +1$

to dopo, la dispensa smonta la parte destra della formula spiegando i tre "ingranaggi" che permettono di calcolare questo risultato:

"dove: $P(E_s|m)$ rappresenta la verosimiglianza o likelihood del fatto che E_s sia stato generato da m : è alta se m fornisce una buona approssimazione della distribuzione dei dati reale (sconosciuta);" $\varnothing +1$

Spiegazione (L'adattamento ai dati): Questo termine è il "test della realtà". Risponde alla domanda: "Se facciamo finta per un attimo che il modello m sia la verità assoluta, quanta probabilità ci sarebbe che generi esattamente i dati E_s che abbiamo davanti?".

Se il modello m è un'ottima imitazione della complessa realtà (la distribuzione reale che noi in fondo non conosciamo), allora la probabilità che generi dati simili a quelli veri è altissima (quindi $P(Es|m)$ è "alta"). Se il modello è sballato, questa probabilità rasenta lo zero. $\varnothing$

" $P(m)$ è la probabilità a priori che codifica il learning bias, indicando i modelli più verosimili a priori e può essere usata per la regolarizzazione inducendo a scegliere modelli più semplici attribuendo loro una maggiore probabilità;" $\varnothing$

Spiegazione (Il buon senso iniziale): Questo è il termine più affascinante. È il nostro pregiudizio (*bias*) prima ancora di guardare i dati. Noi esseri umani sappiamo che l'universo di solito segue regole semplici (il principio del "Rasoio di Occam"). $\varnothing$

Quindi, usiamo $P(m)$ per forzare matematicamente la **regolarizzazione**: assegniamo a priori una probabilità di partenza molto *alta* ai modelli matematicamente *semplici* (es. una linea retta), e una probabilità molto *bassa* ai modelli super contorti. In questo modo, il computer non sceglierà un modello assurdamamente complesso solo perché si adatta perfettamente ai dati, perché quel modello verrà penalizzato dal suo valore $P(m)$ bassissimo. $\varnothing$

" $P(Es)$, detta **funzione di partizione**, viene usata ai fini della **normalizzazione**." $\varnothing$

Spiegazione (La proporzione matematica): Il numeratore della formula ci dà un "punteggio" grezzo per il nostro modello m . Ma le probabilità, per definizione, devono essere numeri compresi tra 0 (impossibile) e 1 (certo, 100%).

Per trasformare il nostro punteggio in una percentuale valida, dobbiamo dividerlo per il totale di tutti i punteggi possibili. Questo termine totale è proprio $P(Es)$, ovvero la probabilità assoluta di ottenere quel dataset Es a prescindere dal modello. Fare questa divisione per rimettere i numeri nella scala "normale" tra 0 e 1 si chiama, appunto, **normalizzazione**. $\varnothing$

"Nella selezione di un singolo modello, il modello MAP (maximum a posteriori) è quello che massimizza la probabilità a posteriori, determinabile in base al numeratore $P(Es|m) \cdot P(m)$ essendo trascurabile il denominatore in quanto non dipende da m ." $\varnothing$

Spiegazione: Se vuoi trovare il modello m che ha la probabilità più alta in assoluto (Maximum A Posteriori), non ti serve calcolare l'intera divisione del teorema di Bayes. Poiché il denominatore $P(Es)$ è identico per *tutti* i modelli che stai valutando (è una costante rispetto a m), il modello che rende massimo il numeratore renderà massima l'intera frazione. Quindi, per trovare il MAP, ti basta massimizzare $P(Es|m) \cdot P(m)$. $\varnothing$

"In alternativa, si può puntare a determinare il modello di massima verosimiglianza (ML) massimizzando solo $P(Es|m)$ " $\varnothing$

Spiegazione: L'approccio Maximum Likelihood (ML) butta via il pregiudizio iniziale $P(m)$. Cerca solo il modello che si adatta meglio ai dati osservati. $\varnothing$

"Quando lo spazio dei modelli è molto ricco, sorge il problema di avere spesso più di un modello m per cui $P(Es|m) = 1$ [...] essi possono rappresentare qualunque funzione discreta, quindi anche sovradattarsi ai dati;" $\varnothing$

Spiegazione: Se usi modelli potentissimi (come alberi di decisione molto profondi), questi possono "imparare a memoria" il dataset. In quel caso, la probabilità che il modello generi esattamente quei dati è del 100% ($P(Es|m) = 1$). Ma questo è un male: il modello si è *sovradattato* (overfitting), imparando anche il rumore casuale. Per questo l'approccio MAP è spesso preferibile all'ML: il termine $P(m)$ penalizza questi modelli troppo complessi, anche se si adattano perfettamente. $\varnothing +2$

"Dati gli esempi di training E_s , per predire Y per un nuovo esempio x , si deve determinare la probabilità a posteriori: $P(Y|x \wedge E_s)$ " $\varnothing$

- **Spiegazione:** Vogliamo scoprire il valore del target Y per un input nuovo x , basandoci su tutta l'esperienza passata E_s . $\varnothing$

Il testo scompone questa probabilità usando un insieme esaustivo M di modelli disgiunti.

Vediamo i tre passaggi matematici: $\varnothing$

Passaggio 1: $P(Y|x \wedge E_s) = \sum_{m \in M} P(Y \wedge m|x \wedge E_s)$ $\varnothing$

- **Spiegazione (Ragionamento per casi):** Per calcolare la probabilità di Y , sommiamo le probabilità di Y calcolate per ogni singolo modello m esistente in M . È come chiedere un parere a mille esperti diversi e sommare le loro opinioni. $\varnothing$

Passaggio 2: $\sum_{m \in M} P(Y|m \wedge x \wedge E_s) \cdot P(m|x \wedge E_s)$ $\varnothing$

- **Spiegazione (Regola del prodotto):** La probabilità congiunta $P(Y \wedge m|...)$ viene spezzata. Diventa la probabilità di Y (dato il modello e i dati) moltiplicata per la probabilità che quel modello sia effettivamente quello giusto (dati i dati). $\varnothing$

Passaggio 3: $= \sum_{m \in M} P(Y|m \wedge x) \cdot P(m|E_s)$ $\varnothing$

- **Spiegazione (Le due assunzioni):** Come siamo arrivati qui? L'autore usa due semplificazioni logiche fondamentali. $\varnothing$
 1. $P(Y|m \wedge x \wedge E_s) \rightarrow P(Y|m \wedge x)$: Se conosco già le regole esatte del modello m , il vecchio dataset E_s non mi serve più a nulla per fare la predizione su x . Il modello "racchiude" già quell'esperienza. $\varnothing +1$

2. $P(m|x \wedge E_s) \rightarrow P(m|E_s)$: Osservare le feature del solo nuovo input x (senza sapere la sua Y) non cambia la mia fiducia generale su quale modello sia il migliore. $\varnothing$

"Invece di scegliere il modello migliore, nel model averaging si mediano le predizioni di tutti i modelli, ponderando con $P(m|E_s)$ " $\varnothing$

- **Spiegazione:** Questa è la formula finale. Fai fare la predizione a tutti i modelli ($P(Y|m \wedge x)$), ma il "voto" di ciascun modello conta in proporzione a quanto quel modello è probabile alla luce dei dati di addestramento ($P(m|E_s)$). $\varnothing$

"Un'assunzione comune è che gli esempi $E_s = \{e_1, \dots, e_k\}$ siano indipendenti e identicamente distribuiti (i.i.d.) dato m [...] $P(E_s|m) = \prod_{i=1}^k P(e_i|m)$ " $\varnothing +1$

- **Spiegazione:** Per calcolare la likelihood di tutto il dataset, assumiamo che gli eventi non si influenzino a vicenda (indipendenti) e seguano le stesse regole (identicamente distribuiti). Grazie a questo, la probabilità congiunta diventa una gigantesca moltiplicazione ($\prod$) delle singole probabilità degli esempi e_i . $\varnothing +1$

Pagine 3-4: L'Apprendimento dei Parametri (Puntina e Beta/Dirichlet)

Il testo applica la probabilità a un parametro numerico ϕ (la probabilità che esca Testa/Head lanciando una puntina). $\varnothing +1$

"Detta ϕ la probabilità di Head, la likelihood sarà $P(E_s|\phi) = \phi^{n_1} \cdot (1 - \phi)^{n_0}$ "

Spiegazione: Se lanci la puntina in modo indipendente, e ottieni n_1 Teste e n_0 Croci, la probabilità totale è la probabilità di Testa (ossia ϕ) moltiplicata per se stessa n_1 volte, per la probabilità di Croce (ossia $1 - \phi$) moltiplicata per se stessa n_0 volte. ϕ

"...da cui si può ottenere la log-likelihood: $\log P(Es|\phi) = n_1 \cdot \log \phi + n_0 \cdot \log(1 - \phi)$ " ϕ

Spiegazione: Moltiplicare tante piccole probabilità causa problemi ai computer (underflow). Si applica il logaritmo. Le proprietà dei logaritmi trasformano le moltiplicazioni in somme e gli esponenti in moltiplicatori. Massimizzando questa derivata si ottiene la stima ML: $\phi = \frac{n_1}{n_0 + n_1}$. $\phi + 1$

"se $n_0 = 0$ o $n_1 = 0$, una probabilità nulla per un evento resta comunque possibile: per calcolare una stima MAP serve anche una distribuzione a priori, al fine di evitare il sovradattamento." ϕ

Spiegazione: Se lanci 3 volte e ottieni 3 Teste ($n_1 = 3, n_0 = 0$), la formula di prima darebbe $\phi = 3/3 = 1$, cioè Croce è impossibile. Per evitare questo overfitting causato da pochi dati, usiamo una prior bayesiana. ϕ

"Si definisce la distribuzione Beta nel modo seguente: $Beta_{\alpha_0, \alpha_1}(\phi) = \frac{1}{Z} \phi^{\alpha_1-1} \cdot (1 - \phi)^{\alpha_0-1}$ " $\phi + 1$

Spiegazione: Trattiamo il parametro ϕ come una variabile casuale. La distribuzione Beta è perfetta per probabilità binarie. Gli α_0 e α_1 sono **pseudo-conteggi**: è come se fingessimo di aver già lanciato la moneta un po' di volte (esperienza a priori). $\phi + 1$

- Se moltiplichiamo questa prior per la likelihood dei nuovi dati, otteniamo una nuova distribuzione Beta (si dicono "conjugate" perché la forma matematica resta identica) aggiornata con i nuovi conteggi n . ϕ
- Il **valore atteso** della probabilità diventa $\frac{c_1 + n_1}{c_0 + n_0 + c_1 + n_1}$. Questa aggiunta di conteggi "fittizi" c per evitare gli zeri si chiama *smoothing di Laplace*. $\phi + 2$

"Nel caso di Y categorica, parametro con k valori, si considera la distribuzione Dirichlet, un'estensione della Beta a più dimensioni" ϕ

- **Spiegazione:** Se non hai solo due esiti (Testa/Croce) ma k esiti diversi (es. un dado a 6 facce), usi la formula estesa della Dirichlet, basata su produttorie $\prod_{j=1}^k$ invece che su due soli termini. ϕ

Pagine 5-6: Classificatori Naive Bayes (NB)

In questa sezione si costruisce il modello Naive Bayes, smontando pezzo per pezzo la probabilità a posteriori. L'obiettivo è calcolare la classe Y dati i valori di input $X_1 = v_1, \dots, X_k = v_k$. ϕ

Passaggio 1 (Teorema di Bayes): $P(Y|X_1 = v_1, \dots) = \frac{P(X_1=v_1, \dots|Y) \cdot P(Y)}{P(X_1=v_1, \dots)}$ ϕ

- **Spiegazione:** È il classico Bayes. La probabilità della classe Y date le feature X , è uguale alla probabilità di osservare quelle feature data la classe, per la prior della classe, diviso l'evidenza totale delle feature. ϕ

Passaggio 2 (Assunzione di Indipendenza Condizionata): =

$$\frac{P(X_1=v_1|Y) \cdot \dots \cdot P(X_k=v_k|Y) \cdot P(Y)}{P(X_1=v_1, \dots, X_k=v_k)}$$
 ϕ

- **Spiegazione:** Questa è la mossa "Naive" (ingenua). Il modello assume che, una volta conosciuta la classe Y , le feature X siano completamente indipendenti tra loro. Quindi, la probabilità congiunta gigante a numeratore si spezza nel prodotto delle singole probabilità $P(X_i|Y)$. $\text{Ⓢ}+1$

Passaggio 3 (Denominatore ed espansione):
$$= \frac{P(Y) \cdot \prod_{i=1}^k P(X_i=v_i|Y)}{\sum_Y P(Y) \cdot \prod_{i=1}^k P(X_i=v_i|Y)}$$
 Ⓢ

- **Spiegazione:** Il numeratore viene riscritto in modo compatto con il simbolo di produttoria ($\prod$). Il denominatore viene espanso usando la regola della probabilità totale: è semplicemente la somma del numeratore calcolato per *tutte le possibili classi* Y . Serve solo come costante di normalizzazione affinché il risultato dia 100%. $\text{Ⓢ}+2$

"Il modello NB è capace di trattare anche esempi con dati mancanti [...] facendo un'assunzione detta MAR, Missing at Random." Ⓢ

- **Spiegazione:** Poiché le X_i sono moltiplicate indipendentemente, se ti manca il valore di X_2 , semplicemente non lo inserisci nella moltiplicazione e condizionato solo su quelle che hai. La regressione logistica non può farlo. $\text{Ⓢ}+1$

Pagine 7-8: Alberi di Decisione e Lunghezza delle Descrizioni (MDL)

Infine, le dispense uniscono l'inferenza bayesiana alla teoria dell'informazione.

"Nell'apprendimento degli alberi di decisione serve un bias che tipicamente andrà a favorire alberi più piccoli [...] Più alberi potranno adattarsi perfettamente ai dati, a meno di rumore." Ⓢ

Spiegazione: Come accennato a pagina 1, un albero che ha una foglia per ogni singolo esempio del dataset avrà zero errori sui dati di training. Però predirà malissimo dati nuovi (overfitting). $\text{Ⓢ}+1$

"In base al criterio MAP, si deve massimizzare $P(Es|m) \cdot P(m)$. Passando ai logaritmi e negando, la funzione da ottimizzare (minimizzare) risulta:
 $(-\log_2 P(Es|m)) + (-\log_2 P(m))$ " Ⓢ

Spiegazione: Ecco il trucco. Se prendiamo il logaritmo in base 2 della probabilità MAP e gli mettiamo un meno davanti, stiamo traducendo il problema in bit (Teoria dell'Informazione). Invece di *massimizzare* una probabilità, vogliamo *minimizzare* la quantità di bit necessari. $\text{Ⓢ}+4$

"...con il criterio della minima lunghezza di descrizione (MDL): vanno preferiti i modelli che minimizzano la somma precedente dove, $-\log_2 P(Es|m)$ rappresenta il numero di bit per descrivere i dati Es , in base a m , e $-\log_2 P(m)$ è il numero di bit per descrivere m ." $\text{Ⓢ}+1$

Spiegazione: Questa equazione è sublime. Ti dice che il modello perfetto è quello che ti permette di zippare/comprimere i dati nel minor numero di bit possibile, calcolando sia quanto spazio occupa il file compresso (i dati dato il modello), sia quanto spazio occupa il programma per decomprimerlo (il modello stesso). $\text{Ⓢ}+1$

"Come approssimazione del MDL da minimizzare si può usare:
 $-\log_2 P(Es|m) + |m| \cdot \log_2(|Es|)$ detto indice BIC (Bayesian information criterion)." Ⓢ

Spiegazione: Poiché calcolare l'esatto numero di bit per descrivere un modello ($-\log_2 P(m)$) è teoricamente arduo, si usa una scorciatoia pratica: la formula BIC. Essa penalizza il modello proporzionalmente al numero dei suoi parametri ($|m|$) e al logaritmo del numero di esempi a disposizione ($|Es|$). $\text{Ⓢ}+2$

3 Apprendimento Non Supervisionato (Pagine 8-9)

Il testo introduce un cambio di paradigma totale: non abbiamo più le "risposte giuste" (le etichette) nei dati.

"Nell'apprendimento non supervisionato il training set non contiene la feature-target. Si cercano o costruiscono pattern nei dati." 

- **Spiegazione:** Nei problemi precedenti avevamo sempre una Y da prevedere. Qui abbiamo solo un ammasso di dati (le feature X) e chiediamo all'algoritmo di trovare da solo una struttura logica o delle regolarità nascoste. 

"Ciò significa che la feature-obiettivo può essere considerata come variabile latente (i suoi valori non sono osservati)." 

- **Spiegazione:** Il testo introduce un concetto fondamentale: la "variabile latente". Anche se non vediamo l'etichetta (es. "questo cliente è un compratore compulsivo"), assumiamo che esista un attributo nascosto (latente, appunto) che raggruppa naturalmente i dati. 

"Il metodo generale per ricostruire i valori della classificazione naturale è il clustering. Si partizionano gli esempi in cluster [...] in modo tale che ogni cluster predica i valori delle feature per gli esempi in esso contenuti." 

- **Spiegazione:** Il "clustering" è l'arte di fare gruppi. L'obiettivo è creare gruppi talmente omogenei che, se sai a quale gruppo appartiene un esempio, puoi quasi indovinare i valori delle sue feature.  +1

"Si distinguono l'hard clustering in cui ogni esempio viene assegnato a un cluster preciso [...] e il soft clustering che calcola una distribuzione di appartenenza degli esempi ai cluster" 

- **Spiegazione:** Questa è la differenza tra bianco/nero e sfumature di grigio. Nell'hard clustering un dato sta nel gruppo A al 100%. Nel soft clustering, un dato potrebbe appartenere al gruppo A al 70% e al gruppo B al 30%. 

3.1 K-MEANS (Pagine 9-12)

Il testo presenta l'algoritmo di hard clustering più famoso: il K-MEANS.

"K-MEANS è un algoritmo per l'hard clustering. Prende in input gli esempi di training e il numero di cluster k da cercare. Si assumerà che il dominio di ogni feature sia cardinale/numerico, ossia con differenze tra valori significative." 

- **Spiegazione:** Per far funzionare K-MEANS devi dirgli tu in quanti gruppi (k) dividere i dati. Inoltre, i dati devono essere numeri (es. età, peso) perché l'algoritmo calcolerà distanze matematiche tra di essi. Non funziona con feature categoriche come "colore degli occhi".  +1

"K-MEANS costruisce una funzione cluster: $E \rightarrow \{1, \dots, k\}$ tale che $cluster(e) = c$ indica che e appartiene al cluster c . Per fare questo si basa su una funzione $pred(j, c)$ che restituisce il valore di X_j per il cluster c " 

Spiegazione: L'algoritmo deve creare due cose:

1. Una funzione $cluster$ che assegna a ogni esempio e un numero da 1 a k . 
2. Una funzione $pred(j, c)$ che rappresenta il "prototipo" (o centroide) del cluster. Ci dice: "Qual è il valore medio della feature j per tutti quelli che stanno nel cluster c ?".

"L'obiettivo dell'algoritmo è quindi quello di trovare cluster e pred ottimali, ossia che minimizzino la somma di loss quadratiche:

$$\sum_{e \in E_s} \sum_{j=1}^n [\text{pred}(j, \text{cluster}(e)) - X_j(e)]^2$$

- **Spiegazione:** Questa è la formula centrale dell'intero algoritmo. Calcola l'errore totale. Per ogni esempio e e per ogni sua feature j , fa la differenza tra il valore reale $X_j(e)$ e il valore predetto dal suo cluster $\text{pred}(j, \text{cluster}(e))$. Eleva questa differenza al quadrato (per punire gli errori grandi e togliere i segni negativi) e somma tutto. Vogliamo che questo numero sia il più piccolo possibile.

I Passaggi dell'Algoritmo K-MEANS

Poiché calcolare l'assegnazione perfetta fin da subito è impossibile ("NP-hard"), l'algoritmo procede per tentativi iterativi.

"1. Per ogni feature X_j e ogni cluster c , definire $\text{pred}(j, c)$ con il valore medio di $X_j(e)$ per ogni e in c : $\text{pred}(j, c) \leftarrow \frac{\sum_{e: \text{cluster}(e)=c} X_j(e)}{|\{e: \text{cluster}(e)=c\}|}$ "

- **Spiegazione:** Aggiorna la posizione dei centroidi. Per calcolare il nuovo centro esatto del cluster c , somma tutti i valori di una feature per gli esempi attualmente in quel cluster, e dividi per il numero totale di quegli esempi (il denominatore). In pratica, fa la media aritmetica.

"2. Riassegnare ogni esempio e a un cluster c : scegliere c che minimizzi

$$\sum_{j=1}^n [\text{pred}(j, c) - X_j(e)]^2$$

- **Spiegazione:** Ora che i centri si sono spostati, ricontra ogni singolo esempio. Calcola la distanza al quadrato (distanza euclidea) tra l'esempio e tutti i centroidi. Riassegna l'esempio al cluster il cui centroide è più vicino.

L'algoritmo ripete questi due passi finché le assegnazioni diventano stabili (nessun punto cambia più gruppo).

Problemi e Soluzioni del K-MEANS (Pag. 11-12)

"L'algoritmo converge verso un minimo locale stabile... Ottenute diverse assegnazioni stabili da più esecuzioni si può scegliere il partizionamento migliore... applicazione del RANDOM RESTART"

- **Spiegazione:** K-MEANS ha un difetto: si ferma appena trova un buon raggruppamento (minimo locale), ma non è detto che sia il migliore in assoluto. La soluzione è il *Random Restart*: fai girare l'algoritmo 10, 100 volte partendo da posizioni iniziali casuali diverse, calcoli l'errore totale per ciascuna, e tieni quella che ha prodotto l'errore più basso.

"È importante notare la sensibilità dell'errore... alle diverse scale delle feature... preventivamente riscalfati"

- **Spiegazione:** Se misuri l'altezza in metri (es. 1.80) e lo stipendio in euro (es. 30000), la distanza matematica sarà dominata dallo stipendio. Bisogna sempre "standardizzare" o riscalfare i dati prima di usare K-MEANS affinché tutte le feature abbiano lo stesso peso.

"scelta del(l'iper-)parametro k ... gomito/elbow... o BIC score"

Spiegazione: Come scegliere k ? Tracci un grafico dell'errore al variare di k . L'errore scenderà sempre, ma a un certo punto formerà un "gomito" (knee): quello è il punto in cui aggiungere nuovi cluster non porta più benefici significativi, ed è il k ideale.

3.2 EXPECTATION MAXIMIZATION (EM) e Soft Clustering (Pagine 12-14)

Se K-MEANS è un approccio geometrico (basato su distanze), l'algoritmo EM è l'approccio probabilistico (basato sulle distribuzioni). È un soft clustering. $\varnothing +1$

"L'algoritmo EXPECTATION MAXIMIZATION (EM) consente di apprendere modelli probabilistici con variabili latenti... Dato Es, EM costruisce un modello NB con uno nodo genitore con la variabile latente C... e come nodi figli uno per ogni feature X_i " $\varnothing +1$

- **Spiegazione:** EM prende l'architettura del Naive Bayes vista nel capitolo precedente, ma con una differenza enorme: il nodo genitore (la Classe/Cluster C) è sconosciuto. Vogliamo calcolare sia a quale classe appartengono i dati, sia le probabilità del modello Naive Bayes contemporaneamente. $\varnothing +1$

"Rispetto a K-MEANS si può considera l'estensione (augmentation) della tabella-dati con due colonne (virtuali): una colonna C per la classe/cluster... una colonna Count per il contatore normalizzato... che insieme rappresentano la distribuzione $P(C|e)$." $\varnothing$

- **Spiegazione (Il trucco della tabella estesa):** Nel soft clustering, l'esempio non va interamente nel cluster 1 o nel cluster 2. Se abbiamo $k = 3$ cluster, l'algoritmo prende una singola riga di dati e la moltiplica in 3 righe identiche. A queste aggiunge una colonna per la classe (1, 2, 3) e una colonna "Count". Se il modello stima che l'esempio appartenga per il 40% alla classe 1, per il 10% alla classe 2 e per il 50% alla classe 3, il contatore per quelle tre righe sarà 0.4, 0.1 e 0.5. La somma dei contatori per un esempio fa sempre 1. $\varnothing +3$

I due passi dell'algoritmo EM

Come K-MEANS, anche EM itera all'infinito due fasi finché non si stabilizza: $\varnothing$

"**EXPECTATION:** aggiorna Count in base alle distribuzioni correnti del modello... il contatore associato a $\langle X_1 = v_1, \dots, X_n = v_n, C = c \rangle$ viene aggiornato con $P(C = c | X_1 = v_1, \dots, X_n = v_n)$ " $\varnothing$

- **Spiegazione (Passo E):** Fai finta che il tuo modello Naive Bayes attuale sia perfetto. Usalo per calcolare la probabilità che ogni esempio appartenga a ciascun cluster. Inserisci questa probabilità (che va da 0 a 1) nella colonna "Count" della tabella estesa. Stiamo "pesando" l'appartenenza. $\varnothing +2$

"**MAXIMIZATION:** inferisce le probabilità del modello dagli esempi estesi... in tabella: esempi completi con valori per tutte le variabili... per la stima delle probabilità del modello si considera la massimizzazione ML (o MAP)." $\varnothing +1$

- **Spiegazione (Passo M):** Ora fai finta che la tabella estesa (con i suoi pesi "Count") sia la verità assoluta. Ricalcola tutte le probabilità del modello Naive Bayes usando questi pesi frazionari, massimizzando la verosimiglianza. $\varnothing +1$

La Formula di EM (Pag. 13)

L'output dell'algoritmo (e il calcolo del passo E) è governato da questa bellissima formula:

$$P(C = c | X_1 = v_1, \dots, X_n = v_n) = \frac{P(C = c) \cdot \prod_{i=1}^n P(X_i = v_i | C = c)}{\sum_{c'} P(C = c') \cdot \prod_{i=1}^n P(X_i = v_i | C = c')}$$

Spiegazione: Questa non è altro che la formula del Naive Bayes che abbiamo analizzato a pagina 5! È il Teorema di Bayes. Il numeratore moltiplica la prior del cluster per le probabilità di osservare quelle specifiche feature in quel cluster. Il denominatore somma tutto per tutte le classi possibili per normalizzare. $\varnothing$

Le Statistiche Sufficienti (cc e fc)

Per far girare EM sui computer, non si creano fisicamente tabelle enormi, ma si tengono aggiornati dei contatori:

- $cc[c]$ (**contatori-classe**): Somma di tutti i "Count" di tutti gli esempi per la classe c . Ci dice "quanto è grande" in media questo cluster.
- $fc[i, v, c]$ (**contatori-feature**): Somma di tutti i "Count" per quegli esempi che hanno la feature i al valore v , solo per la quota assegnata al cluster c .

Al passo M, le nuove probabilità del Naive Bayes si aggiornano così:

- Prior del cluster: $P(C = c) = cc[c]/|Es|$
- Probabilità condizionata della feature: $P(X_i = v|C = c) = fc[i, v, c]/cc[c]$

"Un problema tipico (con $k > 1$) è che EM ha pressoché sempre massimi locali e globali multipli... Per trovare il massimo globale si possono effettuare più restart"

- **Spiegazione:** Esattamente come K-MEANS, l'algoritmo EM può incastrarsi in una soluzione sub-ottimale. Anche in questo caso si fa girare tutto più volte partendo da probabilità casuali diverse e si sceglie il modello che dà la likelihood complessiva più alta.

Costruire a mano una Rete Bayesiana (BN) chiedendo le probabilità a un essere umano è difficilissimo. Ecco perché cerchiamo di farla "imparare" al computer partendo da un dataset.

"L'obiettivo si riduce a imparare, per ogni X_i , la distribuzione condizionata $P(X_i|parents(X_i))$."

- **Spiegazione:** Se sappiamo già quali sono i nodi e come sono collegate le frecce (la struttura è nota), il problema diventa facilissimo. Dobbiamo solo calcolare le tabelline delle probabilità per ogni nodo, basandoci sui suoi "genitori". Diventa una serie di piccoli problemi di apprendimento supervisionato.

"dove: $P(E = t|A = t \wedge B = f) = \frac{n_t + c_t}{n_f + n_t + c_f + c_t}$ "

- **Spiegazione:** Questa formula mostra come calcolare la probabilità di un nodo (es. E è vero, t) sapendo che i suoi genitori hanno certi valori ($A = t$ e $B = f$).
- Si contano banalmente le righe nel dataset in cui succede questo (n_t) diviso tutte le righe in cui i genitori hanno quei valori ($n_t + n_f$).
- I termini c_t e c_f sono i soliti **pseudo-conteggi** per fare *smoothing*: aggiungiamo dei dati "finti" a priori per evitare di avere probabilità a zero se non abbiamo mai visto quella combinazione esatta nei dati reali.

"Un caso un po' più difficile è quello in cui il modello contenga anche variabili latenti ossia variabili i cui valori non siano non osservabili per nessun esempio."

Spiegazione: Immagina di avere sintomi e malattie, ma di non poter mai misurare direttamente un certo ormone nel sangue. Quell'ormone è una variabile "latente" (nascosta). Non c'è nessuna colonna per esso nella nostra tabella dei dati.

"Senza queste variabili nel modello aumentano i parametri per cui si hanno maggiori rischi di sovradattamento."

Spiegazione: Perché infilarci una variabile che non possiamo nemmeno misurare? Per semplificare la matematica! Senza il nodo nascosto E come "collo di bottiglia" tra A, B e C, D , dovremmo connettere tutti a tutti, passando (come dice l'esempio) da 10 a 14 parametri da imparare, aumentando il rischio di imparare il rumore (overfitting).

"Per imparare le probabilità di BN con variabili latenti si può usare EM:"

- **Spiegazione:** Quando ci mancano dei pezzi, l'algoritmo Expectation-Maximization (EM) è la soluzione.
 - "nel passo E... si deriva la distribuzione delle variabili latenti": Il computer finge di conoscere le probabilità e compila la colonna mancante E con dei "pesi" frazionari (es. probabilità 0.71 che sia vero, 0.29 che sia falso).
 - "nel passo M... si impara le probabilità dai dati completi": Usando quei pesi frazionari appena calcolati, aggiorna le tabelline delle probabilità di tutta la rete.

4.2 Dati Mancanti (Pagina 16)

"Oltre al caso delle variabili latenti, i dati potrebbero risultare incompleti per altre cause. [...] Serve cautela nel trattamento di tali esempi: la mancanza potrebbe essere non essere casuale"

- **Spiegazione:** Se un dato manca, non è detto che qualcuno si sia dimenticato di scriverlo (Missing at Random). L'autore fa l'esempio medico: se una terapia fa malissimo, i pazienti più gravi abbandonano il test. Se tu ignori i dati mancanti, penserai erroneamente che la terapia funzioni perché sono rimasti solo i pazienti sani!

$$P(F) = \text{sigmoid}(w_0 + w_1 \cdot S_t + w_2 \cdot S_f + w_3 \cdot A_t + w_4 \cdot A_f)"$$

- **Spiegazione:** Come traduciamo in matematica l'assenza di un dato? Usando delle variabili indicatrici binarie.

- Al posto di una singola variabile S (avere sorelle), ne creiamo due: S_t (vale 1 se vero) e S_f (vale 1 se falso).
- Il trucco geniale è che se il dato è mancante, *entrambe* valgono 0. In questo modo, il modello usa il termine noto (w_0 , il bias) per rappresentare specificamente l'impatto di chi "non vuole rispondere".

4.3 Apprendimento della Struttura (Pagina 17)

Qui il problema esplode: abbiamo i dati completi, ma non sappiamo come disegnare le frecce della rete (la struttura).

"si conduce una ricerca basata su una misura di valutazione (score) delle diverse reti... ad esempio usando una misura basata sul criterio MAP"

- **Spiegazione:** Poiché testare le indipendenze è difficile con pochi dati, l'approccio standard è assegnare un "punteggio" (score) a ogni possibile disegno della rete. Il punteggio migliore vince.

Seguiamo la meravigliosa scomposizione matematica per arrivare alla formula finale da ottimizzare:

$$\text{Passaggio 1: } P(Es|m) \cdot P(m) = \left(\prod_{e \in Es} P(e|m) \right) \cdot P(m)$$

- **Spiegazione:** Il punteggio MAP (da massimizzare) è Verosimiglianza $\cdot$ Prior. Assumendo dati indipendenti, la verosimiglianza di tutto il dataset si "spacca" nel prodotto ($\prod$) delle probabilità di ogni singolo esempio e .

Passaggio 2: $= (\prod_{e \in Es} \prod_{X_i} P_m^e(X_i | par(X_i, m))) \cdot P(m)$ $\varnothing$

Spiegazione: Ogni esempio e è a sua volta descritto dalla Rete Bayesiana. Quindi la sua probabilità si "spacca" ulteriormente nel prodotto di ogni singolo nodo X_i condizionato ai suoi genitori $par(X_i, m)$. $\varnothing$

Passaggio 3 (Logaritmi): $\log P(Es|m) + \log P(m) =$
 $(\sum_{e \in Es} \sum_{X_i} \log P_m^e(X_i | par(X_i, m))) + \sum_{X_i} \log P(m(X_i))$ $\varnothing$

Spiegazione: Per non impazzire coi numeri piccoli, passiamo ai logaritmi. Tutti i prodotti ($\prod$) diventano somme ($\sum$). Anche la probabilità a priori $P(m)$ viene scomposta nella somma delle prior delle singole famiglie di nodi $P(m(X_i))$. $\varnothing+2$

Passaggio 4 (Equazione finale): $\sum_{X_i} (\sum_{e \in Es} \log P_m^e(X_i | par(X_i, m))) +$
 $\log P(m(X_i))$ $\varnothing$

Spiegazione: Il tocco da maestro. L'autore inverte l'ordine delle somme portando $\sum_{X_i}$ fuori da tutto. Cosa significa all'atto pratico? Che non dobbiamo calcolare il punteggio dell'intera rete in un colpo solo! Possiamo calcolare e ottimizzare il punteggio **per ogni singolo nodo separatamente**, scegliendo i migliori genitori per quel nodo. L'unico vincolo è non creare cicli nel grafo. $\varnothing+2$

"Se la struttura non è data, alcune variabili sono latenti... e vi sono anche dati mancanti ci sono diversi problemi da risolvere... Lo spazio sarà finito ma è vastissimo." $\varnothing+1$

Spiegazione: Questa è la tempesta perfetta. Non sai come sono fatte le frecce, ti mancano delle colonne, e i dati che hai sono pure incompleti. Le combinazioni da testare per il computer diventano un numero spaventosamente grande. $\varnothing+1$

"Possibili soluzioni... sono la selezione del miglior modello, ad esempio adottando il criterio MAP oppure effettuare predizioni mediando (averaging) su tutti i modelli" $\varnothing$

Spiegazione: Come se ne esce? O ti accontenti di cercare la singola rete che si avvicina di più al picco massimo (criterio MAP), oppure fai una media probabilistica delle predizioni generate da milioni di reti diverse (Model Averaging). $\varnothing$

"Questa seconda soluzione fornisce in genere predizioni migliori ma risulta più difficile da spiegare / giustificare." E con questa frase l'autore conclude: il Model Averaging matematicamente è quasi imbattibile, ma se devi spiegare a un medico *perché* l'IA ha fatto quella diagnosi, non avrai un singolo "albero" o "rete" da fargli vedere, ma la media di migliaia di reti sovrapposte. $\varnothing$

Ovviamente queste spiegazioni non penso siano esaustive, però credo che potrebbero dare una mano a tutti quanti :)